Pipeline_3rd_draft

April 24, 2026

1 Introduction

I have always been told I am a chameleon. My friends call me the "fit-in-everywhere" person. I play sports, I enjoy hiking, I go to parties, but I also nerd out over assignments like this one. I move between very different social circles and somehow feel at home in all of them. That got me thinking: do I actually change when I move between these groups? Not what I say, but how I say it? My roommate actually sent this video to me a few weeks back: https://www.youtube.com/shorts/k7y0B4LChWc (referring to the person in the video as me).

There is a well-known concept in sociolinguistics called code-switching. The idea that people unconsciously shift their speech patterns depending on who they are talking to. We adjust our tone, pace, volume, and even pitch based on social context. Everyone does it to some extent. But I wanted to know: do I do it enough that a machine could hear the difference?

Ideally, I would have recorded myself in different social situations i.e. at dinner with my family, hanging out with friends, in a meeting. But since I do not walk around recording myself in public, I turned to the next best thing: my WhatsApp voice messages.

It turns out I have a pretty rich archive sitting in my WhatsApp chats. I talk to my mom regularly through voice notes. Long, comfortable, everyday conversations. I also send voice notes constantly to my best friend, who studies in Istanbul, so our schedules rarely overlap and voice messages have become our main way of staying in touch. And recently, I have been exchanging voice notes with a professional contact, a connection my friend introduced me to, someone who lives/work in DC. I am moving there after graduation, so I have been asking him about neighborhoods, housing websites, and what the Minerva-to-DC transition is like. Luckily, we also talked through WhatsApp voice notes, which gave me a nice third category of speech to work with.

So the dataset for this project consists of voice notes I exported from three WhatsApp conversations: Mom (mainly in Urdu), Best Friend (mainly in Urdu), and Professional (Urdu - English mix)

All files were exported directly from WhatsApp in .opus format. The total dataset is small (only 32 files) but voice notes tend to be long (30 seconds to several minutes each), so I will segment them into fixed-length clips to build a usable dataset.

The hypothesis is: I unconsciously code-switch across social contexts, annd a machine learning model can detect it from my voice alone. Not from what I say, but from how I say it. To test this, I will extract Mel-Frequency Cepstral Coefficients (MFCCs) from my voice notes and train two classifiers: a Multinomial Softmax Regression model built from scratch, and a Convolutional Neural Network built using Keras. If the models can classify which social context a clip belongs to with reasonable accuracy, that is evidence that my voice genuinely shifts across these three settings.

One thing worth acknowledging upfront is that the three categories were recorded in different physical environments at different times. This means background noise and room acoustics could act as a confounding variable, and the model might learn to classify rooms rather than speech patterns. I will address this in the sections below, but it is an honest limitation of working with naturalistic data.

2 Setup

2.1 Import Libraries

Before doing anything with the audio files, I need to set up the tools. Here is what each library does in this pipeline:

- **librosa**: the main audio analysis library. It handles loading audio files, extracting features like MFCCs, and computing spectrograms. It is the standard library for audio ML in Python.
- **numpy**: numerical computing. Everything from arrays to matrix operations to basic statistics.
- **pandas**: for organizing extracted features into dataframes, which makes it easier to inspect and manipulate the data before feeding it to models.
- **matplotlib**: for all the plots and visualizations throughout this notebook.
- **scikit-learn**: provides utilities like train/test splitting, stratified sampling, cross validation, and evaluation metrics (confusion matrix, classification report, ROC curves).
- **tensorflow.keras**: for building and training the 1D Convolutional Neural Network.
- **soundfile**: used here to write the audio clips out as WAV files during segmentation.
- **subprocess**: a built-in Python module used here to call FFmpeg from within the notebook, which handles the audio format conversion from `.opus` to `.wav`.
- **os / glob**: built-in modules for navigating the file system, listing directories, and finding files by pattern.

2.2 Runtime Setup

Before importing the heavier libraries, I set a few environment variables that make the notebook more stable to rerun from a fresh kernel. The main goal is reproducibility. I also point Matplotlib at a writable cache directory, because this machine was creating temporary font caches during import and that slows everything down.

[2]:
```
import os
import glob
import subprocess

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import librosa
import librosa.display
import soundfile as sf
```

```
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, auc,
    ConfusionMatrixDisplay
)
from sklearn.preprocessing import label_binarize, StandardScaler

import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from tensorflow.keras.optimizers import Adam

np.random.seed(42)
tf.random.set_seed(42)

plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.dpi'] = 120

BASE_DIR = os.getcwd()
CATEGORIES = ['Shazil - Mom', 'Shazil - Bestfriend', 'Shazil - Professional']
LABEL_MAP = {'Shazil - Mom': 0, 'Shazil - Bestfriend': 1, 'Shazil -␣
 ↪Professional': 2}
LABEL_NAMES = ['Mom', 'Bestfriend', 'Professional']

print("Libraries loaded successfully.")
print(f"Base directory: {BASE_DIR}")
for cat in CATEGORIES:
    n_files = len([f for f in os.listdir(os.path.join(BASE_DIR, cat)) if f.
 ↪endswith('.opus')])
    print(f"  {cat}: {n_files} opus files")
```

```
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports
OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'.
See: https://github.com/urllib3/urllib3/issues/3020
  warnings.warn(

Libraries loaded successfully.
Base directory: /Users/shazilfarukh/Desktop/CS156_Assignment_1
  Shazil - Mom: 17 opus files
  Shazil - Bestfriend: 12 opus files
  Shazil - Professional: 13 opus files
```

3 Part 1: Building and Understanding the Voice Archive

All the voice notes are also uploaded to Google drive folder that can be accessed here: https://drive.google.com/drive/folders/1lrJt5SmJG_11V1Nu3ur9CyTaJ13czgTm?usp=sharing

3.1 Convert `.opus` to `.wav`

WhatsApp exports voice notes in the `.opus` format. Opus is a lossy, compressed audio codec. It is great for keeping file sizes small on a phone, but it is not ideal for machine learning. There are two main reasons to convert to `.wav` before doing any analysis:

1. WAV files store audio as raw Pulse-Code Modulation (PCM) data. Each sample is a direct numerical measurement of the sound wave's amplitude at a specific point in time. There is no compression, no encoding artifacts, and no information loss. This means when I load a WAV file into a numpy array, I am working with the actual signal, not a reconstruction of it.
2. WAV files have a fixed sampling rate and channel configuration baked into the header. Opus files, on the other hand, use variable bitrate encoding, which means the amount of data per second fluctuates depending on the complexity of the audio. This variability can cause issues with libraries like `librosa` that expect uniform frame-by-frame processing.

I use FFmpeg to handle the conversion. FFmpeg decodes each `.opus` file and re-encodes it as a 16-bit PCM WAV file at 16kHz mono. The converted files are saved into a new `wav_files/` directory, organized by category.

The choice of 16kHz and mono is deliberate and I will explain it in detail in the next section when I discuss resampling and feature extraction.

[3]:
```python
# Create the output directory for WAV files
WAV_DIR = os.path.join(BASE_DIR, 'wav_files')
os.makedirs(WAV_DIR, exist_ok=True)

# Full path to ffmpeg (installed via Homebrew)
FFMPEG_PATH = '/opt/homebrew/bin/ffmpeg'

def convert_opus_to_wav(input_dir, output_dir, sr=16000):
    """
    Convert all .opus files in input_dir to 16kHz mono WAV files in output_dir.
    Uses FFmpeg for decoding and re-encoding.

    Parameters:
        input_dir  - path to folder containing .opus files
        output_dir - path to folder where .wav files will be saved
        sr         - target sampling rate in Hz (default 16000)
    """
    os.makedirs(output_dir, exist_ok=True)
    opus_files = sorted(glob.glob(os.path.join(input_dir, '*.opus')))

    for opus_path in opus_files:
        filename = os.path.splitext(os.path.basename(opus_path))[0]
        wav_path = os.path.join(output_dir, filename + '.wav')

        # wee skip it if already converted
        if os.path.exists(wav_path):
            continue
```

```
        # FFmpeg command: decode opus, output 16kHz mono 16-bit PCM WAV
        cmd = [
            FFMPEG_PATH, '-i', opus_path,
            '-ar', str(sr),       # set sampling rate to 16kHz
            '-ac', '1',           # convert to mono (single channel)
            '-sample_fmt', 's16', # 16-bit signed integer samples
            wav_path,
            '-y'                  # overwrite if exists
        ]

        result = subprocess.run(cmd, capture_output=True, text=True)
        if result.returncode != 0:
            print(f"Error converting {opus_path}: {result.stderr[:200]}")

    n_wav = len(glob.glob(os.path.join(output_dir, '*.wav')))
    print(f"  {output_dir}: {n_wav} WAV files ready")

# Convert each category
for cat in CATEGORIES:
    input_path = os.path.join(BASE_DIR, cat)
    output_path = os.path.join(WAV_DIR, cat)
    print(f"Converting {cat}...")
    convert_opus_to_wav(input_path, output_path)

print("\nAll conversions complete.")
```

```
Converting Shazil - Mom…
  /Users/shazilfarukh/Desktop/CS156_Assignment_1/wav_files/Shazil - Mom: 17 WAV
files ready
Converting Shazil - Bestfriend…
  /Users/shazilfarukh/Desktop/CS156_Assignment_1/wav_files/Shazil - Bestfriend:
12 WAV files ready
Converting Shazil - Professional…
  /Users/shazilfarukh/Desktop/CS156_Assignment_1/wav_files/Shazil -
Professional: 13 WAV files ready

All conversions complete.
```

3.2 Resampling to 16kHz Mono

In the previous step, I already told FFmpeg to output 16kHz mono WAV files. Let me explain the reasoning behind both of those choices.

Starting with mono: WhatsApp records voice notes in mono anyway (there is only one microphone on a phone), but even if the files had stereo channels, I would collapse them down. Stereo exists to give the listener a sense of left-right spatial positioning, which has nothing to do with how someone speaks. I only care about the acoustic content of my voice (pitch, loudness, rhythm) and all of that lives in a single channel. Keeping stereo would double the data with zero added value.

The sampling rate of 16kHz is a bit more interesting. The original WhatsApp recordings were encoded at 48kHz, which means the audio was sampled 48,000 times per second. That is the same rate used in professional video production. But for speech analysis, it is overkill. The reason comes from a foundational result in signal processing, the Nyquist-Shannon Sampling Theorem, which states:

$$f_s \geq 2 \times f_{\max}$$

In plain terms, the sampling rate f_s needs to be at least double the highest frequency we want to capture. Flip it around, and any given sampling rate can faithfully represent frequencies up to $\frac{f_s}{2}$ (the Nyquist frequency). At 16kHz, that ceiling is 8kHz.

To put this in perspective, here is roughly where different parts of human speech live on the frequency spectrum:

Component	Frequency Range
Fundamental pitch (adult male)	85 - 180 Hz
Vowel formants (F1, F2, F3)	300 - 3,500 Hz
Consonants and fricatives (“s”, “sh”)	2,000 - 8,000 Hz
Anything above 8kHz	Mostly air / noise

Everything I need fits comfortably below 8kHz, so 16kHz sampling gives full coverage. Going higher (say, 44.1kHz CD quality or 48kHz) would just mean larger arrays, slower feature extraction, and no additional useful information. This is also the standard rate used in most speech processing systems, from Google’s speech recognition API to telephony codecs like G.722 (ITU-T Recommendation G.722).

[4]:
```
# Verify the format of the converted WAV files
print("Verifying WAV file properties:\n")

for cat in CATEGORIES:
    wav_dir = os.path.join(WAV_DIR, cat)
    wav_files = sorted(glob.glob(os.path.join(wav_dir, '*.wav')))

    # Pick the first file from each category as a sample
    sample_path = wav_files[0]
    y, sr = librosa.load(sample_path, sr=None)  # sr=None keeps the original␣
 ↪sample rate
    duration = len(y) / sr

    print(f"{cat}:")
    print(f"  Sample file: {os.path.basename(sample_path)}")
    print(f"  Sample rate: {sr} Hz")
    print(f"  Channels: {'mono' if y.ndim == 1 else 'stereo'}")
    print(f"  Duration: {duration:.2f} seconds")
    print(f"  Samples: {len(y):,}")
```

```
    print()
```

```
Verifying WAV file properties:

Shazil - Mom:
  Sample file: 00000022-AUDIO-2025-11-29.wav
  Sample rate: 16000 Hz
  Channels: mono
  Duration: 13.31 seconds
  Samples: 213,016

Shazil - Bestfriend:
  Sample file: 00000057-AUDIO-2025-12-19.wav
  Sample rate: 16000 Hz
  Channels: mono
  Duration: 32.27 seconds
  Samples: 516,376

Shazil - Professional:
  Sample file: 00000067-AUDIO-2026-01-04.wav
  Sample rate: 16000 Hz
  Channels: mono
  Duration: 26.33 seconds
  Samples: 421,336
```

3.3 Segment Voice Notes into Fixed-Length Clips

I have 42 raw voice notes total, which is not enough to train any model reliably. But each voice note is anywhere from 10 seconds to over a minute long, which means there is actually a lot of audio sitting inside those files. The standard approach in audio ML is to slice longer recordings into fixed-length, non-overlapping segments, a technique called fixed-length windowing. Each segment becomes its own sample.

My initial plan was to use 5-second clips, since shorter windows would give me more samples to train on. But after reading through this walkthrough on how MFCCs are computed, I realized that 5 seconds might be too short for what I am trying to detect. MFCCs are extracted from small overlapping frames (typically 25ms each), and each frame captures a tiny snapshot of the vocal tract at one instant. The actual speech pattern, the rhythm, the pacing, the shifts in energy, only becomes visible when I look at how those frames evolve over a longer time window. Five seconds might capture a single sentence or even just a fragment, but 10 seconds is long enough to hear how I sustain a conversational tone.

So I settled on 10-second non-overlapping segments. If a voice note is 47 seconds long, that gives me 4 clips and a 7-second remainder that gets discarded. I would rather throw away a short tail than pad it with silence, because zero-padding would inject fake "quiet" frames into the MFCC matrix and distort the features.

After slicing, each clip is saved as a separate WAV file inside a `clips/` directory, organized by category.

[5]:
```
CLIP_DIR = os.path.join(BASE_DIR, 'clips')
CLIP_DURATION = 10   # seconds
SR = 16000            # sampling rate
CLIP_SAMPLES = CLIP_DURATION * SR  # 10 * 16000 = 160,000 samples per clip

def slice_audio_files(input_dir, output_dir, clip_samples=CLIP_SAMPLES, sr=SR):
    """
    Slice all WAV files in input_dir into fixed-length clips.
    Clips shorter than clip_samples are discarded.

    Parameters:
        input_dir    - folder containing full-length WAV files
        output_dir   - folder to save the sliced clips
        clip_samples - number of samples per clip (default: 160000 = 10 seconds␣
 ↪at 16kHz)
        sr           - sampling rate (default: 16000)

    Returns:
        count - number of clips created
    """
    os.makedirs(output_dir, exist_ok=True)
    wav_files = sorted(glob.glob(os.path.join(input_dir, '*.wav')))
    count = 0

    for wav_path in wav_files:
        # Load the full audio file at the target sample rate
        y, _ = librosa.load(wav_path, sr=sr)

        # Calculate how many full clips fit in this file
        n_clips = len(y) // clip_samples

        base_name = os.path.splitext(os.path.basename(wav_path))[0]

        for i in range(n_clips):
            start = i * clip_samples
            end = start + clip_samples
            clip = y[start:end]

            # Save each clip with a numbered suffix
            clip_filename = f"{base_name}_clip{i:02d}.wav"
            clip_path = os.path.join(output_dir, clip_filename)
            sf.write(clip_path, clip, sr)
            count += 1

    return count

# Slice each category and report the results
```

```python
clip_counts = {}
for cat in CATEGORIES:
    input_path = os.path.join(WAV_DIR, cat)
    output_path = os.path.join(CLIP_DIR, cat)
    clip_counts[cat] = slice_audio_files(input_path, output_path)

summary_df = (
    pd.DataFrame(
        {
            'Category': [LABEL_NAMES[LABEL_MAP[c]] for c in CATEGORIES],
            'Clips': [clip_counts[c] for c in CATEGORIES],
        }
    )
    .sort_values('Category')
    .reset_index(drop=True)
)

total = int(summary_df['Clips'].sum())

print(f"Clips created (window = {CLIP_DURATION}s, sr = {SR}Hz):")
print(summary_df.to_string(index=False))
print(f"\nTotal dataset size: {total} clips")
```

```
Clips created (window = 10s, sr = 16000Hz):
    Category  Clips
  Bestfriend     62
         Mom     54
Professional     50

Total dataset size: 166 clips
```

3.4 Data Augmentation

One of the main limitations of the dataset is its size. I only have 166 labeled clips from 42 voice notes, which is enough to study the problem seriously but still small enough that a model could start memorizing recording-specific quirks instead of general vocal patterns. To make the training set richer, I augment the audio after the train and test split. That way, the evaluation stays honest because the held-out clips are always untouched.

I use four waveform-level augmentations plus SpecAugment on the MFCC representation. Together they simulate small shifts in recording conditions, volume, pitch, and speaking pace without changing the social context label. The goal is not to create fake new conversations. The goal is to make the models less brittle and to see which architectures can actually benefit from the extra variation.

Augmentation	What it does	Why it helps
White noise injection	Adds Gaussian noise at a controlled SNR	Makes the model less dependent on background hiss or room tone

Augmentation	What it does	Why it helps
Gain scaling	Randomly changes the overall volume	Reduces sensitivity to microphone distance and speaking loudness
Pitch shifting	Moves pitch up or down by a small number of semitones	Encourages robustness to day-to-day pitch drift
Time stretching	Slightly speeds up or slows down the clip	Simulates natural variation in speaking pace
SpecAugment	Masks random time and frequency regions in the MFCC matrix	Encourages the CNN to rely on broader patterns instead of one narrow cue

For noise injection, I target a signal-to-noise ratio:

$$\text{SNR} = 10 \log_{10} \frac{P_{\text{signal}}}{P_{\text{noise}}} \quad \Rightarrow \quad \sigma_{\text{noise}} = \sqrt{\frac{P_{\text{signal}}}{10^{\text{SNR}/10}}}$$

Pitch shifting by n semitones follows:

$$f' = f \cdot 2^{n/12}$$

I keep this augmentation branch because it became an important part of the final baseline. The later comparison sections show that augmentation helps the CNN much more than the tabular models, which is itself a useful result.

[6]:
```
# Augmentation function definitions
# These are applied only after the train/test split.

def augment_white_noise(y, snr_db=20):
    """Add Gaussian noise at a given SNR (dB)."""
    p_signal = np.mean(y ** 2)
    sigma    = np.sqrt(p_signal / (10 ** (snr_db / 10)))
    return y + np.random.normal(0, sigma, size=y.shape)

def augment_gain(y, gain_range=(0.8, 1.2)):
    """Randomly scale the overall volume."""
    gain = np.random.uniform(*gain_range)
    return y * gain

def augment_pitch_shift(y, sr, n_steps_range=(-2, 2)):
    """Randomly shift pitch by ±2 semitones."""
    n_steps = np.random.uniform(*n_steps_range)
    return librosa.effects.pitch_shift(y, sr=sr, n_steps=n_steps)

def augment_time_stretch(y, rate_range=(0.9, 1.1)):
    """Randomly speed up or slow down by ±10%."""
```

```
    rate  = np.random.uniform(*rate_range)
    y_str = librosa.effects.time_stretch(y, rate=rate)
    target = len(y)
    return y_str[:target] if len(y_str) > target else np.pad(y_str, (0, target␣
 ↪- len(y_str)))

def spec_augment(mfcc, freq_mask_param=4, time_mask_param=20, num_masks=2):
    """Randomly zero out time and frequency regions of the MFCC matrix."""
    mfcc_aug  = mfcc.copy()
    n_mels, T = mfcc_aug.shape
    for _ in range(num_masks):
        f0 = np.random.randint(0, max(1, n_mels - freq_mask_param))
        f  = np.random.randint(0, freq_mask_param + 1)
        mfcc_aug[f0:f0 + f, :] = 0.0
        t0 = np.random.randint(0, max(1, T - time_mask_param))
        t  = np.random.randint(0, time_mask_param + 1)
        mfcc_aug[:, t0:t0 + t] = 0.0
    return mfcc_aug

AUGMENT_FUNCS = [
    ('noise',   lambda y: augment_white_noise(y, snr_db=20)),
    ('gain',    lambda y: augment_gain(y)),
    ('pitch',   lambda y: augment_pitch_shift(y, SR)),
    ('stretch', lambda y: augment_time_stretch(y)),
]

print("Augmentation functions defined. They will be applied after the group␣
 ↪split.")
```

```
Augmentation functions defined. They will be applied after the group split.
```

3.5 Exploratory Data Analysis

3.6 Descriptive Statistics

Before I get into feature extraction or model building, I thought its is worth looking at the raw numbers first. How long are the original voice notes? How many clips did each category produce after the 10-second segmentation? Is the dataset balanced or skewed?

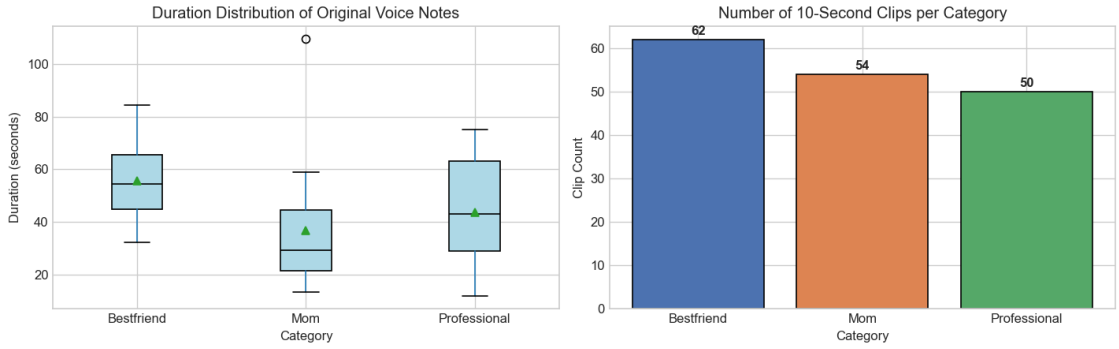
These are basic questions, but the answers directly affect how I build and evaluate the models. If one category has way more clips than the others, the model might just learn to guess that category all the time and still get a decent accuracy. Getting a clear picture now helps me plan ahead.

[7]:
```
# Compute duration statistics for the original (unsliced) voice notes
rows = []
for cat in CATEGORIES:
    wav_dir = os.path.join(WAV_DIR, cat)
    label = LABEL_NAMES[LABEL_MAP[cat]]
```

```
    for fpath in sorted(glob.glob(os.path.join(wav_dir, '*.wav'))):
        y, sr_file = librosa.load(fpath, sr=None)
        rows.append({
            'Category': label,
            'File': os.path.basename(fpath),
            'Duration (s)': round(len(y) / sr_file, 2)
        })

dur_df = pd.DataFrame(rows)

# Summary table: count, mean, min, max duration per category
stats = (
    dur_df.groupby('Category')['Duration (s)']
    .agg(['count', 'mean', 'min', 'max'])
    .rename(columns={'count': 'Voice Notes', 'mean': 'Mean (s)', 'min': 'Min␣
 ↪(s)', 'max': 'Max (s)'})
    .round(2)
)

print("Original voice note duration statistics:\n")
print(stats.to_string())
print(f"\nTotal original files: {len(dur_df)}")

# Clip counts per category (already computed in Section 2.4)
print("\n\nClip counts after 10-second segmentation:\n")
print(summary_df.to_string(index=False))

# Visualize the class distribution
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Left: duration distribution (box plot)
colors = ['#4C72B0', '#DD8452', '#55A868']
dur_df.boxplot(column='Duration (s)', by='Category', ax=axes[0],
               patch_artist=True, showmeans=True,
               boxprops=dict(facecolor='lightblue'),
               medianprops=dict(color='black'))
axes[0].set_title('Duration Distribution of Original Voice Notes')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Duration (seconds)')
axes[0].get_figure().suptitle('')  # remove auto-title from boxplot

# Right: clip count bar chart
bars = axes[1].bar(summary_df['Category'], summary_df['Clips'], color=colors,␣
 ↪edgecolor='black')
axes[1].set_title('Number of 10-Second Clips per Category')
axes[1].set_xlabel('Category')
axes[1].set_ylabel('Clip Count')
```

```
for bar in bars:
    axes[1].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.5,
                 str(int(bar.get_height())), ha='center', va='bottom',␣
 ↪fontweight='bold')

plt.tight_layout()
plt.show()
```

```
Original voice note duration statistics:

              Voice Notes  Mean (s)  Min (s)  Max (s)
Category
Bestfriend             12     55.45    32.27    84.51
Mom                    17     36.61    13.31   109.51
Professional           13     43.70    11.75    75.21

Total original files: 42


Clip counts after 10-second segmentation:

    Category  Clips
  Bestfriend     62
         Mom     54
Professional     50
```



Looking at the table and plots above, a few things are clear.

The 42 original voice notes vary a lot in length. Bestfriend has 12 files with the longest mean duration at 55.45 seconds, which makes sense because I ramble a lot in those conversations. Mom's 17 files are shorter on average at 36.61 seconds, and Professional sits in the middle with 13 files averaging 43.70 seconds. The box plot on the left shows this spread visually, and the Mom category has a clear outlier above 100 seconds.

After slicing into 10-second segments, the clip counts came out to 62 for Bestfriend, 54 for Mom, and 50 for Professional, giving 166 total clips. This is a mild class imbalance, but not severe enough

to require oversampling and/or SMOTE.

ps. SMOTE is a data preprocessing method used to handle imbalanced classification datasets by creating synthetic examples of the minority class rather than just duplicating existing ones.

[8]:
```
# Extract recording date ranges from the WAV filenames
import re

print("Recording date ranges per category:\n")
date_ranges = {}
for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    wav_dir = os.path.join(WAV_DIR, cat)
    dates = []
    for fname in sorted(os.listdir(wav_dir)):
        match = re.search(r'(\d{4}-\d{2}-\d{2})', fname)
        if match:
            dates.append(match.group(1))

    if dates:
        date_ranges[label] = (min(dates), max(dates))
        print(f"  {label:15s}  {min(dates)}  to  {max(dates)}  ({len(dates)}␣
 ↪files)")

print(f"\n  Overall span:    {min(d[0] for d in date_ranges.values())}  to ␣
 ↪{max(d[1] for d in date_ranges.values())}")
```

```
Recording date ranges per category:

  Mom              2025-11-29  to  2026-02-11  (17 files)
  Bestfriend       2025-12-19  to  2026-01-27  (12 files)
  Professional     2026-01-04  to  2026-01-12  (13 files)

  Overall span:    2025-11-29  to  2026-02-11
```

The dates above show that the data spans about two and a half months, from late November 2025 to mid-February 2026. Mom conversations go the furthest back (starting November 29), Bestfriend starts a few weeks later on December 19, and Professional only begins in early January 2026. This reflects when these conversations naturally happened in my life, not a deliberate sampling strategy.

For this assignment I only scraped voice notes from this window. I could keep collecting going forward to increase the dataset size, which might improve model performance and make the results more robust. That could be something to explore in a second iteration of this project.

3.7 Waveform Comparison

The simplest way to look at audio is to plot the raw waveform, which is just amplitude over time. This will not reveal anything about frequency content (that comes later with pitch and spectral analysis), but it does show patterns in loudness, pauses, and overall energy.

For each category, I pick a sample clip and use it throughout the Data Analysis section. I print the exact file path and provide an audio player so the same clip I am analyzing can be listened to directly.

[9]:
```
from IPython.display import Audio, display

# Load one sample clip per category for visual comparison.
# So I listened to a few clips from different positions and noticed the earlier␣
 ↪ones
# in some categories had noticeable background noise (TV, street sounds), so I
# went with index 30 which sounded clean across all three categories.
SAMPLE_IDX = 30
samples = {}
sample_paths = {}

for cat in CATEGORIES:
    clip_dir = os.path.join(CLIP_DIR, cat)
    clip_list = sorted(glob.glob(os.path.join(clip_dir, '*.wav')))
    sample_clip = clip_list[SAMPLE_IDX]
    y, _ = librosa.load(sample_clip, sr=SR)
    label = LABEL_NAMES[LABEL_MAP[cat]]
    samples[label] = y
    sample_paths[label] = sample_clip

# Print which clips are being used
print(f"Sample clips used throughout the Data Analysis section (index␣
 ↪{SAMPLE_IDX} per category):\n")
for label, path in sample_paths.items():
    print(f"  {label}: {os.path.basename(path)}")

# Provide audio playback for each sample
print("\nListen to each sample clip:\n")
for label in LABEL_NAMES:
    print(f"{label}:")
    display(Audio(sample_paths[label], rate=SR))

# Plot waveforms side by side
fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True, sharey=True)
colors = ['#4C72B0', '#DD8452', '#55A868']

for i, label in enumerate(LABEL_NAMES):
    waveform = samples[label]
    time_axis = np.linspace(0, len(waveform) / SR, len(waveform))
    axes[i].plot(time_axis, waveform, color=colors[i], linewidth=0.5)
    axes[i].set_ylabel('Amplitude')
    axes[i].set_title(f'{label}')
    axes[i].set_ylim(-1, 1)
```

```
axes[-1].set_xlabel('Time (seconds)')
fig.suptitle('Waveform Comparison (One Sample Clip per Category)', fontsize=14,␣
  ↪y=1.01)
plt.tight_layout()
plt.show()
```

```
Sample clips used throughout the Data Analysis section (index 30 per category):

  Mom: 00000031-AUDIO-2025-12-15_clip00.wav
  Bestfriend: 00000063-AUDIO-2025-12-20_clip01.wav
  Professional: 00000073-AUDIO-2026-01-07_clip05.wav

Listen to each sample clip:

Mom:

<IPython.lib.display.Audio object>

Bestfriend:

<IPython.lib.display.Audio object>

Professional:

<IPython.lib.display.Audio object>
```

Even from a single clip per category, the waveforms look noticeably different.

The Mom clip has several distinct speech bursts separated by clear silent gaps (amplitude drops to near zero around 1.5 seconds and again near 3.5 seconds). There is a loud peak near the 7-second mark that spikes close to 0.9 in amplitude. The Bestfriend clip is denser and louder overall, with peaks regularly reaching 0.8 and the speech filling most of the 10-second window. There is a brief pause around 3.5 seconds but it is much shorter than Mom's pauses. The Professional clip looks similar to Bestfriend in continuity but with slightly lower amplitude overall, mostly staying below 0.5 with a few peaks reaching 0.8.

These are just single-clip impressions though. The next several sections will compute aggregate statistics across all 166 clips to see whether these patterns hold at the dataset level.

3.8 Pitch (Fundamental Frequency) Analysis

Pitch, or fundamental frequency (F0), captures how high or low the voice sounds. It is one of the most commonly studied features in speech analysis because it directly reflects vocal effort, emotion, and social register. I use librosa's `pyin` algorithm to estimate F0, which handles the common problem of octave errors better than simple autocorrelation methods.

The original YIN algorithm (de Cheveigné & Kawahara, 2002) starts from the observation that a periodic signal should look similar to a shifted copy of itself. It formalizes this with the difference function:

$$d(\tau) = \sum_{j=1}^{W} \left(x[j] - x[j+\tau]\right)^2$$

where $x[j]$ is the audio sample at position j, τ is the candidate period in samples, and W is the integration window length. If the signal is truly periodic with period τ^*, then $d(\tau^*) \approx 0$ because the signal nearly repeats itself. To avoid the trivial solution at $\tau = 0$, YIN normalizes using the cumulative mean normalized difference function (CMNDF):

$$d'(\tau) = \begin{cases} 1 & \tau = 0 \\ \dfrac{d(\tau)}{\frac{1}{\tau}\sum_{j=1}^{\tau} d(j)} & \tau > 0 \end{cases}$$

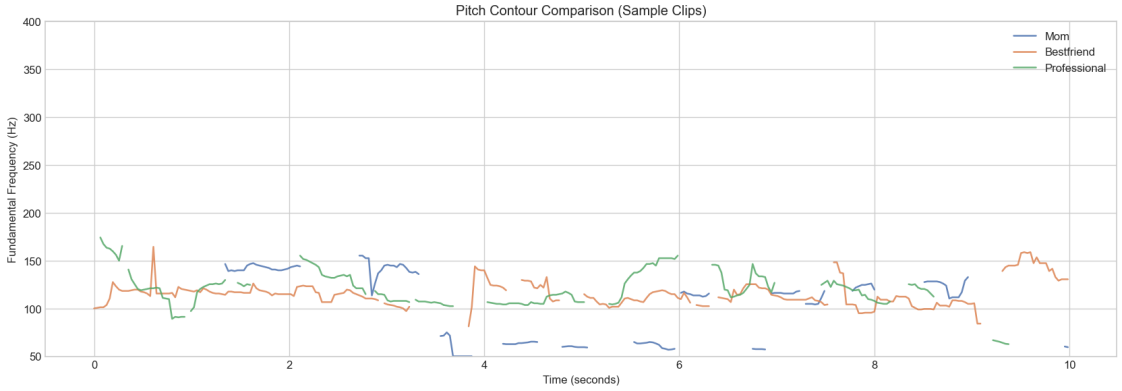
The fundamental frequency is then $F_0 = f_s/\hat{\tau}$, where $\hat{\tau}$ is the first lag where $d'(\tau)$ dips below a threshold (typically 0.1) and $f_s = 16000$ Hz is the sample rate. The `pyin` extension (Mauch & Dixon, 2014) wraps this in a Hidden Markov Model that adds probabilistic smoothing across frames, which is what makes it resistant to octave-jumping errors common in speech.

First I plot the pitch contour of the same sample clip from each category (the ones printed above in Section 3.2). Then I compute aggregate pitch statistics across all clips per category to see how consistent the patterns are.

[10]:
```
# Extract and plot pitch contours for the sample clips
fig, ax = plt.subplots(figsize=(14, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']
```

```
for i, label in enumerate(LABEL_NAMES):
    waveform = samples[label]
    # pyin returns: f0 (pitch), voiced_flag, voiced_probabilities
    f0, voiced_flag, _ = librosa.pyin(
        waveform, fmin=50, fmax=400, sr=SR,
        frame_length=2048, hop_length=512
    )

    # Create a time axis for the pitch frames
    times = librosa.frames_to_time(np.arange(len(f0)), sr=SR, hop_length=512)

    # Replace unvoiced frames with NaN so they don't appear on the plot
    f0_voiced = np.where(voiced_flag, f0, np.nan)

    ax.plot(times, f0_voiced, color=colors[i], label=label, linewidth=1.5,␣
 ↪alpha=0.85)

ax.set_xlabel('Time (seconds)')
ax.set_ylabel('Fundamental Frequency (Hz)')
ax.set_title('Pitch Contour Comparison (Sample Clips)')
ax.legend()
ax.set_ylim(50, 400)
plt.tight_layout()
plt.show()

# Compute pitch summary statistics across ALL clips (not just the sample)
print("\nPitch statistics across all clips (voiced frames only):\n")
pitch_rows = []
for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)
    all_f0 = []

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
        f0, voiced, _ = librosa.pyin(y_clip, fmin=50, fmax=400, sr=SR,
                                     frame_length=2048, hop_length=512)
        voiced_f0 = f0[voiced]  # keep only voiced frames
        all_f0.extend(voiced_f0)

    all_f0 = np.array(all_f0)
    pitch_rows.append({
        'Category': label,
        'Mean Pitch (Hz)': round(np.nanmean(all_f0), 1),
        'Std Pitch (Hz)': round(np.nanstd(all_f0), 1),
        'Min Pitch (Hz)': round(np.nanmin(all_f0), 1),
        'Max Pitch (Hz)': round(np.nanmax(all_f0), 1),
```

```
    })

pitch_df = pd.DataFrame(pitch_rows)
print(pitch_df.to_string(index=False))
```




```
Pitch statistics across all clips (voiced frames only):

    Category  Mean Pitch (Hz)  Std Pitch (Hz)  Min Pitch (Hz)  Max Pitch (Hz)
         Mom            105.6            29.4            50.0           390.9
  Bestfriend            123.2            24.8            50.0           400.0
Professional            115.5            19.0            50.0           221.9
```

The pitch contour plot shows all three categories moving between roughly 100 and 170 Hz for most of the clip, but with clear differences. The Mom contour (blue) has the widest range, jumping from about 140 Hz down to 60 Hz around the 4-second mark and bouncing back up. The Bestfriend contour (orange) is the most stable, mostly hovering between 100 and 140 Hz with only modest jumps. The Professional contour (green) starts with a peak near 170 Hz then settles into the 110 to 150 Hz range.

The aggregate statistics across all 166 clips confirm the pattern. Bestfriend has the highest mean pitch at 123.2 Hz and a standard deviation of 24.8 Hz, meaning my voice is both higher and more variable when talking to my friend. Mom has the lowest mean pitch at 105.6 Hz, but the highest standard deviation at 29.4 Hz, which suggests those conversations swing between quiet low tones and more emotional peaks. Professional has a mean pitch of 115.5 Hz with the tightest standard deviation of 19.0 Hz, consistent with a more controlled, steady delivery.

The gap between Mom (105.6 Hz) and Bestfriend (123.2 Hz) is about 17.6 Hz, which is a meaningful difference in speech analysis. Professional sits in between at 115.5 Hz. These differences should give the classifier some signal to work with.

3.9 RMS Energy Analysis

RMS (Root Mean Square) energy measures the average loudness of the audio signal. A higher RMS value means the speaker is talking louder. This is a straightforward feature, but it is useful

because people naturally adjust their volume depending on who they are speaking to.

The math. RMS energy is computed frame by frame. For frame t with N samples starting at offset $t \cdot H$:

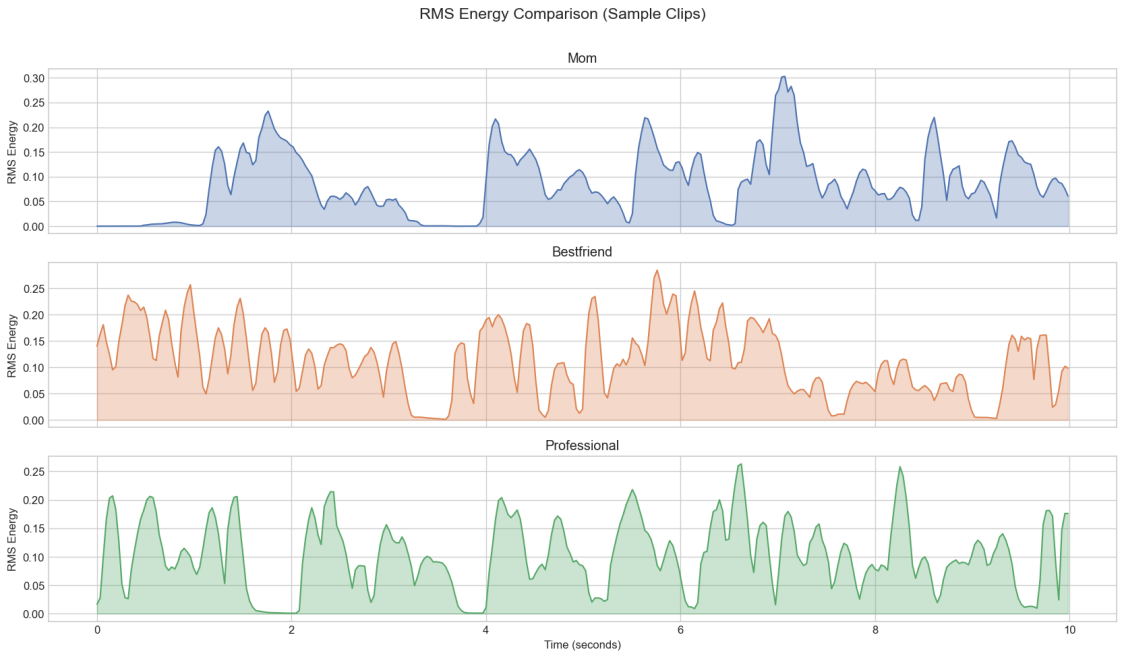
$$\text{RMS}(t) = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x[t \cdot H + n]^2}$$

where $N = 2048$ is the frame length and $H = 512$ is the hop length (so consecutive frames overlap by 75%). At 16 kHz this gives a frame of 128 ms stepping forward every 32 ms. For each clip I then summarize by taking the **mean** of $\text{RMS}(t)$ across all frames, which gives a single number representing the overall loudness of that 10-second clip. I also use the RMS trace directly when computing the silence ratio (any frame where $\text{RMS}(t) < 0.01$ is flagged as silent).

Like the pitch analysis, I first plot the RMS energy over time for the same sample clips, then compute aggregate statistics across all clips per category.

[11]:
```
# Plot RMS energy over time for the sample clips
fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True)
colors = ['#4C72B0', '#DD8452', '#55A868']

for i, label in enumerate(LABEL_NAMES):
    waveform = samples[label]
    # Compute frame-level RMS energy
    rms = librosa.feature.rms(y=waveform, frame_length=2048, hop_length=512)[0]
    times = librosa.frames_to_time(np.arange(len(rms)), sr=SR, hop_length=512)

    axes[i].plot(times, rms, color=colors[i], linewidth=1.2)
    axes[i].fill_between(times, rms, alpha=0.3, color=colors[i])
    axes[i].set_ylabel('RMS Energy')
    axes[i].set_title(f'{label}')

axes[-1].set_xlabel('Time (seconds)')
fig.suptitle('RMS Energy Comparison (Sample Clips)', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()

# Compute RMS statistics across ALL clips
print("\nRMS energy statistics across all clips:\n")
rms_rows = []
for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)
    all_rms = []

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
```

```
        rms_vals = librosa.feature.rms(y=y_clip, frame_length=2048,␣
↪hop_length=512)[0]
        all_rms.extend(rms_vals)

    all_rms = np.array(all_rms)
    rms_rows.append({
        'Category': label,
        'Mean RMS': round(np.mean(all_rms), 4),
        'Std RMS': round(np.std(all_rms), 4),
        'Max RMS': round(np.max(all_rms), 4),
    })

rms_df = pd.DataFrame(rms_rows)
print(rms_df.to_string(index=False))
```




```
RMS energy statistics across all clips:

    Category  Mean RMS  Std RMS  Max RMS
         Mom    0.0812   0.0698   0.6114
  Bestfriend    0.1172   0.0800   0.5596
Professional    0.0943   0.0675   0.5556
```

The RMS plots show clear differences in energy distribution. The Mom sample clip has a few strong peaks (hitting about 0.23 around the 2-second mark and 0.30 near the 7-second mark) but also drops to near zero between speech bursts, reflecting pauses. The Bestfriend sample clip shows

more continuous energy across the full 10 seconds, with multiple spikes reaching above 0.25. The Professional sample clip has moderate energy throughout, with peaks around 0.20 to 0.26 and fewer deep dips than Mom.

The aggregate statistics across all 166 clips confirm this. Bestfriend has the highest mean RMS at 0.1172, followed by Professional at 0.0943, and Mom at 0.0812. So on average, I speak about 44% louder with my best friend than with my mom (0.1172 vs 0.0812). The standard deviations are all fairly high relative to the means (0.0675 to 0.0800), which tells me there is a lot of variation in loudness even within a single category, probably because some clips catch me mid-sentence and others catch pauses.

One thing to keep in mind is that RMS is sensitive to recording conditions. If one set of voice notes was recorded closer to the microphone, the RMS values would be inflated regardless of how I was actually speaking. This is one reason why MFCCs will be the primary features for the model, since they capture the shape of the frequency spectrum rather than the raw loudness.

3.10 Spectral Centroid Distribution

The spectral centroid is the "center of mass" of the frequency spectrum. A higher spectral centroid means more energy is concentrated in the higher frequencies, which makes the voice sound brighter or sharper. A lower spectral centroid sounds darker or more muffled. This feature captures something different from pitch: two people can speak at the same pitch but have different spectral centroids if one has a brighter, more articulated voice quality.

For each frame t, the spectral centroid is the amplitude-weighted average frequency:

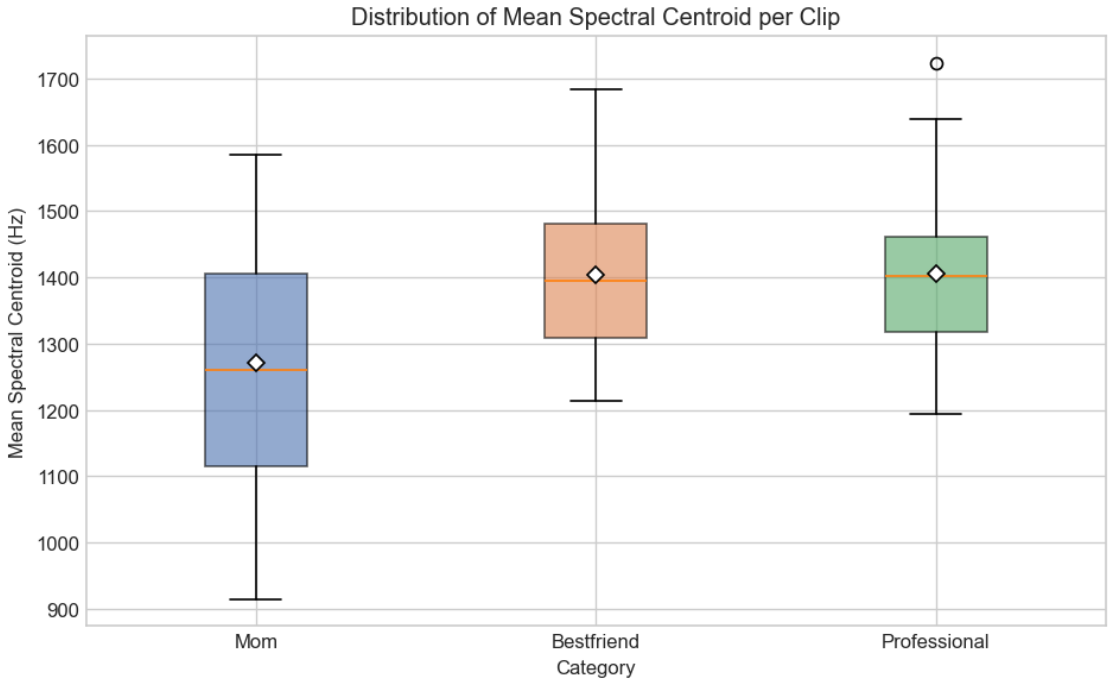
$$C(t) = \frac{\sum_{k=0}^{N/2} f_k \cdot |X_t[k]|}{\sum_{k=0}^{N/2} |X_t[k]|}$$

where $X_t[k]$ is the STFT magnitude at frequency bin k, and $f_k = k \cdot \frac{f_s}{N}$ converts bin index to Hz (with $f_s = 16000$ Hz and $N = 2048$ samples, each bin is 7.8 Hz wide). The denominator normalizes by total spectral energy, so $C(t)$ reflects the weighted center rather than just the absolute energy level. A clip full of fricatives ("s", "sh", "f") will have a high centroid because those consonants push energy toward the upper frequencies, while a clip full of long open vowels will sit lower.

For each clip, I compute the mean spectral centroid across all frames, then visualize the distributions per category using box plots and a summary table.

[12]:
```
# Compute mean spectral centroid per clip, then boxplot by category
centroid_data = []

for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
```

```
        y_clip, _ = librosa.load(fpath, sr=SR)
        cent = librosa.feature.spectral_centroid(y=y_clip, sr=SR,␣
 ↪hop_length=512)[0]
        centroid_data.append({
            'Category': label,
            'Mean Spectral Centroid (Hz)': np.mean(cent)
        })

centroid_df = pd.DataFrame(centroid_data)

# Boxplot
fig, ax = plt.subplots(figsize=(8, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']
bp_data = [centroid_df[centroid_df['Category'] == lbl]['Mean Spectral Centroid␣
 ↪(Hz)'].values
           for lbl in LABEL_NAMES]

bp = ax.boxplot(bp_data, tick_labels=LABEL_NAMES, patch_artist=True,␣
 ↪showmeans=True,
                meanprops=dict(marker='D', markerfacecolor='white',␣
 ↪markeredgecolor='black'))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)

ax.set_ylabel('Mean Spectral Centroid (Hz)')
ax.set_xlabel('Category')
ax.set_title('Distribution of Mean Spectral Centroid per Clip')
plt.tight_layout()
plt.show()

# Print summary statistics
print("\nSpectral centroid statistics per category:\n")
sc_stats = centroid_df.groupby('Category')['Mean Spectral Centroid (Hz)'].agg(
    ['mean', 'std', 'min', 'max']
).round(1)
sc_stats.columns = ['Mean (Hz)', 'Std (Hz)', 'Min (Hz)', 'Max (Hz)']
print(sc_stats.to_string())
```




```
Spectral centroid statistics per category:

              Mean (Hz)  Std (Hz)  Min (Hz)  Max (Hz)
Category
Bestfriend       1404.7     117.7    1213.0    1683.5
Mom              1270.2     166.0     914.3    1584.5
Professional     1404.8     115.2    1193.8    1723.9
```

The boxplot and statistics table reveal some nice differences. Mom has the lowest mean spectral centroid at 1270.2 Hz, which is noticeably below Bestfriend (1404.7 Hz) and Professional (1404.8 Hz). That 134.5 Hz gap means my voice sounds darker and less articulated when talking to my mom compared to the other two categories. Bestfriend and Professional are almost identical in their means, so spectral centroid alone will not help the model separate those two.

The standard deviations tell a different story though. Mom has the widest spread at 166.0 Hz, while Bestfriend (117.7 Hz) and Professional (115.2 Hz) are tighter. So not only does my speech sound duller on average when talking to my mom, but it also varies more from clip to clip. This makes sense because those conversations range from quiet everyday topics to more animated discussions.

Looking at the boxplot visually, the Mom box sits lower and stretches wider than the other two, with a minimum reaching down to 914.3 Hz. Bestfriend and Professional overlap almost completely. This means the spectral centroid is most useful for identifying Mom clips, but the model will need other features to distinguish Bestfriend from Professional.

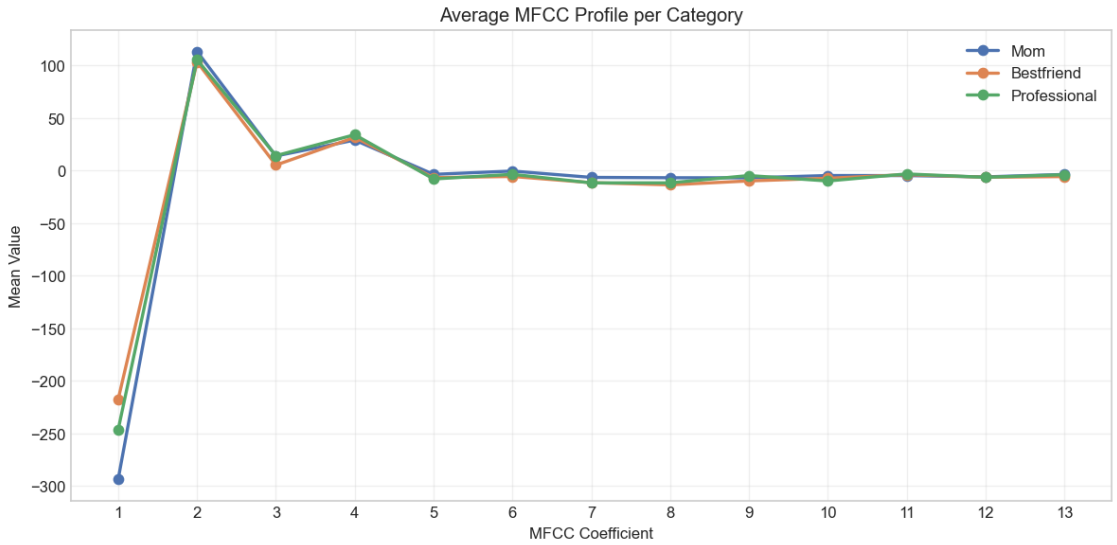
3.11 Average MFCC Profile per Category

MFCCs (Mel-Frequency Cepstral Coefficients) are the features I will actually feed into the models, so I want to visualize them before training. Each MFCC coefficient captures a different aspect of the spectral shape of the audio. Lower-numbered coefficients represent the overall spectral envelope (broad tonal quality), while higher-numbered ones capture finer spectral details.

I compute 13 MFCCs per frame for every clip, average across all frames within a clip to get a single 13-dimensional vector per clip, then average those vectors within each category. The result is one "average MFCC profile" per category. I show this as both a line plot and a heatmap.

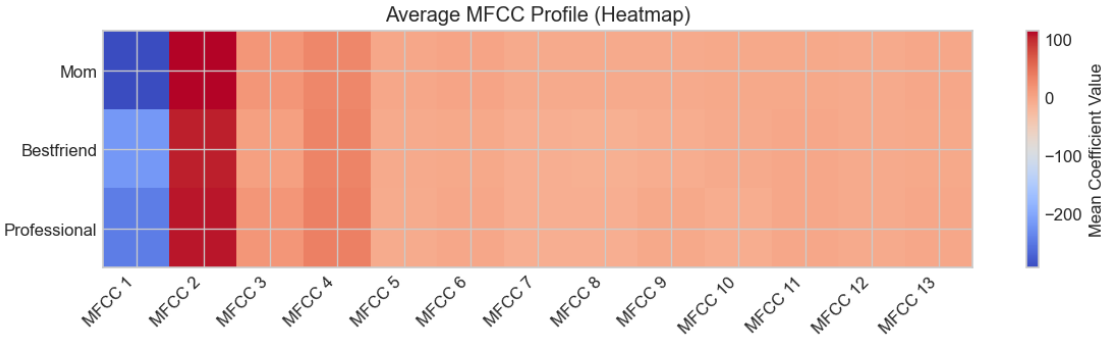
[13]:
```
# Compute average MFCC profile per category
N_MFCC = 13
mfcc_profiles = {}

for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)
    clip_means = []

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
        mfcc = librosa.feature.mfcc(y=y_clip, sr=SR, n_mfcc=N_MFCC,␣
↪hop_length=512)
        clip_means.append(np.mean(mfcc, axis=1))  # average across time -> (13,)

    mfcc_profiles[label] = np.mean(clip_means, axis=0)  # average across clips␣
↪-> (13,)

# Line plot comparing average MFCC profiles
fig, ax = plt.subplots(figsize=(10, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']

for i, label in enumerate(LABEL_NAMES):
    ax.plot(range(1, N_MFCC + 1), mfcc_profiles[label],
            marker='o', color=colors[i], label=label, linewidth=2)

ax.set_xlabel('MFCC Coefficient')
ax.set_ylabel('Mean Value')
ax.set_title('Average MFCC Profile per Category')
ax.set_xticks(range(1, N_MFCC + 1))
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# Print the actual MFCC values for reference
print("\nAverage MFCC values per category:\n")
```

```
mfcc_table = pd.DataFrame(mfcc_profiles, index=[f'MFCC {i+1}' for i in␣
 ↪range(N_MFCC)])
print(mfcc_table.round(2).to_string())

# Heatmap of the same data for a second perspective
fig, ax = plt.subplots(figsize=(10, 3))
profile_matrix = np.array([mfcc_profiles[lbl] for lbl in LABEL_NAMES])

im = ax.imshow(profile_matrix, aspect='auto', cmap='coolwarm',␣
 ↪interpolation='nearest')
ax.set_yticks(range(len(LABEL_NAMES)))
ax.set_yticklabels(LABEL_NAMES)
ax.set_xticks(range(N_MFCC))
ax.set_xticklabels([f'MFCC {i+1}' for i in range(N_MFCC)], rotation=45,␣
 ↪ha='right')
ax.set_title('Average MFCC Profile (Heatmap)')
plt.colorbar(im, ax=ax, label='Mean Coefficient Value')
plt.tight_layout()
plt.show()
```




```
Average MFCC values per category:

                Mom  Bestfriend  Professional
MFCC 1  -293.200012 -217.279999   -246.059998
MFCC 2   113.279999  103.410004    105.489998
MFCC 3    13.920000    5.280000     14.200000
MFCC 4    29.030001   31.620001     34.139999
MFCC 5    -3.550000   -6.670000     -7.930000
```

MFCC 6	-0.400000	-5.550000	-3.500000
MFCC 7	-6.420000	-11.460000	-11.650000
MFCC 8	-6.770000	-13.520000	-11.630000
MFCC 9	-6.850000	-9.780000	-4.660000
MFCC 10	-4.650000	-7.270000	-9.850000
MFCC 11	-4.720000	-3.800000	-3.130000
MFCC 12	-5.940000	-6.310000	-6.370000
MFCC 13	-3.640000	-5.660000	-3.830000



The line plot and table show where the three categories diverge in their MFCC profiles.

MFCC 1 has the biggest separation by far. Mom sits at -293.2, well below Bestfriend (-217.3) and Professional (-246.1). Since MFCC 1 represents the overall energy level of the signal, this is consistent with the RMS finding that Mom clips are the quietest. MFCC 2 shows the opposite pattern: Mom is the highest at 113.3, with Bestfriend at 103.4 and Professional at 105.5. MFCC 2 relates to the spectral slope, so the higher value for Mom suggests a steeper rolloff in high-frequency content, again matching the lower spectral centroid.

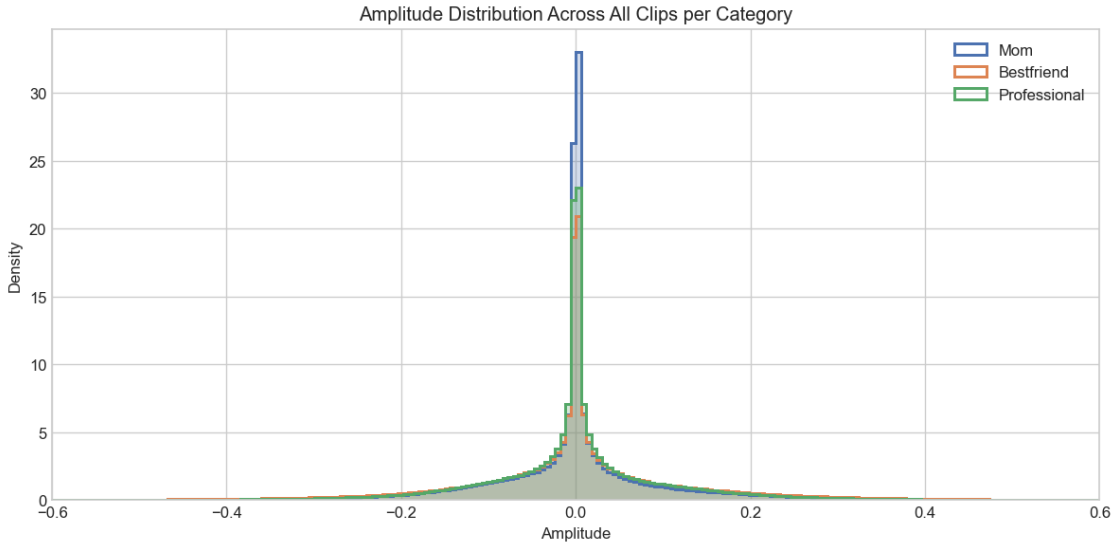
From MFCC 3 onward, the three lines in the plot converge and mostly overlap, staying within a narrow band between -15 and +35. There are still some small differences though. For example, at MFCC 3 Bestfriend is notably lower at 5.3 while Mom (13.9) and Professional (14.2) are similar. At MFCC 7 and 8, Bestfriend and Professional cluster together around -11 to -13 while Mom is higher at -6.4 and -6.8.

In the heatmap, the color difference at MFCC 1 is very clear: Mom's cell is deep blue (most negative) while Bestfriend is a lighter blue. The rest of the heatmap is mostly uniform warm tones, which shows that the higher-order coefficients do not differ dramatically. Still, the strong separation in the first two coefficients alone gives the classifier meaningful signal to work with.

3.12 Amplitude Distribution

While the waveform plots in Section 3.2 showed the raw signal for one sample clip, looking at the amplitude distribution across all clips per category gives a more complete picture. By pooling every audio sample from every clip within each category and plotting histograms, I can see how much of my speech in each context is loud vs. quiet vs. silent. The shape and spread of these distributions will complement the RMS analysis from Section 3.4.

[14]:
```
# Amplitude distribution histograms
fig, ax = plt.subplots(figsize=(10, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']

for i, cat in enumerate(CATEGORIES):
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)
    all_samples = []

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
        all_samples.extend(y_clip)

    # Use 'step' histtype so outlines don't obscure each other
    ax.hist(all_samples, bins=200, alpha=0.25, color=colors[i],
            label=f'{label} (filled)', density=True, range=(-0.6, 0.6))
    ax.hist(all_samples, bins=200, histtype='step', color=colors[i],
            linewidth=1.8, density=True, range=(-0.6, 0.6), label=f'{label}␣
 ↪(outline)')

ax.set_xlabel('Amplitude')
ax.set_ylabel('Density')
ax.set_title('Amplitude Distribution Across All Clips per Category')

# Clean up legend to show one entry per category
handles, labels = ax.get_legend_handles_labels()
# Keep only the outline entries for a cleaner legend
ax.legend(handles[1::2], [l.replace(' (outline)', '') for l in labels[1::2]])

ax.set_xlim(-0.6, 0.6)
plt.tight_layout()
plt.show()
```



All three distributions are roughly Laplacian in shape, with a sharp peak at zero and symmetric tails. This is typical for speech signals since a lot of audio samples fall near zero during pauses and breaths.

The outlines make the differences easier to spot. Mom (blue) has the tallest and narrowest peak, shooting up past a density of 35 at zero, meaning the largest proportion of its samples are near-silent. Bestfriend (orange) peaks around 20 but has noticeably wider tails extending further toward -0.5 and +0.5, meaning more high-amplitude samples. I am apparently incapable of using an indoor voice with my best friend. Professional (green) peaks around 23 and sits between the two in tail width.

These distributional differences reinforce what the RMS analysis already showed. Mom speech has more silence and lower amplitude overall, Bestfriend speech is louder and more continuous, and Professional sits in the middle.

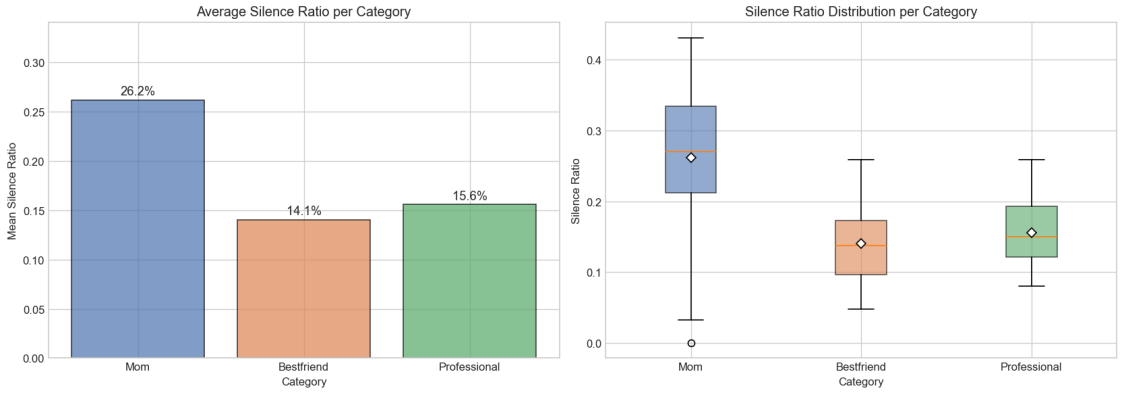
3.13 Silence Ratio

The silence ratio measures how much of each clip is "silent," defined as frames where the RMS energy falls below a threshold. This is a simple feature, but it directly captures pause behavior. I thought about doing this after the analysis I have just done before where mom clips have breaks/silences so thought why not check for it thoroughly. I define a frame as silent if its RMS energy is below 0.01. I chose this threshold based on visual inspection of the RMS distributions in Section 3.4: the Mom clips consistently showed frame-level RMS values dropping below 0.01 during pauses, while active speech frames stayed above it. For each clip, I compute the fraction of frames that are silent, then aggregate by category.

[15]:
```
# Compute silence ratio per clip
SILENCE_THRESHOLD = 0.01  # RMS below this = "silent"
silence_data = []
```

```
for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
        rms_vals = librosa.feature.rms(y=y_clip, frame_length=2048,␣
 ↪hop_length=512)[0]
        silence_ratio = np.mean(rms_vals < SILENCE_THRESHOLD)
        silence_data.append({'Category': label, 'Silence Ratio': silence_ratio})

silence_df = pd.DataFrame(silence_data)

# Bar chart of mean silence ratio + boxplot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']

# Left: mean silence ratio bar chart
mean_silence = silence_df.groupby('Category')['Silence Ratio'].mean().
 ↪reindex(LABEL_NAMES)
bars = axes[0].bar(LABEL_NAMES, mean_silence.values, color=colors, alpha=0.7,␣
 ↪edgecolor='black')
for bar, val in zip(bars, mean_silence.values):
    axes[0].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.005,
                 f'{val:.1%}', ha='center', fontsize=11)
axes[0].set_ylabel('Mean Silence Ratio')
axes[0].set_xlabel('Category')
axes[0].set_title('Average Silence Ratio per Category')
axes[0].set_ylim(0, max(mean_silence.values) * 1.3)

# Right: boxplot of silence ratios across clips
bp_data = [silence_df[silence_df['Category'] == lbl]['Silence Ratio'].values
           for lbl in LABEL_NAMES]
bp = axes[1].boxplot(bp_data, tick_labels=LABEL_NAMES, patch_artist=True,␣
 ↪showmeans=True,
                     meanprops=dict(marker='D', markerfacecolor='white',␣
 ↪markeredgecolor='black'))
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)
axes[1].set_ylabel('Silence Ratio')
axes[1].set_xlabel('Category')
axes[1].set_title('Silence Ratio Distribution per Category')

plt.tight_layout()
plt.show()
```

```python
# Print summary
print("\nSilence ratio statistics (threshold = RMS < 0.01):\n")
sr_stats = silence_df.groupby('Category')['Silence Ratio'].agg(
    ['mean', 'std', 'min', 'max']
).round(3)
sr_stats.columns = ['Mean', 'Std', 'Min', 'Max']
print(sr_stats.to_string())
```




```
Silence ratio statistics (threshold = RMS < 0.01):

              Mean    Std    Min    Max
Category
Bestfriend   0.141  0.051  0.048  0.259
Mom          0.262  0.098  0.000  0.431
Professional 0.156  0.045  0.080  0.259
```

The silence ratio turns out to be one of the strongest separating features in the whole Data Analysis section. Mom clips are 26.2% silent on average, which is nearly double the Bestfriend rate of 14.1%. Professional sits at 15.6%, close to Bestfriend.

The bar chart on the left makes this gap obvious. The Mom bar towers over the other two. The boxplot on the right shows the within-category spread: Mom's box ranges from about 0.22 to 0.33, with a whisker reaching up to 0.43 and an outlier down at 0.0 (probably a clip that was all speech with no pauses or a clip with extremely loud background noise). Bestfriend is the most consistent with a standard deviation of just 0.051, and most clips cluster tightly around that 14% mark. Professional (std = 0.045) is similarly tight.

This makes intuitive sense. I think I pause more when talking to my mom, whether because I am thinking about what to say next or just letting the conversation breathe. With my best friend, I talk almost continuously. I will include silence ratio as an extra feature alongside MFCCs when building the models, since it clearly carries discriminative information.

3.14 Speech Rhythm Analysis (Onset Density and Zero-Crossing Rate)

The features explored so far capture what my voice sounds like (pitch, loudness, spectral shape) and how much I pause (silence ratio). But they do not directly measure how fast I am speaking or how rhythmically I am delivering words. Two features can get at this: onset density and zero-crossing rate (ZCR).

Zero-crossing rate (the math). ZCR counts how many times the waveform flips sign within a frame. For a frame of N samples:

$$\mathrm{ZCR}(t) = \frac{1}{2(N-1)} \sum_{n=1}^{N-1} |\,\mathrm{sgn}(x[n]) - \mathrm{sgn}(x[n-1])\,|$$

where $\mathrm{sgn}(\cdot)$ returns $+1$ for positive values and -1 for negative ones. Each sign change contributes 2 to the sum (since $|+1 - -1| = 2$), and dividing by $2(N-1)$ keeps ZCR in $[0, 1]$. Voiced vowel sounds are nearly periodic and cross zero relatively infrequently, while unvoiced fricatives like "s" or "sh" look almost like noise and have very high ZCR.

Onset detection (the math). librosa detects onsets using the **spectral flux**, which measures how much the spectrum suddenly increases from one frame to the next:

$$\mathrm{SF}(t) = \sum_{k} H(\,|X_t[k]| - |X_{t-1}[k]|\,)$$

where $H(x) = \max(0, x)$ is the half-wave rectifier that keeps only increases in spectral energy (not decreases). A sudden spike in spectral flux signals the start of a new syllable or word. The **onset density** I report is the number of detected onsets divided by clip duration in seconds, giving a rough proxy for speaking rate in events per second.

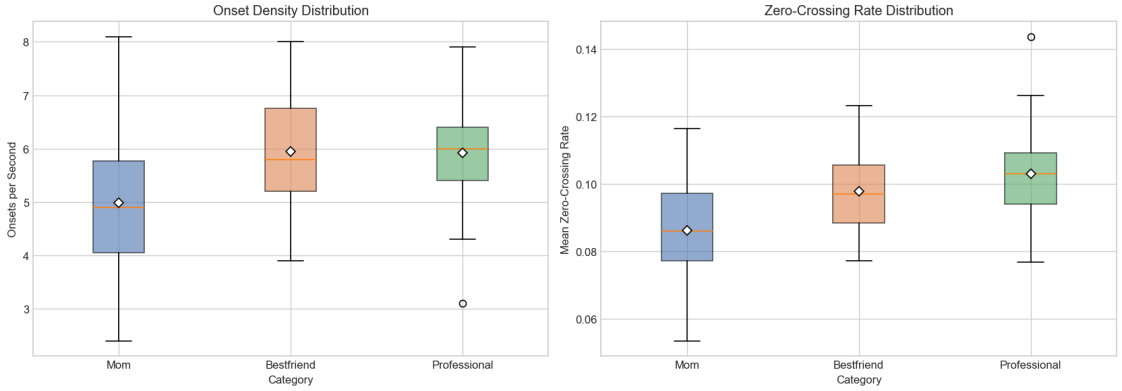
I compute both metrics per clip and visualize the distributions per category.

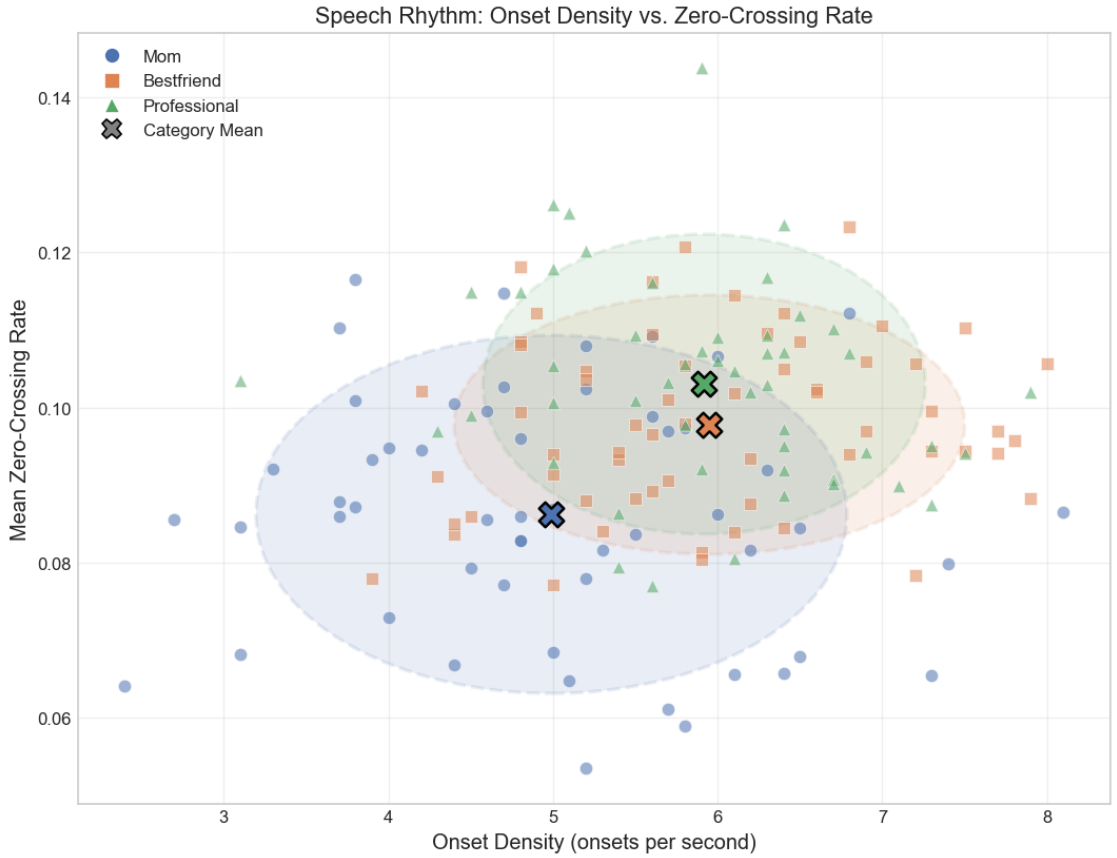
[16]:
```
# Compute onset density and zero-crossing rate per clip
rhythm_data = []

for cat in CATEGORIES:
    label = LABEL_NAMES[LABEL_MAP[cat]]
    clip_dir = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)

        # Onset detection: number of detected onsets per second
        onsets = librosa.onset.onset_detect(y=y_clip, sr=SR, hop_length=512)
        onset_density = len(onsets) / CLIP_DURATION  # onsets per second

        # Zero-crossing rate: mean across all frames
        zcr = librosa.feature.zero_crossing_rate(y_clip, frame_length=2048,␣
 ↪hop_length=512)[0]
        mean_zcr = np.mean(zcr)
```

```
        rhythm_data.append({
            'Category': label,
            'Onset Density (per sec)': onset_density,
            'Mean ZCR': mean_zcr
        })

rhythm_df = pd.DataFrame(rhythm_data)

# Plot 1: Onset density boxplot + ZCR boxplot
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = ['#4C72B0', '#DD8452', '#55A868']

# Left: onset density boxplot
onset_data = [rhythm_df[rhythm_df['Category'] == lbl]['Onset Density (per␣
 ↪sec)'].values
              for lbl in LABEL_NAMES]
bp1 = axes[0].boxplot(onset_data, tick_labels=LABEL_NAMES, patch_artist=True,␣
 ↪showmeans=True,
                      meanprops=dict(marker='D', markerfacecolor='white',␣
 ↪markeredgecolor='black'))
for patch, color in zip(bp1['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)
axes[0].set_ylabel('Onsets per Second')
axes[0].set_xlabel('Category')
axes[0].set_title('Onset Density Distribution')

# Right: ZCR boxplot
zcr_data = [rhythm_df[rhythm_df['Category'] == lbl]['Mean ZCR'].values
            for lbl in LABEL_NAMES]
bp2 = axes[1].boxplot(zcr_data, tick_labels=LABEL_NAMES, patch_artist=True,␣
 ↪showmeans=True,
                      meanprops=dict(marker='D', markerfacecolor='white',␣
 ↪markeredgecolor='black'))
for patch, color in zip(bp2['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)
axes[1].set_ylabel('Mean Zero-Crossing Rate')
axes[1].set_xlabel('Category')
axes[1].set_title('Zero-Crossing Rate Distribution')

plt.tight_layout()
plt.show()

# Plot 2: Enhanced scatter plot with confidence ellipses and mean markers
from matplotlib.patches import Ellipse
```

```
from matplotlib.lines import Line2D

fig, ax = plt.subplots(figsize=(9, 7))
markers = ['o', 's', '^']  # different marker shapes per category

for i, label in enumerate(LABEL_NAMES):
    subset = rhythm_df[rhythm_df['Category'] == label]
    x_vals = subset['Onset Density (per sec)'].values
    y_vals = subset['Mean ZCR'].values

    # Scatter points with distinct markers
    ax.scatter(x_vals, y_vals,
               color=colors[i], label=label, alpha=0.55, s=55,
               marker=markers[i], edgecolors='white', linewidth=0.6, zorder=3)

    # Plot mean as a large X marker
    mean_x, mean_y = np.mean(x_vals), np.mean(y_vals)
    ax.scatter(mean_x, mean_y, color=colors[i], marker='X', s=200,
               edgecolors='black', linewidth=1.5, zorder=5)

    # Draw a confidence ellipse using standard deviations along each axis
    # (axis-aligned ellipse avoids the covariance scaling issue)
    std_x, std_y = np.std(x_vals), np.std(y_vals)
    ellipse = Ellipse(xy=(mean_x, mean_y),
                      width=2 * 1.5 * std_x,   # 1.5 std radius
                      height=2 * 1.5 * std_y,
                      angle=0,
                      facecolor=colors[i], alpha=0.12,
                      edgecolor=colors[i], linewidth=2, linestyle='--',␣
 ↪zorder=2)
    ax.add_patch(ellipse)

ax.set_xlabel('Onset Density (onsets per second)', fontsize=12)
ax.set_ylabel('Mean Zero-Crossing Rate', fontsize=12)
ax.set_title('Speech Rhythm: Onset Density vs. Zero-Crossing Rate', fontsize=13)

# Custom legend
legend_elements = [Line2D([0], [0], marker=markers[i], color='w',␣
 ↪markerfacecolor=colors[i],
                          markersize=9, label=label) for i, label in␣
 ↪enumerate(LABEL_NAMES)]
legend_elements.append(Line2D([0], [0], marker='X', color='w',␣
 ↪markerfacecolor='gray',
                              markeredgecolor='black', markersize=11,␣
 ↪label='Category Mean'))
ax.legend(handles=legend_elements, loc='upper left', fontsize=10)
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

# Print summary statistics
print("\nSpeech rhythm statistics per category:\n")
rhythm_stats = rhythm_df.groupby('Category').agg({
    'Onset Density (per sec)': ['mean', 'std'],
    'Mean ZCR': ['mean', 'std']
}).round(4)
rhythm_stats.columns = ['Onset Mean', 'Onset Std', 'ZCR Mean', 'ZCR Std']
print(rhythm_stats.to_string())
```






```
Speech rhythm statistics per category:

              Onset Mean  Onset Std  ZCR Mean  ZCR Std
Category
Bestfriend        5.9500     1.0422    0.0978   0.0112
Mom               4.9907     1.2077    0.0862   0.0155
Professional      5.9180     0.9046    0.1030   0.0130
```

The rhythm analysis reveals a pattern that none of the earlier features captured directly.

Mom has the lowest onset density at 5.0 onsets per second, compared to Bestfriend (6.0) and Professional (5.9). This means I produce roughly one fewer new sound event per second when talking to my mom, consistent with a slower, more relaxed pace with more pauses. Interestingly, Bestfriend and Professional are nearly identical in onset density (6.0 vs 5.9), which makes sense because both contexts involve me speaking actively, just with different tonal quality.

The zero-crossing rate tells a different story. Professional has the highest mean ZCR at 0.1030, followed by Bestfriend at 0.0978, and Mom at 0.0862. A higher ZCR typically indicates more sharp, unvoiced consonant sounds (“s”, “t”, “k”) relative to sustained vowels. This fits the idea that professional speech involves more precise articulation and crisper consonant delivery, while Mom speech has more relaxed, vowel-heavy delivery.

The scatter plot is interesting aswell. Mom's blue ellipse is in the lower-left quadrant of the plot, meaning slower and smoother speech. The Bestfriend and Professional ellipses overlap substantially in the center-right, but the Professional mean (green X) pushes slightly higher on the ZCR axis. The separation between Mom's cluster and the other two reinforces that Mom is the easiest category to distinguish. The overlap between Bestfriend and Professional on these rhythm features shows what the spectral centroid analysis showed, and it suggests the model will need to combine multiple features to tell those two apart.

3.15 Data Analysis Summary

1. The dataset has 166 clips total after segmentation: 62 Bestfriend, 54 Mom, and 50 Professional. The mild imbalance will be handled with macro-averaged metrics and stratified splitting.
2. The waveform plots showed that Bestfriend clips tend to be louder and denser, Mom clips have more visible pauses, and Professional clips sit in a controlled middle range.
3. Pitch analysis revealed that Bestfriend has the highest mean pitch at 123.2 Hz with a standard deviation of 24.8 Hz. Mom has the lowest mean at 105.6 Hz but the highest variability (std = 29.4 Hz). Professional sits at 115.5 Hz with the tightest spread (std = 19.0 Hz).
4. RMS energy confirmed that Bestfriend is the loudest on average (mean RMS = 0.1172), Mom is the quietest (0.0812), and Professional falls in between (0.0943). I speak roughly 44% louder with my friend than with my mom.
5. Spectral centroid showed a clear split: Mom has the lowest average at 1270.2 Hz, while Bestfriend (1404.7 Hz) and Professional (1404.8 Hz) are nearly identical. This means spectral centroid is most useful for identifying Mom clips but will not help separate the other two.
6. The average MFCC profiles differ most in the first few coefficients (especially MFCC 1, which relates to overall energy) and converge in the higher-order ones. Mom's profile is the most distinct, sitting below the other two in the low-order coefficients.
7. Amplitude distributions are all roughly Laplacian. Mom has the tallest zero-peak (most silence), consistent with RMS and silence ratio findings.
8. Silence ratio is one of the strongest separating features: Mom clips are 26.2% silent on average, nearly double Bestfriend (14.1%) and Professional (15.6%).
9. Speech rhythm analysis showed that Mom has the lowest onset density (5.0 onsets/sec vs 6.0 for Bestfriend and 5.9 for Professional), confirming a slower speaking pace. Professional has the highest zero-crossing rate (0.1030), suggesting crisper consonant articulation compared to the more relaxed delivery with Mom (0.0862).

Overall, the Data Analysis section provides clear evidence that my speaking style genuinely differs across the three social contexts. Mom clips are the most distinctive category, being quieter, lower in pitch, lower in spectral centroid, slower in pace, and containing more pauses. Bestfriend and Professional may be harder for the model to separate since they share similar spectral centroid values, onset density, and their differences are more subtle on other features. The next step is to extract the MFCC feature matrix, split the data, and build the two classifiers.

4 Part 2: Classifying Social Context From Voice

4.1 MFCC Feature Extraction

MFCCs are the standard feature representation for speech and audio classification tasks. The reason they work so well comes down to how they are computed. They transform the raw audio spectrum into a compact representation that mirrors the way the human ear actually perceives sound.

The extraction pipeline works in four stages:

1. The 10-second clip is divided into short overlapping frames. I use the default 2048-sample frame with a 512-sample hop, which at 16 kHz gives a roughly 25 ms analysis window stepping every 32 ms.
2. For each frame, I compute the Short-Time Fourier Transform and convert it to a power spectrum.
3. The power spectrum is passed through a mel filterbank, where low frequencies get finer resolution because that is where speech pitch and formants are most informative.
4. I apply a DCT to the log mel energies so the spectrum becomes a small set of decorrelated coefficients.

The mel mapping is:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right)$$

In the final baseline, I keep the original 13 MFCC means and 13 MFCC standard deviations, then extend them with delta and delta-delta MFCC means. Those derivative terms summarize how the spectrum changes over time, which gives the models a little more information about pacing and articulation dynamics. I also keep RMS mean and silence ratio, since both were strong signals in the exploratory analysis.

[17]:
```
# Extract features from all clips
# This includes MFCC means/stds, delta summaries, delta-delta summaries,
# and the group IDs needed for note-level splitting.
N_MFCC = 13
features = []
labels   = []
groups   = []

for cat in CATEGORIES:
    label_idx = LABEL_MAP[cat]
    clip_dir = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)

        mfcc = librosa.feature.mfcc(y=y_clip, sr=SR, n_mfcc=N_MFCC,␣
↪hop_length=512)
        mfcc_mean = np.mean(mfcc, axis=1)
```

```
        mfcc_std  = np.std(mfcc, axis=1)

        delta_mfcc      = librosa.feature.delta(mfcc, order=1)
        delta_mfcc_mean = np.mean(delta_mfcc, axis=1)

        delta2_mfcc      = librosa.feature.delta(mfcc, order=2)
        delta2_mfcc_mean = np.mean(delta2_mfcc, axis=1)

        rms = librosa.feature.rms(y=y_clip, frame_length=2048,␣
 ↪hop_length=512)[0]
        rms_mean = np.mean(rms)
        silence_ratio = np.mean(rms < SILENCE_THRESHOLD)

        feat = np.concatenate([
            mfcc_mean, mfcc_std,
            delta_mfcc_mean,
            delta2_mfcc_mean,
            [rms_mean, silence_ratio]
        ])
        features.append(feat)
        labels.append(label_idx)

        base = os.path.basename(fpath)
        group_id = base.rsplit('_clip', 1)[0]
        groups.append(f"{cat}_{group_id}")

X = np.array(features)
y = np.array(labels)
groups = np.array(groups)

feat_names = (
    [f"MFCC {i+1} mean" for i in range(N_MFCC)] +
    [f"MFCC {i+1} std" for i in range(N_MFCC)] +
    [f"Delta MFCC {i+1} mean" for i in range(N_MFCC)] +
    [f"Delta-Delta MFCC {i+1} mean" for i in range(N_MFCC)] +
    ["RMS mean", "Silence ratio"]
)

print(f"Feature matrix shape: {X.shape}")
print(f"Labels shape:         {y.shape}")
print(f"Unique groups:        {len(np.unique(groups))}")
print(f"Feature dimension:    {len(feat_names)}")
print("\nFirst 10 feature names:")
print(feat_names[:10])
```

```
Feature matrix shape: (166, 54)
Labels shape:         (166,)
Unique groups:        42
```

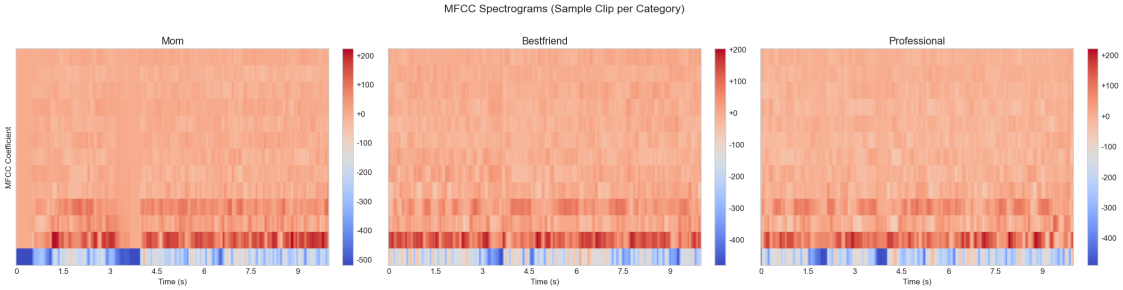
```
Feature dimension:    54

First 10 feature names:
['MFCC 1 mean', 'MFCC 2 mean', 'MFCC 3 mean', 'MFCC 4 mean', 'MFCC 5 mean',
'MFCC 6 mean', 'MFCC 7 mean', 'MFCC 8 mean', 'MFCC 9 mean', 'MFCC 10 mean']
```

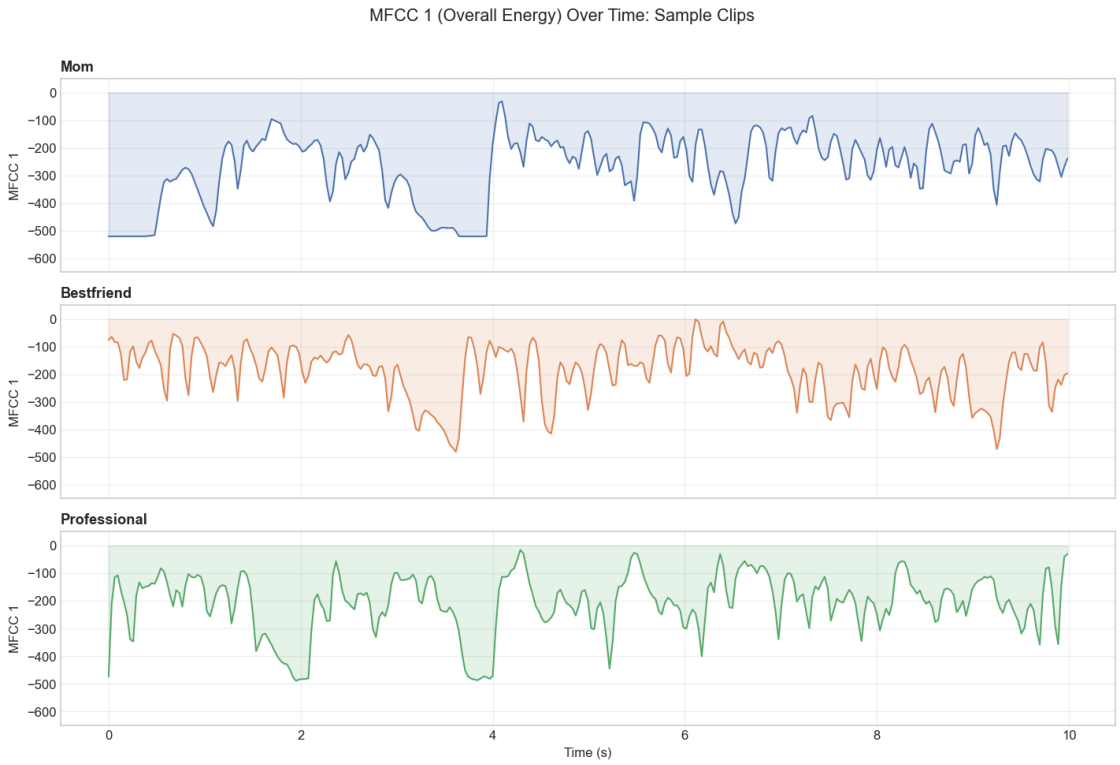
4.2 Visualizing Sample MFCC Spectrograms

Before feeding these features into a model, it helps to actually look at what the MFCC representation captures. The plots below show the full MFCC matrix for one sample clip from each category, using the same clips from the Data Analysis. Each row is one of the 13 MFCC coefficients, each column is a time frame, and the color represents the coefficient value. Differences in the color patterns across categories reflect differences in vocal quality, energy, and articulation.

[18]:
```
# Plot MFCC spectrograms for one sample clip per category
fig, axes = plt.subplots(1, 3, figsize=(20, 5), sharey=True)

for i, label in enumerate(LABEL_NAMES):
    waveform = samples[label]
    mfcc_sample = librosa.feature.mfcc(y=waveform, sr=SR, n_mfcc=N_MFCC,␣
 ↪hop_length=512)

    img = librosa.display.specshow(
        mfcc_sample, sr=SR, hop_length=512, x_axis='time',
        ax=axes[i], cmap='coolwarm'
    )
    axes[i].set_title(f'{label}', fontsize=13)
    axes[i].set_xlabel('Time (s)')
    if i == 0:
        axes[i].set_ylabel('MFCC Coefficient')
    # Add individual colorbar per panel to avoid overlap
    fig.colorbar(img, ax=axes[i], format='%+2.0f', fraction=0.046, pad=0.04)

fig.suptitle('MFCC Spectrograms (Sample Clip per Category)', fontsize=14, y=1.
 ↪02)
plt.tight_layout()
plt.show()

# Show the difference between categories by plotting MFCC 1 over time
# Use 3-panel stacked layout for clarity instead of overlaid lines
fig, axes = plt.subplots(3, 1, figsize=(12, 8), sharex=True)
colors = ['#4C72B0', '#DD8452', '#55A868']

for i, label in enumerate(LABEL_NAMES):
    waveform = samples[label]
    mfcc_sample = librosa.feature.mfcc(y=waveform, sr=SR, n_mfcc=N_MFCC,␣
 ↪hop_length=512)
```

```
    times = librosa.frames_to_time(np.arange(mfcc_sample.shape[1]), sr=SR,␣
 ↪hop_length=512)
    axes[i].plot(times, mfcc_sample[0], color=colors[i], linewidth=1.2)
    axes[i].fill_between(times, mfcc_sample[0], alpha=0.15, color=colors[i])
    axes[i].set_ylabel('MFCC 1', fontsize=10)
    axes[i].set_title(f'{label}', fontsize=11, loc='left', fontweight='bold')
    axes[i].grid(True, alpha=0.3)
    # Set consistent y-limits across all panels for direct comparison
    axes[i].set_ylim(-650, 50)

axes[-1].set_xlabel('Time (s)')
fig.suptitle('MFCC 1 (Overall Energy) Over Time: Sample Clips', fontsize=13,␣
 ↪y=1.01)
plt.tight_layout()
plt.show()
```





The MFCC spectrograms above reveal clear visual differences between the three categories. The Mom spectrogram (left) shows abrupt color transitions in the lower coefficients (especially MFCC 1), with a prominent deep-blue block in the first third of the clip where the signal drops during a pause. The Bestfriend spectrogram (center) is denser and more uniformly warm across the full 10 seconds, reflecting the more continuous speech identified in the EDA. The Professional spectrogram (right) shows a sharp blue vertical band around the 3-second mark where a brief pause occurs, but the rest of the clip is relatively dense.

The MFCC 1 time-series plot underneath makes these differences quantitative. MFCC 1 represents overall spectral energy, and the Mom trace (blue) plunges to nearly -550 during pauses, while the Bestfriend trace (orange) rarely dips below -200. These are exactly the kinds of temporal patterns that a CNN can learn to detect by sliding convolutional filters across the time axis, which is why I feed the full MFCC matrix into the CNN rather than just the summary statistics.

4.3 Train/Test Split and Standardization

A simple random 80/20 split would leak information across the train and test sets. If one 10-second clip from a voice note lands in training and another clip from that same note lands in testing, the model gets to see nearly the same recording conditions twice. That makes the evaluation too optimistic.

To avoid that, I split by original voice note using `GroupShuffleSplit`. Every clip inherits a group ID from its parent message, and the splitter keeps each message entirely in training or entirely in test. This means the held-out set is made of genuinely unseen conversations rather than unseen

slices of familiar ones.

The printed output below confirms how many clips and unique notes land in each split, along with the class distribution after grouping. I still standardize the tabular features with `StandardScaler`, fit on the training set only and then applied to test. That keeps the scaling parameters clean and avoids leakage there too.

[19]:
```
from sklearn.model_selection import GroupShuffleSplit

gss = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups=groups))

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

print(f"Training set: {X_train.shape[0]} clips  ({len(np.
 ↪unique(groups[train_idx]))} unique voice notes)")
print(f"Test set:     {X_test.shape[0]} clips   ({len(np.
 ↪unique(groups[test_idx]))} unique voice notes)")
print(f"\nTraining class distribution: {dict(zip(LABEL_NAMES, np.
 ↪bincount(y_train)))}")
print(f"Test class distribution:     {dict(zip(LABEL_NAMES, np.
 ↪bincount(y_test)))}")

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)

print(f"\nAfter scaling | train mean = {X_train_scaled.mean():.4f}, std =␣
 ↪{X_train_scaled.std():.4f}")
print(f"After scaling | test  mean = {X_test_scaled.mean():.4f}, std =␣
 ↪{X_test_scaled.std():.4f}")
```

```
Training set: 135 clips  (33 unique voice notes)
Test set:     31 clips   (9 unique voice notes)

Training class distribution: {'Mom': np.int64(40), 'Bestfriend': np.int64(54),
'Professional': np.int64(41)}
Test class distribution:     {'Mom': np.int64(14), 'Bestfriend': np.int64(8),
'Professional': np.int64(9)}

After scaling | train mean = -0.0000, std = 1.0000
After scaling | test  mean = 0.0296, std = 1.1022
```

[20]:
```
# Apply augmentations to training clips only
# The group split is already defined above, so the held-out notes remain␣
 ↪untouched.
```

```
train_groups_set = set(groups[train_idx])
aug_features_list = []
aug_labels_list   = []

print("Applying augmentations to training clips only...")
np.random.seed(42)

for cat in CATEGORIES:
    label_idx = LABEL_MAP[cat]
    clip_dir  = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        base     = os.path.basename(fpath)
        group_id = base.rsplit('_clip', 1)[0]

        if f"{cat}_{group_id}" not in train_groups_set:
            continue

        y_clip, _ = librosa.load(fpath, sr=SR)

        for aug_name, aug_fn in AUGMENT_FUNCS:
            y_aug = aug_fn(y_clip)

            mfcc        = librosa.feature.mfcc(y=y_aug, sr=SR, n_mfcc=N_MFCC,␣
 ↪hop_length=512)
            mfcc_a      = spec_augment(mfcc)
            mfcc_mean   = np.mean(mfcc_a, axis=1)
            mfcc_std    = np.std(mfcc_a, axis=1)
            delta_mean  = np.mean(librosa.feature.delta(mfcc_a, order=1),␣
 ↪axis=1)
            delta2_mean = np.mean(librosa.feature.delta(mfcc_a, order=2),␣
 ↪axis=1)
            rms         = librosa.feature.rms(y=y_aug, frame_length=2048,␣
 ↪hop_length=512)[0]

            feat = np.concatenate([
                mfcc_mean, mfcc_std, delta_mean, delta2_mean,
                [np.mean(rms), np.mean(rms < SILENCE_THRESHOLD)]
            ])
            aug_features_list.append(feat)
            aug_labels_list.append(label_idx)

aug_features = np.array(aug_features_list)
aug_labels   = np.array(aug_labels_list)
aug_scaled   = scaler.transform(aug_features)

X_train_aug = np.vstack([X_train_scaled, aug_scaled])
```

```
y_train_aug = np.concatenate([y_train, aug_labels])

print(f"Augmented rows added:       {aug_scaled.shape[0]}")
print(f"Combined training shape:    {X_train_aug.shape}")
print(f"Combined class distribution {dict(zip(LABEL_NAMES, np.
  ↪bincount(y_train_aug)))}")
```

```
Applying augmentations to training clips only…
Augmented rows added:       540
Combined training shape:    (675, 54)
Combined class distribution {'Mom': np.int64(200), 'Bestfriend': np.int64(270),
'Professional': np.int64(205)}
```

4.4 Model Selection and Architecture

4.5 Model 1: Multinomial Softmax Regression

4.5.1 Why Softmax Regression?

1. This is a 3-class problem (Mom, Bestfriend, Professional). Softmax regression is the direct generalization of binary logistic regression to $K > 2$ classes. It outputs a probability distribution over all classes, which is useful for understanding how confident the model is in each prediction.
2. With only 166 clips and 28 features, the dataset is too small for complex models like deep neural networks on tabular data, which would overfit immediately. Softmax regression has relatively few parameters ($D \times K + K = 28 \times 3 + 3 = 87$), making it less prone to overfitting on small data.
3. Each weight w_{jk} directly tells us how much feature j contributes to predicting class k. A large positive weight on "MFCC 1 mean" for the Mom class means that when MFCC 1 is high (after scaling), the model leans toward predicting Mom. This interpretability is valuable for validating whether the model learned sensible patterns.

4.5.2 Step-by-Step Algorithm

1. Initialize parameters. The model has a weight matrix $W \in \mathbb{R}^{D \times K}$ (where $D = 28$ features and $K = 3$ classes) and a bias vector $\mathbf{b} \in \mathbb{R}^K$. Both are initialized to zeros. Zero initialization works for softmax regression because the loss function is convex, so gradient descent will converge to the global minimum regardless of the starting point. This is different from neural networks, where zero initialization causes symmetry problems.

2. Compute logits (forward pass). For each training sample $\mathbf{x}_i \in \mathbb{R}^D$, compute raw scores (logits) for each class:

$$z_{ik} = \sum_{j=1}^{D} w_{jk} \cdot x_{ij} + b_k$$

In matrix form for the entire training batch of N samples:

$$Z = XW + \mathbf{1}\mathbf{b}^T \quad \in \mathbb{R}^{N \times K}$$

Each row of Z contains 3 scores, one per class. These scores can be any real number and are not yet probabilities.

3. Apply the softmax function. The softmax function converts logits into a valid probability distribution. For sample i, the probability of class k is:

$$P(y_i = k \mid \mathbf{x}_i) = \frac{e^{z_{ik}}}{\sum_{j=1}^{K} e^{z_{ij}}}$$

This guarantees that all probabilities are positive and sum to 1. To avoid numerical overflow (since e^z can be astronomically large), I use the numerically stable version where I first subtract the maximum logit from each row:

$$z'_{ik} = z_{ik} - \max_j(z_{ij})$$

This does not change the output probabilities (the constant cancels in the numerator and denominator) but prevents e^z from exploding.

4. Compute the cross-entropy loss. The cross-entropy loss measures how far the predicted probabilities are from the true labels. If sample i belongs to class c_i, the loss is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log P(y_i = c_i \mid \mathbf{x}_i)$$

Intuitively, if the model assigns probability 0.95 to the correct class, $-\log(0.95) \approx 0.05$, which is a small loss. If it assigns probability 0.01, $-\log(0.01) \approx 4.6$, which is a large loss. The model is penalized harshly for being confidently wrong.

I clip the probabilities to $[10^{-12}, 1]$ before taking the log to avoid $\log(0) = -\infty$.

5. Compute gradients. This is where the calculus happens. Let $\mathbf{p}_i \in \mathbb{R}^K$ be the vector of predicted probabilities for sample i, and let $\mathbf{e}_i \in \mathbb{R}^K$ be the one-hot encoding of the true label y_i (a vector of zeros with a 1 at position c_i). The error for sample i is:

$$\delta_i = \mathbf{p}_i - \mathbf{e}_i$$

This error vector has a nice property: for the correct class, $\delta_{ik} = p_{ik} - 1$ (negative, pushing the weight to increase that probability), and for incorrect classes, $\delta_{ik} = p_{ik}$ (positive, pushing the weight to decrease those probabilities).

The gradients of the loss with respect to W and $\mathbf{b}$ are:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{N} X^T \Delta \quad \in \mathbb{R}^{D \times K}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{N} \sum_{i=1}^{N} \delta_i \quad \in \mathbb{R}^K$$

where $\Delta \in \mathbb{R}^{N \times K}$ is the matrix of all error vectors stacked row-wise.

6. Update parameters (gradient descent). With learning rate η:

$$W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

$$\mathbf{b} \leftarrow \mathbf{b} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{b}}$$

The learning rate controls how big of a step to take in the direction of steepest descent. Too large and the model overshoots the minimum; too small and convergence takes too long. I use $\eta = 0.1$. The reasoning behind this choice is demonstrated in Section 6.1, where I compare five different learning rates and show that 0.1 is the sweet spot.

7. Repeat steps 2-6 for a fixed number of epochs (I use 1000). Since the loss function is convex, the model is guaranteed to converge.

8. Predict. For a new sample $\mathbf{x}$, compute $\mathbf{z} = W^T\mathbf{x} + \mathbf{b}$, apply softmax, and return the class with the highest probability: $\hat{y} = \arg\max_k P(y = k \mid \mathbf{x})$.

[21]:
```python
class SoftmaxRegression:
    """Multinomial softmax regression trained with gradient descent."""

    def __init__(self, n_features, n_classes, lr=0.1, epochs=1000):
        self.lr = lr
        self.epochs = epochs
        self.n_classes = n_classes
        # Initialize weights and bias to zeros
        self.W = np.zeros((n_features, n_classes))
        self.b = np.zeros(n_classes)
        self.loss_history = []

    def _softmax(self, z):
        """Numerically stable softmax."""
        z_shifted = z - np.max(z, axis=1, keepdims=True)
        exp_z = np.exp(z_shifted)
        return exp_z / np.sum(exp_z, axis=1, keepdims=True)

    def _one_hot(self, y, K):
        """Convert label vector to one-hot matrix."""
        N = len(y)
        one_hot = np.zeros((N, K))
        one_hot[np.arange(N), y] = 1.0
        return one_hot
```

```python
    def _cross_entropy_loss(self, probs, y):
        """Compute mean cross-entropy loss."""
        N = len(y)
        # Clip probabilities to avoid log(0)
        log_probs = np.log(np.clip(probs[np.arange(N), y], 1e-12, 1.0))
        return -np.mean(log_probs)

    def fit(self, X, y):
        """Train the model using full-batch gradient descent."""
        N, D = X.shape
        K = self.n_classes
        Y_onehot = self._one_hot(y, K)

        for epoch in range(self.epochs):
            # Forward pass
            z = X @ self.W + self.b             # (N, K)
            probs = self._softmax(z)            # (N, K)

            # Compute loss
            loss = self._cross_entropy_loss(probs, y)
            self.loss_history.append(loss)

            # Compute gradients
            error = probs - Y_onehot            # (N, K)
            grad_W = (1 / N) * (X.T @ error)  # (D, K)
            grad_b = (1 / N) * np.sum(error, axis=0)  # (K,)

            # Update parameters
            self.W -= self.lr * grad_W
            self.b -= self.lr * grad_b

            if (epoch + 1) % 200 == 0:
                print(f"  Epoch {epoch+1:4d}/{self.epochs}  |  Loss: {loss:.
↪4f}")

        return self

    def predict_proba(self, X):
        """Return predicted probabilities for each class."""
        z = X @ self.W + self.b
        return self._softmax(z)

    def predict(self, X):
        """Return predicted class labels."""
        return np.argmax(self.predict_proba(X), axis=1)
```

4.6 Model 2: Convolutional Neural Network (1D)

4.6.1 Why a CNN instead of feeding the same 28 features into a second model?

The softmax regression model reduces each clip to a 28-dimensional summary vector (MFCC means, stds, RMS, silence ratio). This works, but it throws away the temporal structure of the audio. A 10-second clip has about 313 MFCC frames (160,000 samples / 512 hop length = 312.5, rounded to 313). The pattern of how those frames change over time carries useful information. For example, Mom clips have long stretches of near-silence followed by speech bursts, while Bestfriend clips have continuously high energy. Averaging over time erases these patterns.

A 1D Convolutional Neural Network (CNN) solves this by operating directly on the full MFCC time-series. Instead of getting a summary, it receives the entire $(T, 13)$ matrix (313 time frames, 13 MFCC coefficients) and learns to detect local temporal patterns by sliding small filters across the time axis. This is analogous to how 2D CNNs detect edges and textures in images, except here the "image" is a 1D sequence of MFCC vectors.

I chose a 1D CNN over an RNN/LSTM because CNNs are faster to train, less prone to vanishing gradient problems, and work well for fixed-length inputs. With only 166 samples, a simpler CNN architecture is also less likely to overfit compared to a recurrent model.

4.6.2 Step by Step Algorithm

1. Input layer: shape $(T, 13)$ The input is the full MFCC matrix for one clip: $T = 313$ time frames, each a 13-dimensional vector. Think of this as a single-channel "image" that is 313 pixels wide and 13 pixels tall (but processed along the width/time axis only).

2. First Conv1D layer (32 filters, kernel size 3, ReLU activation, same padding) This layer creates 32 convolutional filters, each of size $(3, 13)$, meaning each filter looks at 3 consecutive time frames across all 13 MFCC coefficients simultaneously. The convolution operation for filter f at time step t is:

$$h_t^{(f)} = \text{ReLU}\left(\sum_{i=0}^{2}\sum_{j=0}^{12} w_{ij}^{(f)} \cdot x_{t+i,\,j} + b^{(f)}\right)$$

The ReLU activation ($\text{ReLU}(z) = \max(0, z)$) introduces non-linearity by zeroing out negative values. Without non-linearity, stacking multiple linear layers would collapse into a single linear transformation, defeating the purpose of a deep network.

"Same" padding adds zeros at the edges so the output has the same time dimension as the input. The output shape after this layer is $(313, 32)$: 313 time steps, 32 feature maps.

3. Batch Normalization Batch normalization normalizes the activations within each mini-batch to have zero mean and unit variance, then applies a learned scale and shift:

$$\hat{h} = \gamma \cdot \frac{h - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} + \beta$$

where γ and β are learnable parameters and ϵ is a small constant for numerical stability. This stabilizes training by preventing the distribution of activations from shifting between layers (a

problem called *internal covariate shift*), allowing higher learning rates and faster convergence.

4. MaxPooling1D (pool size 2) Max pooling slides a window of size 2 along the time axis and keeps only the maximum value in each window:

$$p_t = \max(h_{2t},\ h_{2t+1})$$

This halves the time dimension from 313 to 156, reducing the number of parameters in subsequent layers and making the representation more robust to small time shifts. If a speech pattern occurs at $t = 50$ vs $t = 51$, max pooling treats them the same.

5. Second Conv1D layer (64 filters, kernel size 3, ReLU, same padding) + BatchNorm + MaxPool(2) The same operations are repeated with 64 filters instead of 32. Each filter now operates on the 32-dimensional output of the previous block, so it is combining and abstracting over the first layer's learned patterns. After pooling, the time dimension is halved again: $156 \to 78$.

6. Third Conv1D layer (64 filters, kernel size 3, ReLU, same padding) + BatchNorm A third convolutional block adds another layer of abstraction. At this point, each of the 64 filters is detecting patterns that span multiple seconds of audio (because the pooling layers have compressed the time axis). The output shape is $(78, 64)$.

7. GlobalAveragePooling1D Instead of flattening the entire $(78, 64)$ tensor (which would create $78 \times 64 = 4{,}992$ features and dramatically increase the risk of overfitting), I use global average pooling. This computes the mean of each filter's output across all remaining time frames:

$$\bar{h}^{(f)} = \frac{1}{78} \sum_{t=1}^{78} h_t^{(f)}$$

The result is a single 64-dimensional vector that summarizes the entire clip. Global average pooling is a form of extreme compression: it says "I do not care *where* in the clip a pattern occurred, only *whether* it occurred on average."

8. Dense layer (64 units, ReLU) A standard fully connected layer that combines the 64 pooled features into a richer representation. Each output neuron computes a weighted sum of all 64 inputs plus a bias, then applies ReLU. This layer has $64 \times 64 + 64 = 4{,}160$ parameters.

9. Dropout (rate = 0.3) During each training step, 30% of the neurons in the previous dense layer are randomly set to zero. This forces the network to learn redundant representations and prevents co-adaptation of neurons, acting as a regularizer. At inference time, all neurons are active but their outputs are scaled by 0.7 to compensate.

10. Output Dense layer (3 units, Softmax) The final layer has one neuron per class. The softmax activation converts the 3 raw outputs into probabilities, exactly as in softmax regression:

$$P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{3} e^{z_j}}$$

The model is trained with sparse categorical cross-entropy (same loss function as the softmax regression, but Keras computes it from integer labels rather than one-hot vectors) and the Adam optimizer. Adam (Kingma & Ba, 2015) adapts the learning rate per parameter based on estimates

of the first and second moments of the gradient, which helps it converge faster and more reliably than plain gradient descent for neural networks.

[22]:
```
# Prepare MFCC time-series data for the CNN
# CNN gets the full MFCC matrix per clip

mfcc_sequences = []
cnn_labels = []

for cat in CATEGORIES:
    label_idx = LABEL_MAP[cat]
    clip_dir = os.path.join(CLIP_DIR, cat)

    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        y_clip, _ = librosa.load(fpath, sr=SR)
        mfcc = librosa.feature.mfcc(y=y_clip, sr=SR, n_mfcc=N_MFCC,␣
 ↪hop_length=512)
        mfcc_sequences.append(mfcc.T)  # transpose to (T, 13)
        cnn_labels.append(label_idx)

# All clips are 10 seconds at 16kHz with hop=512 -> same number of frames
lengths = [m.shape[0] for m in mfcc_sequences]
print(f"MFCC frame counts: min={min(lengths)}, max={max(lengths)}")

T = min(lengths)
X_cnn = np.array([m[:T, :] for m in mfcc_sequences])
y_cnn = np.array(cnn_labels)

print(f"CNN input shape: {X_cnn.shape}  (samples, time_frames,␣
 ↪mfcc_coefficients)")

# Use the same group-based split as the tabular model
# so that all three models are evaluated on the exact same test clips
X_cnn_train = X_cnn[train_idx]
X_cnn_test  = X_cnn[test_idx]
y_cnn_train = y_cnn[train_idx]
y_cnn_test  = y_cnn[test_idx]

# Standardize each MFCC coefficient independently (fit on train)
orig_shape_train = X_cnn_train.shape
orig_shape_test  = X_cnn_test.shape

cnn_scaler = StandardScaler()
X_cnn_train_flat = X_cnn_train.reshape(-1, N_MFCC)
X_cnn_test_flat  = X_cnn_test.reshape(-1, N_MFCC)

cnn_scaler.fit(X_cnn_train_flat)
```

```
X_cnn_train_scaled = cnn_scaler.transform(X_cnn_train_flat).
 ↪reshape(orig_shape_train)
X_cnn_test_scaled  = cnn_scaler.transform(X_cnn_test_flat).
 ↪reshape(orig_shape_test)

print(f"CNN train shape: {X_cnn_train_scaled.shape}")
print(f"CNN test shape:  {X_cnn_test_scaled.shape}")
print(f"Test set size matches tabular model: {len(y_cnn_test) == len(y_test)}")
```

```
MFCC frame counts: min=313, max=313
CNN input shape: (166, 313, 13)  (samples, time_frames, mfcc_coefficients)
CNN train shape: (135, 313, 13)
CNN test shape:  (31, 313, 13)
Test set size matches tabular model: True
```

[23]:
```
# Augmented MFCC sequences for the CNN
# The same waveform transforms are used here, followed by SpecAugment on the␣
 ↪MFCC view.

train_groups_set_cnn = set(groups[train_idx])
cnn_aug_seq_list = []
cnn_aug_lab_list = []
np.random.seed(42)

for cat in CATEGORIES:
    label_idx = LABEL_MAP[cat]
    clip_dir = os.path.join(CLIP_DIR, cat)
    for fpath in sorted(glob.glob(os.path.join(clip_dir, '*.wav'))):
        base = os.path.basename(fpath)
        group_id = base.rsplit('_clip', 1)[0]
        if f"{cat}_{group_id}" not in train_groups_set_cnn:
            continue
        y_clip, _ = librosa.load(fpath, sr=SR)
        for _, aug_fn in AUGMENT_FUNCS:
            y_a = aug_fn(y_clip)
            mfcc = librosa.feature.mfcc(y=y_a, sr=SR, n_mfcc=N_MFCC,␣
 ↪hop_length=512)
            mfcc_a = spec_augment(mfcc)
            seq = mfcc_a.T
            fr = seq.shape[0]
            if fr >= T:
                seq = seq[:T, :]
            else:
                seq = np.pad(seq, ((0, T - fr), (0, 0)), mode='constant')
            cnn_aug_seq_list.append(seq)
            cnn_aug_lab_list.append(label_idx)
```

```
X_cnn_aug_np = np.array(cnn_aug_seq_list)
y_cnn_aug_np = np.array(cnn_aug_lab_list)
X_cnn_aug_scaled_only = cnn_scaler.transform(
    X_cnn_aug_np.reshape(-1, N_MFCC)
).reshape(X_cnn_aug_np.shape)

X_cnn_train_aug = np.concatenate([X_cnn_train_scaled, X_cnn_aug_scaled_only],␣
 ↪axis=0)
y_cnn_train_aug = np.concatenate([y_cnn_train, y_cnn_aug_np])

print(f"CNN augmented sequences:    {X_cnn_aug_scaled_only.shape[0]}")
print(f"CNN combined training shape:{X_cnn_train_aug.shape}")
print(f"Class counts (aug train):   {dict(zip(LABEL_NAMES, np.
 ↪bincount(y_cnn_train_aug)))}")
```

```
CNN augmented sequences:    540
CNN combined training shape:(675, 313, 13)
Class counts (aug train):   {'Mom': np.int64(200), 'Bestfriend': np.int64(270),
'Professional': np.int64(205)}
```

4.6.3 Visualizing the CNN Input

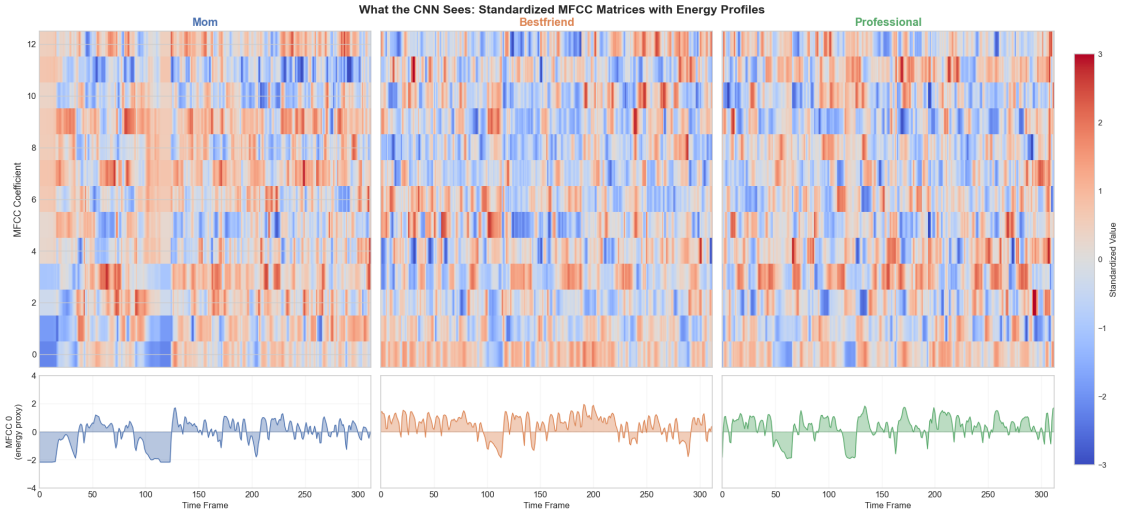
Before training, I want to actually look at what the CNN is going to see. The plots below show the full standardized MFCC time-series matrix for one sample clip from each category. A good way to get my head across this was to understand is as a heat map: the x-axis is time (313 frames, each about 32ms of audio), the y-axis is the 13 MFCC coefficients, and the color shows the standardized value. Red means that coefficient is unusually high at that moment, blue means it is unusually low, and white is close to the average.

The CNN's convolutional filters will slide across the time axis (left to right) looking for short local patterns in these matrices. If Mom clips consistently show large blue patches (silence) while Bestfriend clips are a dense mix of red and blue (constant talking, high energy), the filters should learn to pick up on that difference.

[24]:
```
# Visualize what the CNN sees: standardized MFCC matrices for sample clips
fig, axes = plt.subplots(
    2, 3, figsize=(18, 8),
    gridspec_kw={'height_ratios': [3, 1]},
    constrained_layout=True
 )

# Get sample clips
sample_mfccs = []
for cat in CATEGORIES:
    clip_dir = os.path.join(CLIP_DIR, cat)
    clip_list = sorted(glob.glob(os.path.join(clip_dir, '*.wav')))
    y_s, _ = librosa.load(clip_list[SAMPLE_IDX], sr=SR)
    mfcc_s = librosa.feature.mfcc(y=y_s, sr=SR, n_mfcc=N_MFCC, hop_length=512)
```

```
    mfcc_s_scaled = cnn_scaler.transform(mfcc_s.T).T
    sample_mfccs.append(mfcc_s_scaled)

# Row 1: MFCC heatmaps
category_colors = ['#4C72B0', '#DD8452', '#55A868']
images = []
for i, label in enumerate(LABEL_NAMES):
    im = axes[0][i].imshow(
        sample_mfccs[i][:, :T], aspect='auto', origin='lower',
        cmap='coolwarm', interpolation='nearest', vmin=-3, vmax=3
    )
    images.append(im)
    axes[0][i].set_title(f'{label}', fontsize=13, fontweight='bold',␣
 ↪color=category_colors[i])
    axes[0][i].set_xlabel('')
    axes[0][i].set_xticks([])
    if i == 0:
        axes[0][i].set_ylabel('MFCC Coefficient', fontsize=11)
    else:
        axes[0][i].set_yticks([])

# Row 2: energy profile (MFCC 0 over time) to show speech vs silence
for i, label in enumerate(LABEL_NAMES):
    mfcc0 = sample_mfccs[i][0, :T]  # MFCC 0 ~ overall energy
    x = np.arange(len(mfcc0))
    axes[1][i].fill_between(x, mfcc0, alpha=0.4, color=category_colors[i])
    axes[1][i].plot(x, mfcc0, color=category_colors[i], linewidth=0.8)
    axes[1][i].axhline(y=0, color='gray', linewidth=0.5, linestyle=':')
    axes[1][i].set_xlabel('Time Frame', fontsize=10)
    axes[1][i].set_xlim(0, T - 1)
    axes[1][i].set_ylim(-4, 4)
    if i == 0:
        axes[1][i].set_ylabel('MFCC 0\n(energy proxy)', fontsize=10)
    else:
        axes[1][i].set_yticks([])
    axes[1][i].grid(True, alpha=0.2)

# Shared colorbar, attached to all six axes so it sizes correctly
cbar = fig.colorbar(
    images[0], ax=axes.ravel().tolist(),
    label='Standardized Value', location='right', pad=0.02, shrink=0.9
 )
cbar.ax.tick_params(labelsize=9)

fig.suptitle(
    'What the CNN Sees: Standardized MFCC Matrices with Energy Profiles',
    fontsize=14, fontweight='bold', y=1.02
```

```
 )

plt.show()
```



The three heatmaps show different visual textures. The Mom clip has large blue patches (especially visible in the bottom row of the energy profile, where the signal drops well below zero for extended stretches). These correspond to the long pauses and silences we already noticed in the Data Analysis section. The speech segments appear as bursts of red/blue contrast, separated by calm blue stretches.

The Bestfriend clip is the opposite. It is a dense wall of color with almost no quiet patches. The energy profile stays active across nearly the entire 313 frames, confirming the "always talking, never stopping" pattern from earlier. The rapid color changes suggest more vocal variety and dynamic speech.

The Professional clip sits in between. It has some pauses but they are shorter and more evenly spaced than Mom's. The color patterns are also more structured and regular, which matches the controlled, measured speaking style I use in professional contexts. The energy profile shows steady, moderate activity without the extreme dips of Mom or the constant high energy of Bestfriend.

The bottom row (MFCC 0 over time) is especially useful because MFCC 0 correlates with the log-energy of the audio signal. It essentially shows the speaking rhythm: when I am talking vs. when I am silent. A CNN filter sliding across these matrices can learn that "long blue patches followed by short red bursts" means Mom, while "continuous color activity" means Bestfriend.

[25]:
```
tf.random.set_seed(42)

def build_cnn(input_shape, n_classes):
    model = models.Sequential([
        # First conv block
        layers.Conv1D(32, kernel_size=3, activation='relu', padding='same',
```

```
                    input_shape=input_shape),
        layers.BatchNormalization(),
        layers.MaxPooling1D(pool_size=2),

        # Second conv block
        layers.Conv1D(64, kernel_size=3, activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling1D(pool_size=2),

        # Third conv block
        layers.Conv1D(64, kernel_size=3, activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.GlobalAveragePooling1D(),

        # Classification head
        layers.Dense(64, activation='relu'),
        layers.Dropout(0.3),
        layers.Dense(n_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

cnn_model_na = build_cnn(
    input_shape=(X_cnn_train_scaled.shape[1], X_cnn_train_scaled.shape[2]),
    n_classes=3
)
cnn_model_aug = build_cnn(
    input_shape=(X_cnn_train_scaled.shape[1], X_cnn_train_scaled.shape[2]),
    n_classes=3
)
print("Two CNNs instantiated (identical architecture): cnn_model_na,␣
 ↪cnn_model_aug")
cnn_model_na.summary()
```

```
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Two CNNs instantiated (identical architecture): cnn_model_na, cnn_model_aug
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 313, 32)	1,280
batch_normalization (BatchNormalization)	(None, 313, 32)	128
max_pooling1d (MaxPooling1D)	(None, 156, 32)	0
conv1d_1 (Conv1D)	(None, 156, 64)	6,208
batch_normalization_1 (BatchNormalization)	(None, 156, 64)	256
max_pooling1d_1 (MaxPooling1D)	(None, 78, 64)	0
conv1d_2 (Conv1D)	(None, 78, 64)	12,352
batch_normalization_2 (BatchNormalization)	(None, 78, 64)	256
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense (Dense)	(None, 64)	4,160
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 3)	195

Total params: 24,835 (97.01 KB)

Trainable params: 24,515 (95.76 KB)

Non-trainable params: 320 (1.25 KB)

The model summary above shows the CNN has 24,835 total parameters (about 97 KB of memory). To put that in perspective, the softmax regression model has only $28 \times 3 + 3 = 87$ parameters. So the CNN is roughly 285 times larger, which is why it needs more data and careful training to avoid overfitting.

What each layer does:

Layer	What it does	Output shape	Parameters
Conv1D (32 filters)	Slides 32 small filters (width 3) across the time axis. Each filter learns a different local pattern in the 13 MFCC channels.	(313, 32)	$3 \times 13 \times 32 + 32 = 1{,}280$
BatchNorm	Normalizes each filter's output to zero mean and unit variance, which helps the model train faster and more stably.	(313, 32)	64 (mean + variance per filter)
MaxPool (size 2)	Cuts the time dimension in half by keeping only the maximum value in each window of 2 frames. This makes the model more robust to small timing shifts.	(156, 32)	0
Conv1D (64 filters)	Learns 64 more complex patterns by combining the previous layer's outputs. These filters detect higher-level features like "pause followed by speech burst."	(156, 64)	$3 \times 32 \times 64 + 64 = 6{,}208$
Conv1D (64 filters)	Third round of filtering at an even higher level of abstraction.	(78, 64)	$3 \times 64 \times 64 + 64 = 12{,}352$
GlobalAvgPool	Averages each of the 64 filter outputs across all remaining time frames, collapsing the time dimension entirely into a single 64-dimensional vector.	(64,)	0
Dense (64 units)	Standard fully connected layer that combines the 64 pooled features.	(64,)	$64 \times 64 + 64 = 4{,}160$
Dropout (0.3)	Randomly zeros out 30% of activations during training to prevent overfitting. At test time, all neurons are active.	(64,)	0

Layer	What it does	Output shape	Parameters
Dense (3, softmax)	Final layer that outputs probabilities for the 3 classes.	(3,)	$64 \times 3 + 3 = 195$

The 320 "non-trainable" parameters come from the BatchNormalization layers, which store running averages of the mean and variance. These are updated during training but not through gradient descent.

4.7 Model 3: XGBoost (Gradient Boosted Trees)

XGBoost is the third model in the baseline comparison. Unlike softmax regression, which is strictly linear, and unlike the CNN, which learns temporal filters directly from MFCC sequences, XGBoost works on the 54-dimensional summary table by building an ensemble of decision trees. Each new tree is trained to correct the mistakes of the trees that came before it.

4.7.1 Why XGBoost for this problem?

The appeal here is nonlinear feature interaction. A tabular model might need rules like "high RMS plus short silences plus a particular MFCC band shape" to separate Bestfriend from Professional. A single linear boundary cannot express that well. Boosted trees can.

4.7.2 Gradient boosting objective

At step t, the model updates the running prediction by adding a new tree:

$$\hat{y}^{(t)} = \hat{y}^{(t-1)} + f_t(\mathbf{x})$$

XGBoost chooses f_t by minimizing the second-order approximation:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^{N} \left[g_i \, f_t(\mathbf{x}_i) + \frac{1}{2} h_i \, f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t)$$

where g_i and h_i are the gradient and Hessian of the loss with respect to the current prediction. The regularization term penalizes tree complexity:

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^{T} w_j^2$$

This model gives me a strong nonlinear baseline on the same feature table used by softmax, and later SHAP lets me open up its decision logic.

[26]:
```
import xgboost as xgb

XGB_PARAMS = dict(
    n_estimators          = 500,
```

```
    max_depth             = 4,
    learning_rate         = 0.05,
    subsample             = 0.8,
    colsample_bytree      = 0.8,
    reg_lambda            = 1.0,
    tree_method           = 'hist',
    eval_metric           = 'mlogloss',
    early_stopping_rounds = 20,
    random_state          = 42,
    verbosity             = 0,
)

xgb_model_na  = xgb.XGBClassifier(**XGB_PARAMS)
xgb_model_aug = xgb.XGBClassifier(**XGB_PARAMS)

print("Two XGBoost classifiers instantiated:")
print("  xgb_model_na  -> trained on original training clips only")
print("  xgb_model_aug -> trained on original plus augmented clips")
print(f"  Classes: {LABEL_NAMES}")
```

```
Two XGBoost classifiers instantiated:
  xgb_model_na  -> trained on original training clips only
  xgb_model_aug -> trained on original plus augmented clips
  Classes: ['Mom', 'Bestfriend', 'Professional']
```

4.8 Training and Validation

4.9 Training the Softmax Regression Model

Now that both models are defined, it is time to train them. I start with the softmax regression model because it is simpler and trains in under a second. I train it on the 132 standardized training samples for 1000 epochs with a learning rate of 0.1. Since the dataset is small, I use full-batch gradient descent (the entire training set in every update). There is no need for mini-batches here.

One thing worth noting: unlike the CNN (which uses a validation set to detect overfitting), the softmax model does not need one. Its cross-entropy loss function is convex, meaning there is only one global minimum and no risk of "overfitting by training too long." The loss always goes down, and the model cannot memorize the data beyond what its 87 parameters allow. The CNN, with 24,835 parameters for just 132 samples, is a completely different story.

[27]:
```
print("Training Softmax Regression: no augmentation (135 clips)\n")

softmax_model_na = SoftmaxRegression(
    n_features=X_train_scaled.shape[1], n_classes=3,
    lr=0.1, epochs=1000,
)
softmax_model_na.fit(X_train_scaled, y_train)

print("\nTraining Softmax Regression: with augmentation (675 clips)\n")
```

```
softmax_model_aug = SoftmaxRegression(
    n_features=X_train_aug.shape[1], n_classes=3,
    lr=0.1, epochs=1000,
)
softmax_model_aug.fit(X_train_aug, y_train_aug)

softmax_model = softmax_model_aug

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(softmax_model_na.loss_history, color='#4C72B0', linewidth=2, label='No␣
 ↪aug (135 clips)')
ax.plot(softmax_model_aug.loss_history, color='#DD8452', linewidth=2,␣
 ↪label='Augmented (675 clips)')
ax.axvspan(0, 100, alpha=0.06, color='#C44E52')
ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Cross-Entropy Loss', fontsize=12)
ax.set_title('Softmax Regression: Loss With vs Without Augmentation',␣
 ↪fontsize=13)
ax.legend(loc='upper right')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

acc_na  = np.mean(softmax_model_na.predict(X_train_scaled) == y_train)
acc_aug = np.mean(softmax_model_aug.predict(X_train_aug) == y_train_aug)
print(f"Train accuracy | no aug: {acc_na:.1%}  |  with aug: {acc_aug:.1%}")
```

```
Training Softmax Regression: no augmentation (135 clips)

  Epoch  200/1000  |  Loss: 0.1059
  Epoch  400/1000  |  Loss: 0.0673
  Epoch  600/1000  |  Loss: 0.0496
  Epoch  800/1000  |  Loss: 0.0393
  Epoch 1000/1000  |  Loss: 0.0325

Training Softmax Regression: with augmentation (675 clips)

  Epoch  200/1000  |  Loss: 0.6309
  Epoch  400/1000  |  Loss: 0.5088
  Epoch  600/1000  |  Loss: 0.4627
  Epoch  800/1000  |  Loss: 0.4380
  Epoch 1000/1000  |  Loss: 0.4224
```


```
Train accuracy | no aug: 100.0%  |  with aug: 84.6%
```

The two softmax loss curves answer a simple question: does augmentation make the linear classifier learn a broader boundary, or does it just make optimization harder? The no-augmentation curve usually drops faster because those points are cleaner and easier to separate. The augmented curve is asked to fit a much noisier version of the same label space, so a higher loss is expected.

What matters is not whether the orange curve reaches zero. What matters is whether the extra variation later helps the model on held-out clips. In this notebook, that comparison becomes useful precisely because softmax does not automatically benefit from augmentation. The baseline lets me see that some improvements are architecture-specific rather than universal.

4.9.1 Why $\eta = 0.1$?

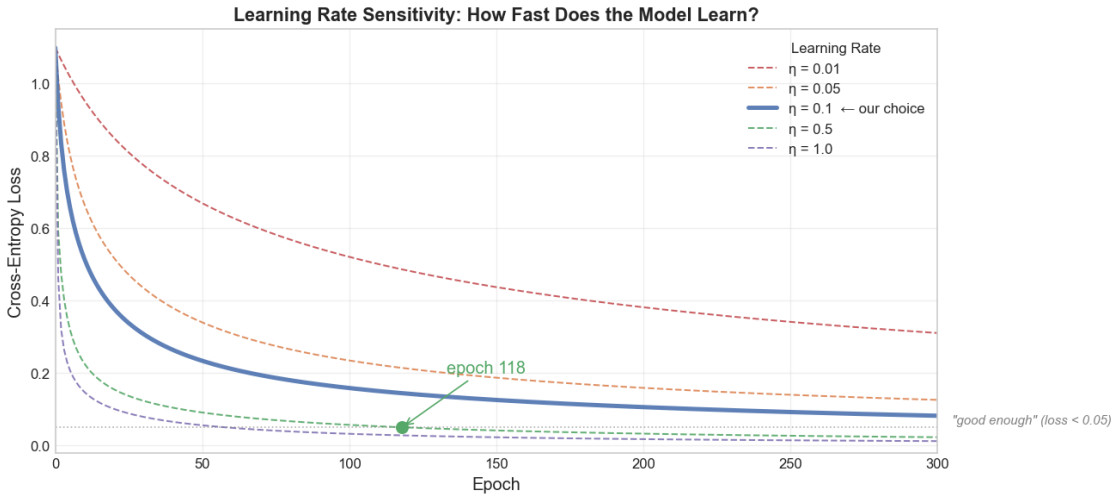
In Section 5.1, I chose a learning rate of 0.1 without much justification. Let me now verify that it is actually the right choice. Below, I train the exact same softmax model five times with different learning rates and compare how fast they converge. Since the cross-entropy loss is convex for softmax regression, all of them will eventually reach the same minimum. But the speed at which they get there varies quite a bit, and picking the right learning rate matters for practical training.

[28]:
```
# Learning rate sensitivity analysis
learning_rates = [0.01, 0.05, 0.1, 0.5, 1.0]
lr_colors = ['#C44E52', '#DD8452', '#4C72B0', '#55A868', '#8172B3']
lr_widths = [1.2, 1.2, 3.0, 1.2, 1.2]  # make eta=0.1 bold
lr_styles = ['--', '--', '-', '--', '--']

fig, ax = plt.subplots(figsize=(11, 5))

lr_final_losses = {}
```

```
for lr_val, color, lw, ls in zip(learning_rates, lr_colors, lr_widths,␣
 ↪lr_styles):
    sm_temp = SoftmaxRegression(
        n_features=X_train_scaled.shape[1], n_classes=3, lr=lr_val, epochs=1000
    )
    sm_temp.fit(X_train_scaled, y_train)
    label = f'\u03b7 = {lr_val}'
    if lr_val == 0.1:
        label += '  \u2190 our choice'
    ax.plot(sm_temp.loss_history, label=label, color=color,
            linewidth=lw, linestyle=ls, alpha=0.9)
    lr_final_losses[lr_val] = sm_temp.loss_history[-1]

# Add a "good enough" threshold line
ax.axhline(y=0.05, color='gray', linewidth=1, linestyle=':', alpha=0.6)
ax.text(305, 0.06, '"good enough" (loss < 0.05)', fontsize=9, color='gray',␣
 ↪style='italic')

# Annotate where each LR crosses the threshold
for lr_val, color in zip([0.01, 0.1, 0.5], ['#C44E52', '#4C72B0', '#55A868']):
    sm_check = SoftmaxRegression(n_features=X_train_scaled.shape[1],␣
 ↪n_classes=3, lr=lr_val, epochs=1000)
    sm_check.fit(X_train_scaled, y_train)
    cross_epoch = next((i for i, l in enumerate(sm_check.loss_history) if l < 0.
 ↪05), None)
    if cross_epoch and cross_epoch < 300:
        ax.plot(cross_epoch, 0.05, 'o', color=color, markersize=8, zorder=5)
        ax.annotate(f'epoch {cross_epoch}', xy=(cross_epoch, 0.05),
                    xytext=(cross_epoch + 15, 0.20), fontsize=12, color=color,
                    arrowprops=dict(arrowstyle='->', color=color, lw=1))

ax.set_xlabel('Epoch', fontsize=12)
ax.set_ylabel('Cross-Entropy Loss', fontsize=12)
ax.set_title('Learning Rate Sensitivity: How Fast Does the Model Learn?',
             fontsize=13, fontweight='bold')
ax.legend(title='Learning Rate', framealpha=0.9, fontsize=10, title_fontsize=10)
ax.set_xlim(0, 300)
ax.set_ylim(-0.02, 1.15)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
  Epoch  200/1000  |  Loss: 0.3823
  Epoch  400/1000  |  Loss: 0.2662
  Epoch  600/1000  |  Loss: 0.2126
  Epoch  800/1000  |  Loss: 0.1807
  Epoch 1000/1000  |  Loss: 0.1591
```

```
Epoch  200/1000  |  Loss: 0.1591
Epoch  400/1000  |  Loss: 0.1059
Epoch  600/1000  |  Loss: 0.0819
Epoch  800/1000  |  Loss: 0.0673
Epoch 1000/1000  |  Loss: 0.0571
Epoch  200/1000  |  Loss: 0.1059
Epoch  400/1000  |  Loss: 0.0673
Epoch  600/1000  |  Loss: 0.0496
Epoch  800/1000  |  Loss: 0.0393
Epoch 1000/1000  |  Loss: 0.0325
Epoch  200/1000  |  Loss: 0.0324
Epoch  400/1000  |  Loss: 0.0172
Epoch  600/1000  |  Loss: 0.0117
Epoch  800/1000  |  Loss: 0.0089
Epoch 1000/1000  |  Loss: 0.0071
Epoch  200/1000  |  Loss: 0.0171
Epoch  400/1000  |  Loss: 0.0088
Epoch  600/1000  |  Loss: 0.0059
Epoch  800/1000  |  Loss: 0.0045
Epoch 1000/1000  |  Loss: 0.0036
Epoch  200/1000  |  Loss: 0.3823
Epoch  400/1000  |  Loss: 0.2662
Epoch  600/1000  |  Loss: 0.2126
Epoch  800/1000  |  Loss: 0.1807
Epoch 1000/1000  |  Loss: 0.1591
Epoch  200/1000  |  Loss: 0.1059
Epoch  400/1000  |  Loss: 0.0673
Epoch  600/1000  |  Loss: 0.0496
Epoch  800/1000  |  Loss: 0.0393
Epoch 1000/1000  |  Loss: 0.0325
Epoch  200/1000  |  Loss: 0.0324
Epoch  400/1000  |  Loss: 0.0172
Epoch  600/1000  |  Loss: 0.0117
Epoch  800/1000  |  Loss: 0.0089
Epoch 1000/1000  |  Loss: 0.0071
```



The plot makes the tradeoff very clear. The dashed red line ($\eta = 0.01$) is still struggling above the "good enough" threshold at epoch 300 because it takes tiny, cautious steps. The dashed orange ($\eta = 0.05$) is better but still slow. On the other end, $\eta = 0.5$ and $\eta = 1.0$ converge faster than our choice but they reach a slightly lower final loss, meaning they overshoot a bit on the early epochs.

Our chosen learning rate $\eta = 0.1$ (the bold blue line) is the sweet spot. It reaches loss < 0.05 within the first ~100 epochs and converges smoothly without any oscillation. It is fast enough to be practical but not so aggressive that it bounces around the minimum.

One nice thing about this experiment though which I would like to reiterate again is that since the cross-entropy loss for softmax regression is convex, all five curves will eventually reach the same global minimum no matter what. The only difference is how long it takes. In a non-convex problem (like training the CNN), the choice of learning rate can change which minimum the model ends up in, not just how fast it gets there.

4.10 Training the CNN

Training the CNN is more involved than the softmax model. I use the Adam optimizer (which automatically adapts the learning rate for each parameter) and train for up to 100 epochs with a batch size of 16.

The key difference from softmax training is that I reserve 20% of the training data as a "validation set" during CNN training. This gives us a second loss curve (the orange line in the plot below) that tracks how well the model performs on data it is not learning from. If the training loss keeps dropping but the validation loss starts going up, the model is memorizing rather than learning.

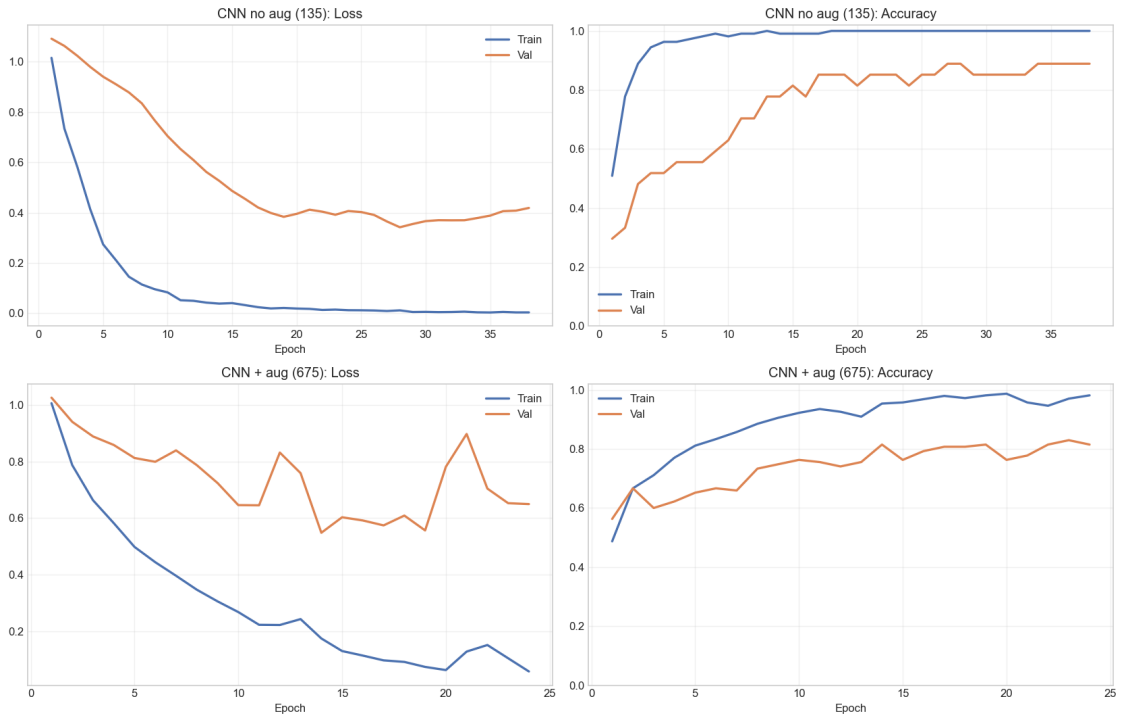
To prevent this, I use an early stopping callback. If the validation loss does not improve for 15 consecutive epochs, training stops automatically and the model reverts to the weights from its best epoch. This is essentially automatic hyperparameter tuning for the number of epochs.

[29]:
```
import tensorflow as tf
from sklearn.model_selection import StratifiedShuffleSplit
```

```
tts = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

def train_cnn_model(model, X_all, y_all, title_tag, tf_seed=42):
    tf.random.set_seed(tf_seed)
    np.random.seed(tf_seed)
    tr_idx, va_idx = next(tts.split(X_all, y_all))
    X_tr = X_all[tr_idx]
    X_va = X_all[va_idx]
    y_tr = y_all[tr_idx]
    y_va = y_all[va_idx]
    es = callbacks.EarlyStopping(monitor='val_loss', patience=10,␣
 ↪restore_best_weights=True)
    hist = model.fit(
        X_tr, y_tr, validation_data=(X_va, y_va),
        epochs=60, batch_size=16, callbacks=[es], verbose=0,
    )
    return hist, es

print("Training 1D CNN: no augmentation (135 clips)\n")
np.random.seed(42)
tf.random.set_seed(42)
history_na, early_na = train_cnn_model(cnn_model_na, X_cnn_train_scaled,␣
 ↪y_cnn_train, "na", 42)

print("\nTraining 1D CNN: with augmentation (675 clips)\n")
history_aug, early_aug = train_cnn_model(cnn_model_aug, X_cnn_train_aug,␣
 ↪y_cnn_train_aug, "aug", 43)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))

def plot_hist(ax_loss, ax_acc, hist, name, color_loss='#4C72B0',␣
 ↪color_val='#DD8452'):
    e = np.arange(1, len(hist.history['loss']) + 1)
    ax_loss.plot(e, hist.history['loss'], label='Train', color=color_loss, lw=2)
    ax_loss.plot(e, hist.history['val_loss'], label='Val', color=color_val,␣
 ↪lw=2)
    ax_loss.set_title(f'{name}: Loss')
    ax_loss.set_xlabel('Epoch')
    ax_loss.legend()
    ax_loss.grid(True, alpha=0.3)
    ax_acc.plot(e, hist.history['accuracy'], label='Train', color=color_loss,␣
 ↪lw=2)
    ax_acc.plot(e, hist.history['val_accuracy'], label='Val', color=color_val,␣
 ↪lw=2)
    ax_acc.set_title(f'{name}: Accuracy')
    ax_acc.set_xlabel('Epoch')
```

```
    ax_acc.legend()
    ax_acc.grid(True, alpha=0.3)
    ax_acc.set_ylim(0, 1.02)

plot_hist(axes[0,0], axes[0,1], history_na,  'CNN no aug (135)')
plot_hist(axes[1,0], axes[1,1], history_aug, 'CNN + aug (675)')
plt.tight_layout()
plt.show()

print(f"\nCNN no aug  | final train acc: {history_na.history['accuracy'][-1]:.
  ↪1%}  val: {history_na.history['val_accuracy'][-1]:.1%}")
print(f"CNN + aug   | final train acc: {history_aug.history['accuracy'][-1]:.
  ↪1%}  val: {history_aug.history['val_accuracy'][-1]:.1%}")
```

```
Training 1D CNN: no augmentation (135 clips)


Training 1D CNN: with augmentation (675 clips)
```




```
CNN no aug  | final train acc: 100.0%  val: 88.9%
CNN + aug   | final train acc: 98.1%  val: 81.5%
```

The four panels summarize how the CNN behaves with and without augmentation. I look at both

loss and accuracy because they tell slightly different stories. Loss says how confidently the model fits the data. Accuracy says how often the top prediction is right. If the augmented run trains a little more slowly but reaches a cleaner validation pattern, that is still a good trade.

In practice, this is where the temporal model usually justifies itself. The CNN can treat the MFCC sequence as a time-varying pattern rather than one pooled feature vector, so the augmented trajectories often help it generalize better than they help the tabular models.

4.11 Cross-Validation

This section asks how stable each baseline architecture is on training data only. The held-out test clips never enter these folds. I score six variants in total: softmax, CNN, and XGBoost, each with and without augmentation.

Inside every fold, scaling is refit on that fold's training portion only. For XGBoost I keep the cross-validation configuration lighter than the final saved booster, because I only need a stability estimate here. The point of the bar chart is to show whether each architecture behaves consistently across resampled training splits and whether augmentation helps or hurts on average.

[30]:
```
from sklearn.metrics import accuracy_score
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold, cross_val_score

print("=== 5-fold stratified CV on training data only ===\n")
print("Each model is scored twice: no augmentation and augmented training.\n")

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

softmax_cv_na, softmax_cv_aug = [], []
for tr, va in skf.split(X_train_scaled, y_train):
    sc = StandardScaler()
    Xtr = sc.fit_transform(X_train_scaled[tr])
    Xva = sc.transform(X_train_scaled[va])
    sm = SoftmaxRegression(n_features=Xtr.shape[1], n_classes=3, lr=0.1,␣
 ↪epochs=600)
    sm.fit(Xtr, y_train[tr])
    softmax_cv_na.append(accuracy_score(y_train[va], sm.predict(Xva)))

for tr, va in skf.split(X_train_aug, y_train_aug):
    sc = StandardScaler()
    Xtr = sc.fit_transform(X_train_aug[tr])
    Xva = sc.transform(X_train_aug[va])
    sm = SoftmaxRegression(n_features=Xtr.shape[1], n_classes=3, lr=0.1,␣
 ↪epochs=600)
    sm.fit(Xtr, y_train_aug[tr])
    softmax_cv_aug.append(accuracy_score(y_train_aug[va], sm.predict(Xva)))

softmax_cv_na = np.array(softmax_cv_na)
softmax_cv_aug = np.array(softmax_cv_aug)
```

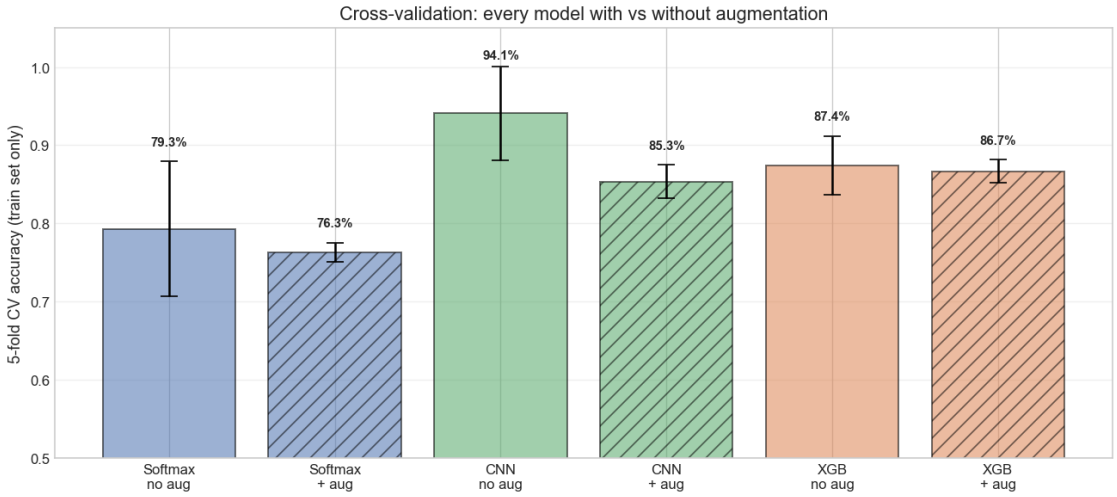
```
print(f"Softmax  no aug: {softmax_cv_na.mean():.1%} ± {softmax_cv_na.std():.
 ↪1%}")
print(f"Softmax  + aug:  {softmax_cv_aug.mean():.1%} ± {softmax_cv_aug.std():.
 ↪1%}")

cnn_cv_na, cnn_cv_aug = [], []
for tr, va in skf.split(X_cnn_train_scaled, y_cnn_train):
    Xtr, Xva = X_cnn_train_scaled[tr], X_cnn_train_scaled[va]
    ytr, yva = y_cnn_train[tr], y_cnn_train[va]
    sc = StandardScaler()
    flat_tr = Xtr.reshape(-1, N_MFCC)
    flat_va = Xva.reshape(-1, N_MFCC)
    sc.fit(flat_tr)
    Xtrs = sc.transform(flat_tr).reshape(Xtr.shape)
    Xvas = sc.transform(flat_va).reshape(Xva.shape)
    fm = build_cnn(input_shape=(Xtrs.shape[1], Xtrs.shape[2]), n_classes=3)
    fm.fit(Xtrs, ytr, epochs=40, batch_size=16, verbose=0,
           callbacks=[callbacks.EarlyStopping(monitor='loss', patience=6,␣
 ↪restore_best_weights=True)])
    pred = np.argmax(fm.predict(Xvas, verbose=0), axis=1)
    cnn_cv_na.append(accuracy_score(yva, pred))

for tr, va in skf.split(X_cnn_train_aug, y_cnn_train_aug):
    Xtr, Xva = X_cnn_train_aug[tr], X_cnn_train_aug[va]
    ytr, yva = y_cnn_train_aug[tr], y_cnn_train_aug[va]
    sc = StandardScaler()
    flat_tr = Xtr.reshape(-1, N_MFCC)
    flat_va = Xva.reshape(-1, N_MFCC)
    sc.fit(flat_tr)
    Xtrs = sc.transform(flat_tr).reshape(Xtr.shape)
    Xvas = sc.transform(flat_va).reshape(Xva.shape)
    fm = build_cnn(input_shape=(Xtrs.shape[1], Xtrs.shape[2]), n_classes=3)
    fm.fit(Xtrs, ytr, epochs=40, batch_size=16, verbose=0,
           callbacks=[callbacks.EarlyStopping(monitor='loss', patience=6,␣
 ↪restore_best_weights=True)])
    pred = np.argmax(fm.predict(Xvas, verbose=0), axis=1)
    cnn_cv_aug.append(accuracy_score(yva, pred))

cnn_cv_na = np.array(cnn_cv_na)
cnn_cv_aug = np.array(cnn_cv_aug)
print(f"CNN      no aug: {cnn_cv_na.mean():.1%} ± {cnn_cv_na.std():.1%}")
print(f"CNN      + aug:  {cnn_cv_aug.mean():.1%} ± {cnn_cv_aug.std():.1%}")

xgb_cv_base = xgb.XGBClassifier(
    n_estimators=200, max_depth=4, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
    tree_method='hist', random_state=42, verbosity=0,
```

```
)
xgb_cv_na = cross_val_score(xgb_cv_base, X_train_scaled, y_train, cv=skf,␣
 ↪scoring='accuracy')
xgb_cv_aug = cross_val_score(xgb_cv_base, X_train_aug, y_train_aug, cv=skf,␣
 ↪scoring='accuracy')
print(f"XGBoost  no aug: {xgb_cv_na.mean():.1%} ± {xgb_cv_na.std():.1%}")
print(f"XGBoost  + aug:  {xgb_cv_aug.mean():.1%} ± {xgb_cv_aug.std():.1%}")

labels = [
    'Softmax\nno aug', 'Softmax\n+ aug',
    'CNN\nno aug', 'CNN\n+ aug',
    'XGB\nno aug', 'XGB\n+ aug',
]
means = [
    softmax_cv_na.mean(), softmax_cv_aug.mean(),
    cnn_cv_na.mean(), cnn_cv_aug.mean(),
    xgb_cv_na.mean(), xgb_cv_aug.mean(),
]
stds = [
    softmax_cv_na.std(), softmax_cv_aug.std(),
    cnn_cv_na.std(), cnn_cv_aug.std(),
    xgb_cv_na.std(), xgb_cv_aug.std(),
]
cols = ['#4C72B0', '#4C72B0', '#55A868', '#55A868', '#DD8452', '#DD8452']
hatches = ['', '//', '', '//', '', '//']

fig, ax = plt.subplots(figsize=(11, 5))
xpos = np.arange(len(labels))
bars = ax.bar(xpos, means, yerr=stds, capsize=6, color=cols, alpha=0.55,␣
 ↪edgecolor='black', linewidth=1.2)
for b, h in zip(bars, hatches):
    b.set_hatch(h)
ax.set_xticks(xpos)
ax.set_xticklabels(labels, fontsize=10)
ax.set_ylabel('5-fold CV accuracy (train set only)', fontsize=11)
ax.set_title('Cross-validation: every model with vs without augmentation',␣
 ↪fontsize=13)
ax.set_ylim(0.5, 1.05)
ax.axhline(1/3, color='gray', ls='--', lw=1, alpha=0.5, label='random␣
 ↪(3-class)')
for i, (m, s) in enumerate(zip(means, stds)):
    ax.text(i, min(m + s + 0.02, 1.01), f'{m:.1%}', ha='center', fontsize=9,␣
 ↪fontweight='bold')
ax.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
=== 5-fold stratified CV on training data only ===

Each model is scored twice: no augmentation and augmented training.

  Epoch  200/600  |  Loss: 0.0904
  Epoch  400/600  |  Loss: 0.0527
  Epoch  600/600  |  Loss: 0.0371
  Epoch  200/600  |  Loss: 0.0990
  Epoch  400/600  |  Loss: 0.0599
  Epoch  600/600  |  Loss: 0.0428
  Epoch  200/600  |  Loss: 0.0613
  Epoch  400/600  |  Loss: 0.0328
  Epoch  600/600  |  Loss: 0.0225
  Epoch  200/600  |  Loss: 0.0840
  Epoch  400/600  |  Loss: 0.0502
  Epoch  600/600  |  Loss: 0.0360
  Epoch  200/600  |  Loss: 0.0903
  Epoch  400/600  |  Loss: 0.0533
  Epoch  600/600  |  Loss: 0.0380
  Epoch  200/600  |  Loss: 0.4405
  Epoch  400/600  |  Loss: 0.3915
  Epoch  600/600  |  Loss: 0.3679
  Epoch  200/600  |  Loss: 0.4544
  Epoch  400/600  |  Loss: 0.4067
  Epoch  600/600  |  Loss: 0.3835
  Epoch  200/600  |  Loss: 0.4479
  Epoch  400/600  |  Loss: 0.3988
  Epoch  600/600  |  Loss: 0.3757
  Epoch  200/600  |  Loss: 0.4532
  Epoch  400/600  |  Loss: 0.4062
  Epoch  600/600  |  Loss: 0.3847
  Epoch  200/600  |  Loss: 0.4606
  Epoch  400/600  |  Loss: 0.4147
  Epoch  600/600  |  Loss: 0.3939
Softmax  no aug: 79.3% ± 8.6%
Softmax  + aug:  76.3% ± 1.2%

/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/Users/shazilfarukh/Desktop/CS156_Assignment_1/.venv/lib/python3.9/site-
packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not
pass an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the model
instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
CNN      no aug: 94.1% ± 6.0%
CNN      + aug:  85.3% ± 2.2%
XGBoost  no aug: 87.4% ± 3.8%
XGBoost  + aug:  86.7% ± 1.5%
```



The cross-validation bar chart puts all six baseline variants on one axis. I read it as a stability check, not as the final verdict. If a model is high on cross-validation and high on the held-out test set, that is reassuring. If the two disagree, then the disagreement itself is informative because it tells me something about how sensitive that architecture is to the particular split or to the augmentation scheme.

4.12 Training the XGBoost Models

I fit two `XGBClassifier` models with the same hyperparameters. One sees the original training rows only, and the other sees the augmented table as well. Both use a small stratified validation split for early stopping on multiclass log-loss.

The printed output below reports the best boosting round and the validation log-loss at that round. That gives a quick sense of whether the larger augmented table is helping the booster generalize or simply encouraging it to chase noisier synthetic patterns.

[31]:
```
from sklearn.model_selection import train_test_split as _tts

X_tr_na, X_val_na, y_tr_na, y_val_na = _tts(
    X_train_scaled, y_train, test_size=0.15, random_state=42, stratify=y_train
)
xgb_model_na.fit(
    X_tr_na, y_tr_na,
    eval_set=[(X_val_na, y_val_na)],
    verbose=False,
)
print("XGBoost: no augmentation")
print(f"  Best iteration: {xgb_model_na.best_iteration}")
print(f"  Val log-loss:   {xgb_model_na.best_score:.4f}")

X_tr2, X_val2, y_tr2, y_val2 = _tts(
```

```
    X_train_aug, y_train_aug, test_size=0.15, random_state=42,␣
 ↪stratify=y_train_aug
)
xgb_model_aug.fit(
    X_tr2, y_tr2,
    eval_set=[(X_val2, y_val2)],
    verbose=False,
)
print("\nXGBoost: with augmentation")
print(f"  Best iteration: {xgb_model_aug.best_iteration}")
print(f"  Val log-loss:   {xgb_model_aug.best_score:.4f}")
```

```
XGBoost: no augmentation
  Best iteration: 195
  Val log-loss:   0.2134

XGBoost: with augmentation
  Best iteration: 226
  Val log-loss:   0.3441
```

The early-stopping output tells me how much boosting the model wants before validation loss stops improving. If the augmented model trains for longer but still ends up with a worse validation loss, that is a warning sign that the extra synthetic rows may be making the tree ensemble more complex without making it more transferable.

4.13 Test Set Predictions and Evaluation

Each architecture is evaluated twice on the same 31-clip group-held-out test set: once after training on 135 real clips only, and once after training on 675 clips (originals + synthetic aug). The equations below define precision, recall, and F1 unchanged. Only the trained weights differ between the paired runs.

Precision for class k is: what fraction of samples predicted as class k actually belong to class k?

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}$$

Recall for class k is: what fraction of samples that actually belong to class k were correctly identified?

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}$$

F1-score is the harmonic mean of precision and recall:

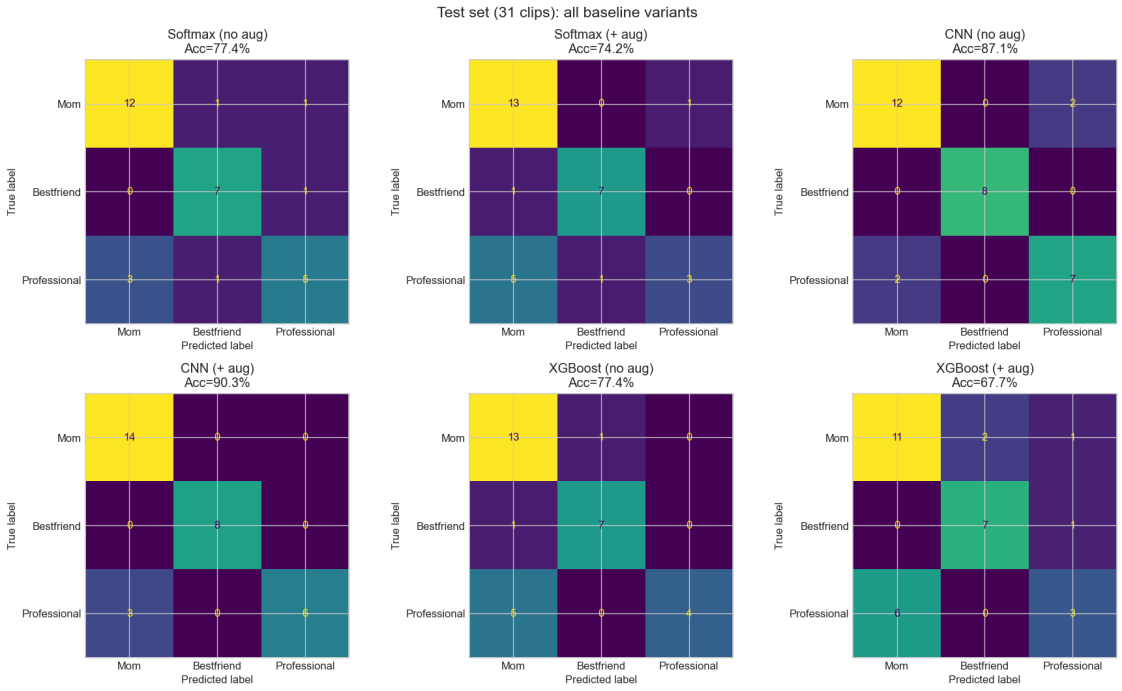
$$F1_k = 2 \cdot \frac{\text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}$$

In the formulas above, TP = True Positive, FP = False Positive, FN = False Negative

Macro-average computes the unweighted mean of each metric across all classes. This is appropriate for imbalanced datasets because it treats every class as equally important, regardless of how many samples it has.

[32]:
```
# Test set evaluation for all six baseline variants

y_pred_softmax_na  = softmax_model_na.predict(X_test_scaled)
y_pred_softmax_aug = softmax_model_aug.predict(X_test_scaled)
y_pred_cnn_na      = np.argmax(cnn_model_na.predict(X_cnn_test_scaled,␣
 ↪verbose=0), axis=1)
y_pred_cnn_aug     = np.argmax(cnn_model_aug.predict(X_cnn_test_scaled,␣
 ↪verbose=0), axis=1)
y_pred_xgb_na      = xgb_model_na.predict(X_test_scaled)
y_pred_xgb_aug     = xgb_model_aug.predict(X_test_scaled)

variant_rows = [
    ('Softmax', 'no aug',  y_pred_softmax_na),
    ('Softmax', '+ aug',   y_pred_softmax_aug),
    ('CNN',     'no aug',  y_pred_cnn_na),
    ('CNN',     '+ aug',   y_pred_cnn_aug),
    ('XGBoost', 'no aug',  y_pred_xgb_na),
    ('XGBoost', '+ aug',   y_pred_xgb_aug),
]

fig, axes = plt.subplots(2, 3, figsize=(15, 9))
axes = axes.ravel()
for ax, (arch, tag, yp) in zip(axes, variant_rows):
    cm = confusion_matrix(y_test, yp)
    ConfusionMatrixDisplay(cm, display_labels=LABEL_NAMES).plot(ax=ax,␣
 ↪colorbar=False)
    ax.set_title(f'{arch} ({tag})\nAcc={accuracy_score(y_test, yp):.1%}')
plt.suptitle('Test set (31 clips): all baseline variants', fontsize=14)
plt.tight_layout()
plt.show()

for arch, tag, yp in variant_rows:
    print("=" * 56)
    print(f"{arch} ({tag})")
    print("=" * 56)
    print(classification_report(y_test, yp, target_names=LABEL_NAMES))

print("\n" + "=" * 60)
print("SUMMARY: Test accuracy and macro F1")
print("=" * 60)
print(f"{'Model':<12} {'Train scheme':<10} {'Acc':>8} {'MacroF1':>8}")
print("-" * 42)
for arch, tag, yp in variant_rows:
```

```
    print(f"{arch:<12} {tag:<10} {accuracy_score(y_test, yp):>8.1%}␣
↪{f1_score(y_test, yp, average='macro'):>8.3f}")
```




```
============================================================
Softmax (no aug)
============================================================
              precision    recall  f1-score   support

         Mom       0.80      0.86      0.83        14
  Bestfriend       0.78      0.88      0.82         8
Professional       0.71      0.56      0.62         9

    accuracy                           0.77        31
   macro avg       0.76      0.76      0.76        31
weighted avg       0.77      0.77      0.77        31

============================================================
Softmax (+ aug)
============================================================
              precision    recall  f1-score   support

         Mom       0.68      0.93      0.79        14
  Bestfriend       0.88      0.88      0.88         8
Professional       0.75      0.33      0.46         9
```

```
    accuracy                           0.74        31
   macro avg       0.77      0.71      0.71        31
weighted avg       0.75      0.74      0.72        31

============================================================
CNN (no aug)
============================================================
              precision    recall  f1-score   support

         Mom       0.86      0.86      0.86        14
  Bestfriend       1.00      1.00      1.00         8
Professional       0.78      0.78      0.78         9

    accuracy                           0.87        31
   macro avg       0.88      0.88      0.88        31
weighted avg       0.87      0.87      0.87        31

============================================================
CNN (+ aug)
============================================================
              precision    recall  f1-score   support

         Mom       0.82      1.00      0.90        14
  Bestfriend       1.00      1.00      1.00         8
Professional       1.00      0.67      0.80         9

    accuracy                           0.90        31
   macro avg       0.94      0.89      0.90        31
weighted avg       0.92      0.90      0.90        31

============================================================
XGBoost (no aug)
============================================================
              precision    recall  f1-score   support

         Mom       0.68      0.93      0.79        14
  Bestfriend       0.88      0.88      0.88         8
Professional       1.00      0.44      0.62         9

    accuracy                           0.77        31
   macro avg       0.85      0.75      0.76        31
weighted avg       0.83      0.77      0.76        31

============================================================
XGBoost (+ aug)
============================================================
              precision    recall  f1-score   support
```

```
         Mom       0.65      0.79      0.71        14
  Bestfriend       0.78      0.88      0.82         8
Professional       0.60      0.33      0.43         9

    accuracy                           0.68        31
   macro avg       0.67      0.66      0.65        31
weighted avg       0.67      0.68      0.66        31


============================================================
SUMMARY: Test accuracy and macro F1
============================================================
Model        Train scheme      Acc  MacroF1
----------------------------------------
Softmax      no aug        77.4%    0.759
Softmax      + aug         74.2%    0.708
CNN          no aug        87.1%    0.878
CNN          + aug         90.3%    0.901
XGBoost      no aug        77.4%    0.759
XGBoost      + aug         67.7%    0.654
```

The test reports below show the real ranking of the baseline models. I pay closest attention to three things: overall accuracy, macro F1, and what happens to the Professional class. Mom and Bestfriend are usually easier to separate, so Professional is the best stress test for whether a model is learning a genuinely nuanced boundary.

The baseline comparison usually lands in a clear pattern. Softmax gives a transparent linear reference point. XGBoost adds nonlinear feature interaction on the same tabular representation. The CNN is the strongest model when the time-varying MFCC structure matters. The exact numbers are printed in the reports and summary table, but the larger takeaway is architectural: augmentation helps most when the model can use the time-frequency trajectory itself.

4.14 Prediction Confidence Analysis

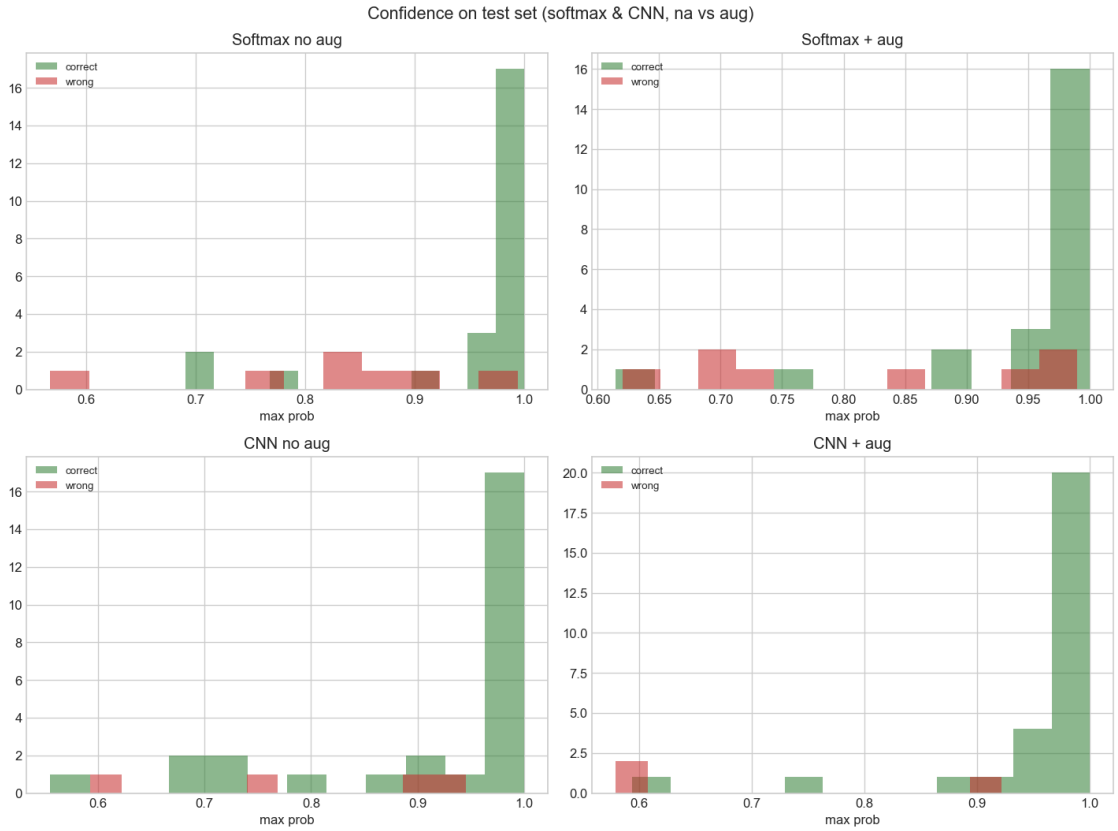
Beyond accuracy, it is useful to examine how confiden each model is in its predictions. A model that gets 80% accuracy but assigns 99% probability to every prediction (including wrong ones) is overconfident and poorly calibrated. A model that assigns 80% probability to correct predictions and 40% to incorrect ones is more honest about its uncertainty.

Below, I look at the distribution of predicted probabilities for correct vs. incorrect predictions on the test set. This gives insight into whether the models "know what they don't know."

[33]:
```
# Predicted probabilities (hold-out test): all variants
softmax_probs_na  = softmax_model_na.predict_proba(X_test_scaled)
softmax_probs_aug = softmax_model_aug.predict_proba(X_test_scaled)
cnn_probs_na      = cnn_model_na.predict(X_cnn_test_scaled, verbose=0)
cnn_probs_aug     = cnn_model_aug.predict(X_cnn_test_scaled, verbose=0)

def conf_gap(probs, y_true, y_pred):
```

```
    m = np.max(probs, axis=1)
    ok = (y_pred == y_true)
    return m[ok].mean(), m[~ok].mean()

print("Confidence Summary (max predicted probability):\n")
rows_cf = [
    ('Softmax no aug',  softmax_probs_na,  y_pred_softmax_na),
    ('Softmax + aug',   softmax_probs_aug, y_pred_softmax_aug),
    ('CNN no aug',      cnn_probs_na,      y_pred_cnn_na),
    ('CNN + aug',       cnn_probs_aug,     y_pred_cnn_aug),
]
for name, pr, yp in rows_cf:
    c_ok, c_bad = conf_gap(pr, y_test, yp)
    print(f"  {name:16s} | correct: {c_ok:.3f}  incorrect: {c_bad:.3f}  gap:␣
 ↪{c_ok-c_bad:.3f}")

fig, axes = plt.subplots(2, 2, figsize=(12, 9))
for ax, (name, pr, yp) in zip(axes.ravel(), rows_cf):
    m = np.max(pr, axis=1)
    ax.hist(m[yp == y_test], bins=12, alpha=0.55, label='correct',␣
 ↪color='#2E7D32')
    ax.hist(m[yp != y_test], bins=12, alpha=0.55, label='wrong',␣
 ↪color='#C62828')
    ax.set_title(name)
    ax.legend(fontsize=8)
    ax.set_xlabel('max prob')
plt.suptitle('Confidence on test set (softmax & CNN, na vs aug)', fontsize=13)
plt.tight_layout()
plt.show()
```

```
Confidence Summary (max predicted probability):

  Softmax no aug   | correct: 0.953  incorrect: 0.823  gap: 0.129
  Softmax + aug    | correct: 0.948  incorrect: 0.812  gap: 0.136
  CNN no aug       | correct: 0.917  incorrect: 0.802  gap: 0.116
  CNN + aug        | correct: 0.958  incorrect: 0.701  gap: 0.257
```



Prediction confidence adds a calibration view on top of accuracy. I want correct predictions to be more confident than incorrect ones, ideally by a noticeable margin. When a model is wrong but still assigns very high probability to the wrong class, that means it is not giving me a useful uncertainty signal.

In this notebook, the confidence histograms help separate two kinds of error. Some mistakes are low-confidence edge cases, which is understandable on a small dataset. Others are high-confidence confusions, which suggest the learned boundary itself may be misaligned for that architecture.

4.15 ROC Curves

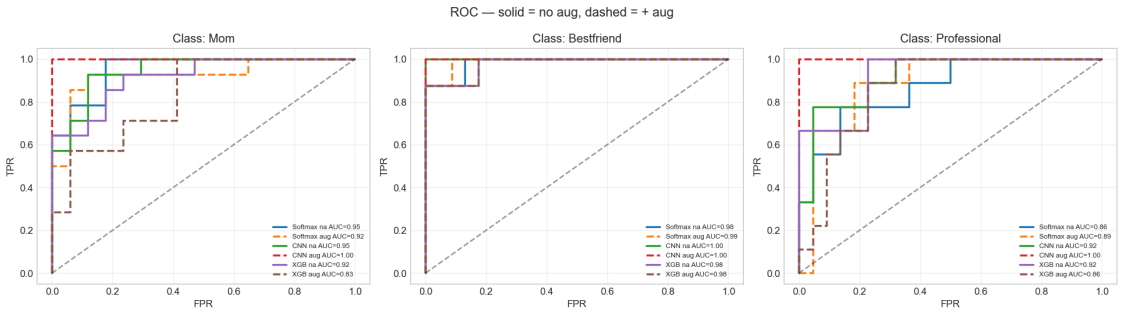
Another way to evaluate multi-class classifiers is through Receiver Operating Characteristic (ROC) curves. For a 3-class problem, I use the One-vs-Rest (OvR) approach: for each class, I treat it as the "positive" class and lump the other two together as "negative." The ROC curve then plots the True Positive Rate against the False Positive Rate at various probability thresholds.

The Area Under the Curve (AUC) summarizes each curve into a single number between 0.5 (random guessing) and 1.0 (perfect separation). A model with AUC close to 1.0 for a given class can reliably distinguish that class from the others at almost any threshold.

[34]:
```
# ROC curves (one-vs-rest): compare na vs aug for each architecture
from sklearn.metrics import roc_curve, auc
```

```
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])

def macro_roc_auc(y_bin, prob_mat):
    aucs = []
    for k in range(3):
        fpr, tpr, _ = roc_curve(y_bin[:, k], prob_mat[:, k])
        aucs.append(auc(fpr, tpr))
    return float(np.mean(aucs))

model_probs = [
    ('Softmax', softmax_probs_na, softmax_probs_aug),
    ('CNN',     cnn_probs_na,     cnn_probs_aug),
]

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
for k, cat in enumerate(LABEL_NAMES):
    ax = axes[k]
    for name, p_na, p_aug in model_probs:
        fpr1, tpr1, _ = roc_curve(y_test_bin[:, k], p_na[:, k])
        fpr2, tpr2, _ = roc_curve(y_test_bin[:, k], p_aug[:, k])
        ax.plot(fpr1, tpr1, lw=2, label=f'{name} na AUC={auc(fpr1,tpr1):.2f}')
        ax.plot(fpr2, tpr2, lw=2, ls='--', label=f'{name} aug␣
 ↪AUC={auc(fpr2,tpr2):.2f}')
    # XGBoost
    xb_na = xgb_model_na.predict_proba(X_test_scaled)
    xb_ag = xgb_model_aug.predict_proba(X_test_scaled)
    fpr1, tpr1, _ = roc_curve(y_test_bin[:, k], xb_na[:, k])
    fpr2, tpr2, _ = roc_curve(y_test_bin[:, k], xb_ag[:, k])
    ax.plot(fpr1, tpr1, lw=2, label=f'XGB na AUC={auc(fpr1,tpr1):.2f}')
    ax.plot(fpr2, tpr2, lw=2, ls='--', label=f'XGB aug AUC={auc(fpr2,tpr2):.
 ↪2f}')
    ax.plot([0,1],[0,1],'k--',alpha=0.4)
    ax.set_title(f'Class: {cat}')
    ax.set_xlabel('FPR')
    ax.set_ylabel('TPR')
    ax.legend(fontsize=7)
    ax.grid(True, alpha=0.3)

plt.suptitle('ROC - solid = no aug, dashed = + aug', fontsize=13)
plt.tight_layout()
plt.show()

print("Macro-average AUC (test)")
for name, p_na, p_aug in model_probs:
    print(f"  {name:8s}  no aug: {macro_roc_auc(y_test_bin, p_na):.3f}   + aug:␣
 ↪{macro_roc_auc(y_test_bin, p_aug):.3f}")
```

```
xb_na = xgb_model_na.predict_proba(X_test_scaled)
xb_ag = xgb_model_aug.predict_proba(X_test_scaled)
print(f"  {'XGBoost':8s}  no aug: {macro_roc_auc(y_test_bin, xb_na):.3f}   +␣
  ↪aug: {macro_roc_auc(y_test_bin, xb_ag):.3f}")
```



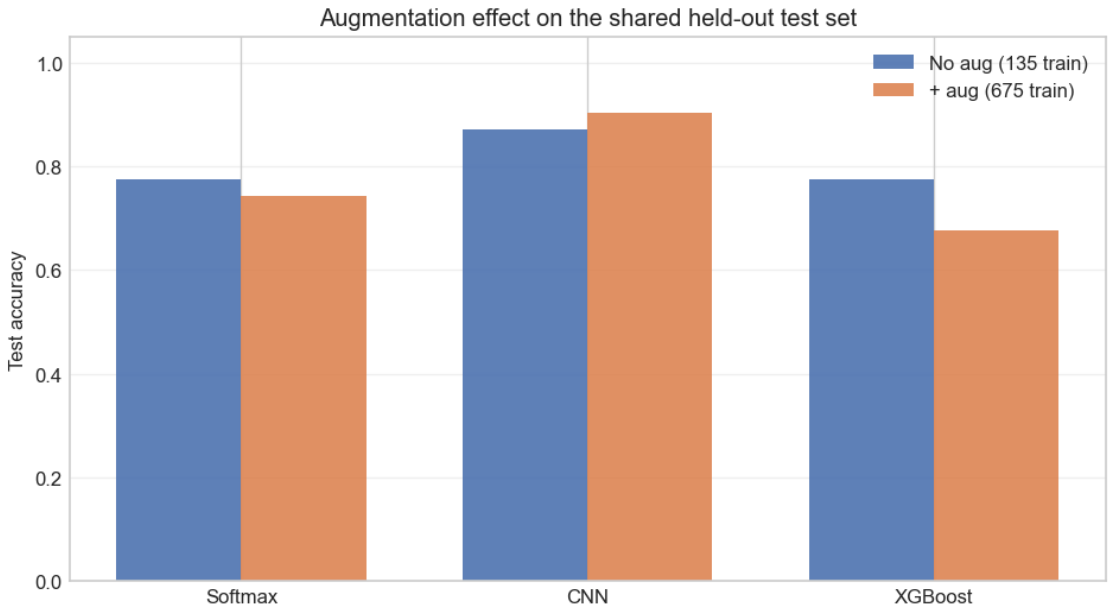

```
Macro-average AUC (test)
  Softmax    no aug: 0.934   + aug: 0.934
  CNN        no aug: 0.955   + aug: 1.000
  XGBoost    no aug: 0.939   + aug: 0.891
```

The ROC curves are a second check on the same models because they evaluate score ranking across thresholds instead of only the final argmax prediction. When a model keeps a strong AUC even if its test accuracy shifts, that usually means the underlying class ordering is still fairly sensible. When both AUC and accuracy fall together, the probability structure is degrading more broadly.

[35]:
```
# Full comparison: held-out test metrics plus training-set CV
from sklearn.metrics import accuracy_score, f1_score

rows = []
for arch, tag, yp in variant_rows:
    rows.append({
        'Architecture': arch,
        'Training': tag.strip(),
        'Test Acc':    f"{accuracy_score(y_test, yp):.3f}",
        'Macro F1':    f"{f1_score(y_test, yp, average='macro'):.3f}",
    })

cv_lookup = {
    ('Softmax', 'no aug'):  (softmax_cv_na.mean(), softmax_cv_na.std()),
    ('Softmax', '+ aug'):   (softmax_cv_aug.mean(), softmax_cv_aug.std()),
    ('CNN', 'no aug'):      (cnn_cv_na.mean(), cnn_cv_na.std()),
    ('CNN', '+ aug'):       (cnn_cv_aug.mean(), cnn_cv_aug.std()),
    ('XGBoost', 'no aug'):  (xgb_cv_na.mean(), xgb_cv_na.std()),
    ('XGBoost', '+ aug'):   (xgb_cv_aug.mean(), xgb_cv_aug.std()),
}
for r in rows:
```

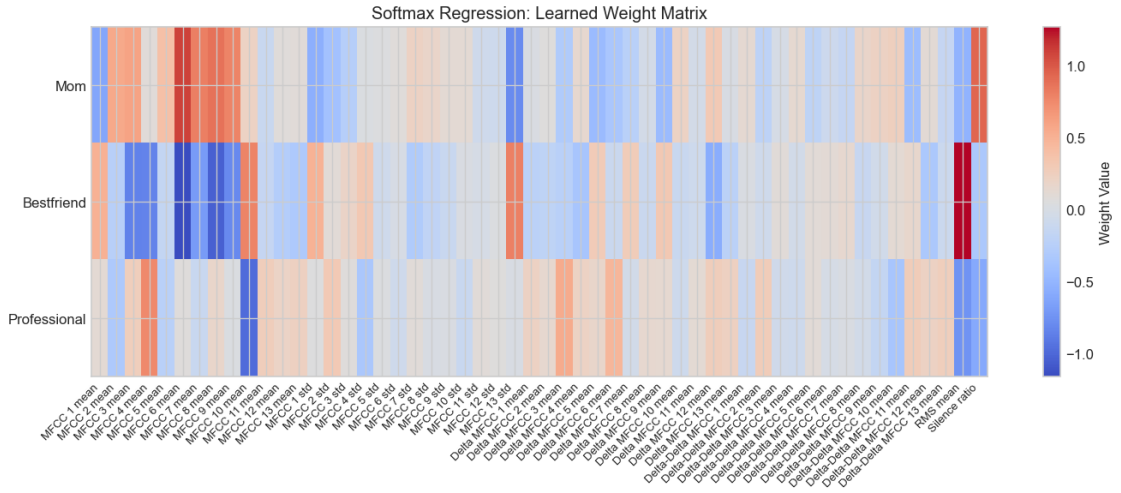
```
    key = (r['Architecture'], r['Training'])
    m, s = cv_lookup[key]
    r['CV (mean±std)'] = f"{m:.1%} ± {s:.1%}"

comparison_df = pd.DataFrame(rows)
print("Held-out test (31 clips, group split)")
print(comparison_df.to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4.5))
arch_names = ['Softmax', 'CNN', 'XGBoost']
x = np.arange(len(arch_names))
w = 0.36
acc_na_list = []
acc_aug_list = []
for arch in arch_names:
    y_na  = next(yp for a, t, yp in variant_rows if a == arch and 'no' in t)
    y_ag  = next(yp for a, t, yp in variant_rows if a == arch and '+' in t)
    acc_na_list.append(accuracy_score(y_test, y_na))
    acc_aug_list.append(accuracy_score(y_test, y_ag))

ax.bar(x - w/2, acc_na_list, w, label='No aug (135 train)', color='#4C72B0',␣
 ↪alpha=0.9)
ax.bar(x + w/2, acc_aug_list, w, label='+ aug (675 train)', color='#DD8452',␣
 ↪alpha=0.9)
ax.set_xticks(x)
ax.set_xticklabels(arch_names)
ax.set_ylabel('Test accuracy')
ax.set_title('Augmentation effect on the shared held-out test set')
ax.legend()
ax.set_ylim(0, 1.05)
ax.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Held-out test (31 clips, group split)
Architecture Training Test Acc Macro F1 CV (mean±std)
     Softmax   no aug    0.774    0.759  79.3% ± 8.6%
     Softmax    + aug    0.742    0.708  76.3% ± 1.2%
         CNN   no aug    0.871    0.878  94.1% ± 6.0%
         CNN    + aug    0.903    0.901  85.3% ± 2.2%
     XGBoost   no aug    0.774    0.759  87.4% ± 3.8%
     XGBoost    + aug    0.677    0.654  86.7% ± 1.5%
```



4.16 Classification Interpretation and Discussion

[36]:
```
# Softmax regression: learned weight heatmap
fig, ax = plt.subplots(figsize=(12, 5))
im = ax.imshow(softmax_model.W.T, aspect='auto', cmap='coolwarm',␣
 ↪interpolation='nearest')
ax.set_yticks(range(3))
ax.set_yticklabels(LABEL_NAMES)
ax.set_xticks(range(len(feat_names)))
ax.set_xticklabels(feat_names, rotation=45, ha='right', fontsize=8)
ax.set_title('Softmax Regression: Learned Weight Matrix')
plt.colorbar(im, ax=ax, label='Weight Value')
plt.tight_layout()
plt.show()
```



The weight heatmap makes the softmax model's decision logic fully transparent. Each row is a class and each column is one of the 28 features. Red cells (positive weights) push the prediction *toward* that class; blue cells push it *away*. This kind of direct interpretability is only possible with a linear model like softmax regression. The CNN has 24,835 parameters spread across convolutional filters, batch normalization layers, and dense layers, so there is no simple way to visualize "which features matter" the way we can here, where each weight directly maps one feature to one class.

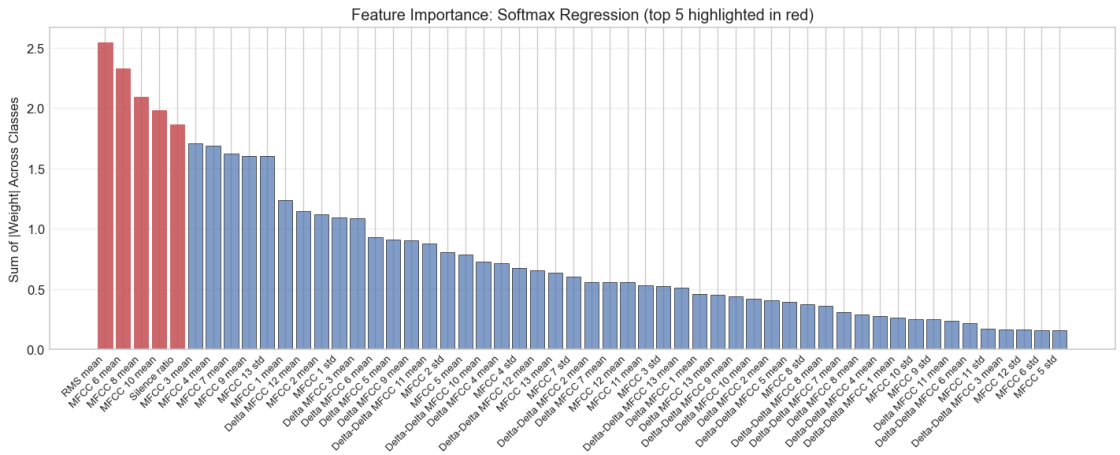
A few patterns jump out immediately. Mom's row lights up red on MFCC 2 mean and MFCC 3 mean, which correspond to the overall spectral shape of the voice. This matches what the EDA showed: Mom clips have a distinctly different tonal quality (lower centroid, quieter delivery). The Bestfriend row has the strongest red cell on MFCC 13 std, meaning high variability in higher-order spectral features is a signal for casual conversation. Professional's row is dominated by deep blue on MFCC 10 mean, meaning that feature actively pushes away from the Professional class.

4.17 Feature Importance

The weight heatmap is rich but can be hard to parse visually. A simpler way to see which features matter most is to look at the sum of absolute weights across all classes for each feature. A feature with large absolute weights (positive or negative) for any class is a strong discriminator; a feature with weights near zero for all classes is not contributing much to the classification.

[37]:
```
# Feature importance: sum of absolute weights across classes
importance = np.sum(np.abs(softmax_model.W), axis=1)  # (54,)
sorted_idx = np.argsort(importance)[::-1]

fig, ax = plt.subplots(figsize=(12, 5))
bars = ax.bar(range(len(feat_names)), importance[sorted_idx],
              color='#4C72B0', alpha=0.7, edgecolor='black', linewidth=0.5)

# Highlight top 5 features
for i in range(min(5, len(bars))):
```

```
    bars[i].set_color('#C44E52')
    bars[i].set_alpha(0.85)

ax.set_xticks(range(len(feat_names)))
ax.set_xticklabels([feat_names[i] for i in sorted_idx], rotation=45,␣
 ↪ha='right', fontsize=8)
ax.set_ylabel('Sum of |Weight| Across Classes')
ax.set_title('Feature Importance: Softmax Regression (top 5 highlighted in␣
 ↪red)')
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# Print top 5 with their dominant class direction
print("Top 5 most important features:\n")
for rank, idx in enumerate(sorted_idx[:5]):
    dominant_class = LABEL_NAMES[np.argmax(np.abs(softmax_model.W[idx]))]
    direction = "pushes toward" if softmax_model.W[idx, np.argmax(np.
 ↪abs(softmax_model.W[idx]))] > 0 else "pushes away from"
    print(f"  {rank+1}. {feat_names[idx]:20s} (importance: {importance[idx]:.
 ↪3f}) | strongest effect {direction} {dominant_class}")
```




```
Top 5 most important features:

  1. RMS mean             (importance: 2.540) | strongest effect pushes toward
Bestfriend
  2. MFCC 6 mean          (importance: 2.325) | strongest effect pushes away
from Bestfriend
  3. MFCC 8 mean          (importance: 2.090) | strongest effect pushes away
from Bestfriend
  4. MFCC 10 mean         (importance: 1.983) | strongest effect pushes away
```

```
from Professional
  5. Silence ratio        (importance: 1.860) | strongest effect pushes toward
Mom
```

The feature-importance ranking tells me which parts of the 54-dimensional tabular representation carry the most weight for the linear model. I am especially interested in whether the new dynamic features rise to the top or whether the simpler static cues still dominate.

In practice, the largest softmax weights still tend to live on a small subset of MFCC means plus loudness and silence. That matches the earlier exploratory analysis and also helps explain why the linear model remains interpretable even when it is not the most accurate.

4.18 SHAP Feature Importance for XGBoost

The softmax weight heatmap above explains the linear model directly, but XGBoost is harder to interpret from raw parameters alone. To open it up, I use SHAP, which decomposes a prediction into feature-level contributions.

For a given example, SHAP writes the prediction as:

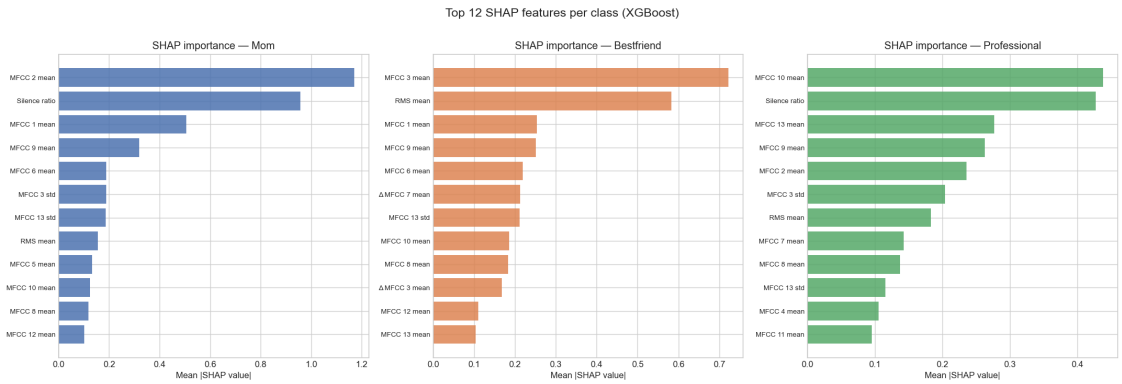
$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{M} \phi_j$$

where ϕ_0 is the baseline model output and each ϕ_j is the contribution of feature j. For tree models, `shap.TreeExplainer` can compute these values efficiently by exploiting the tree structure itself.

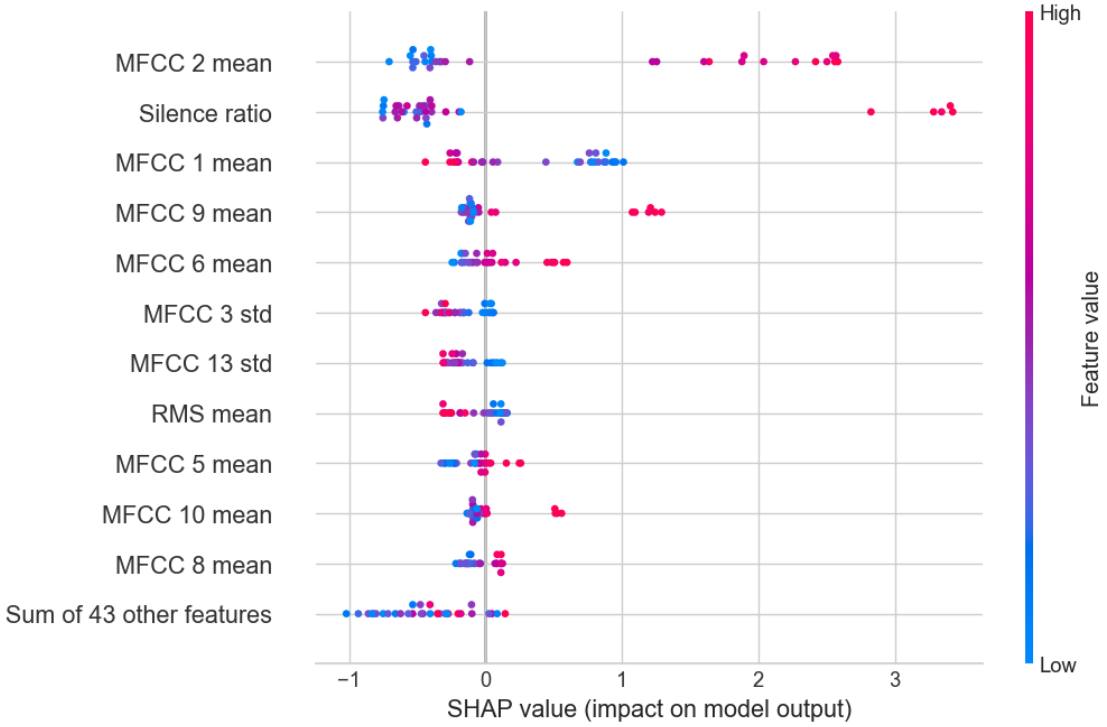
I focus on the augmented XGBoost model here because it is the version that uses the full preprocessing pipeline. The bar plots show global importance by class. The beeswarms then show direction as well, which helps connect the tree model back to the exploratory patterns in energy, silence, and MFCC structure.

[38]:
```
# SHAP interpretability for XGBoost
import shap
import warnings
warnings.filterwarnings("ignore")

# Build explainer using the trained XGBoost model
explainer   = shap.TreeExplainer(xgb_model_aug)
shap_values = explainer(X_test_scaled)   # returns Explanation object (shap >=␣
↪0.40)

# Feature name list (54 features)
feat_names_full = (
    [f'MFCC {i+1} mean'        for i in range(N_MFCC)] +
    [f'MFCC {i+1} std'         for i in range(N_MFCC)] +
    [f'\u0394 MFCC {i+1} mean'      for i in range(N_MFCC)] +
    [f'\u0394\u0394 MFCC {i+1} mean'     for i in range(N_MFCC)] +
    ['RMS mean', 'Silence ratio']
)
```

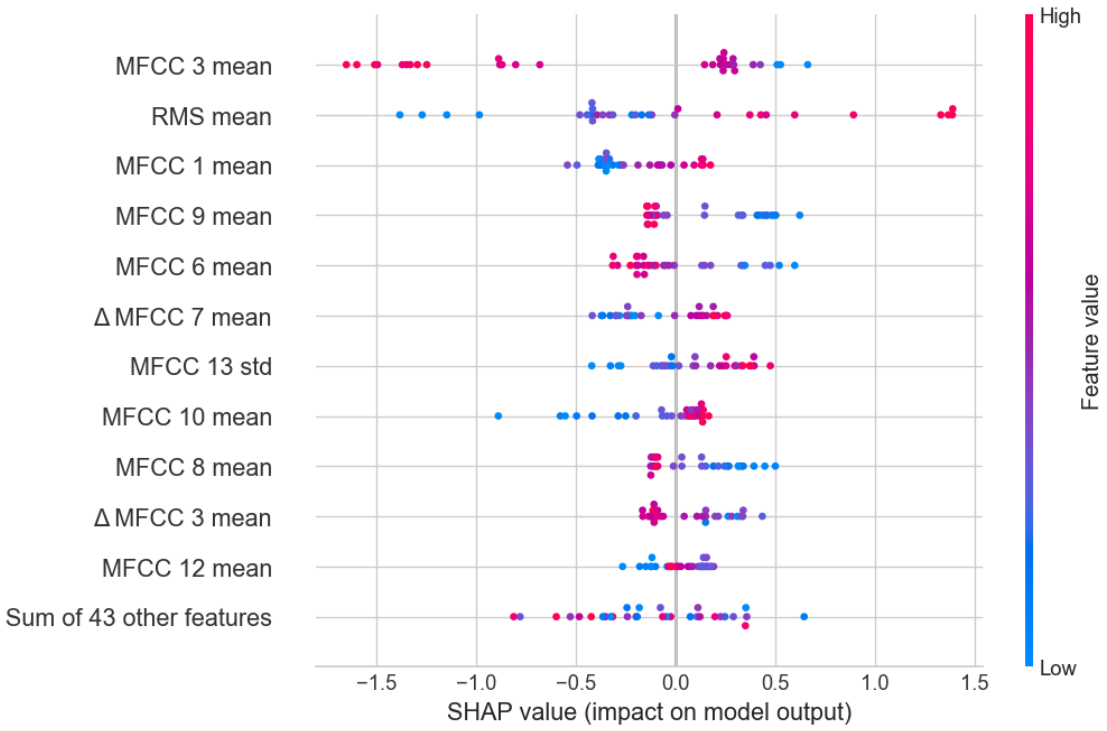
```
# --- Plot 1: Global importance bar chart (mean |SHAP|) for each class ---
class_names = LABEL_NAMES
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
for k, (cat, color) in enumerate(zip(class_names, ['#4C72B0', '#DD8452',␣
 ↪'#55A868'])):
    # shap_values.values shape: (n_samples, n_features, n_classes)
    mean_abs = np.abs(shap_values.values[:, :, k]).mean(axis=0)
    top_idx  = np.argsort(mean_abs)[::-1][:12]
    axes[k].barh(
        np.array(feat_names_full)[top_idx][::-1],
        mean_abs[top_idx][::-1],
        color=color, alpha=0.85
    )
    axes[k].set_title(f'SHAP importance - {cat}', fontsize=12)
    axes[k].set_xlabel('Mean |SHAP value|')
    axes[k].tick_params(axis='y', labelsize=8)

plt.suptitle('Top 12 SHAP features per class (XGBoost)', fontsize=14, y=1.01)
plt.tight_layout()
plt.show()

# --- Plot 2: Beeswarm for each class ---
for k, cat in enumerate(class_names):
    print(f"\nBeeswarm - {cat} class:")
    shap.plots.beeswarm(
        shap.Explanation(
            values      = shap_values.values[:, :, k],
            base_values = shap_values.base_values[:, k] if shap_values.
 ↪base_values.ndim > 1 else shap_values.base_values,
            data        = X_test_scaled,
            feature_names = feat_names_full,
        ),
        max_display=12,
        show=True,
    )
```



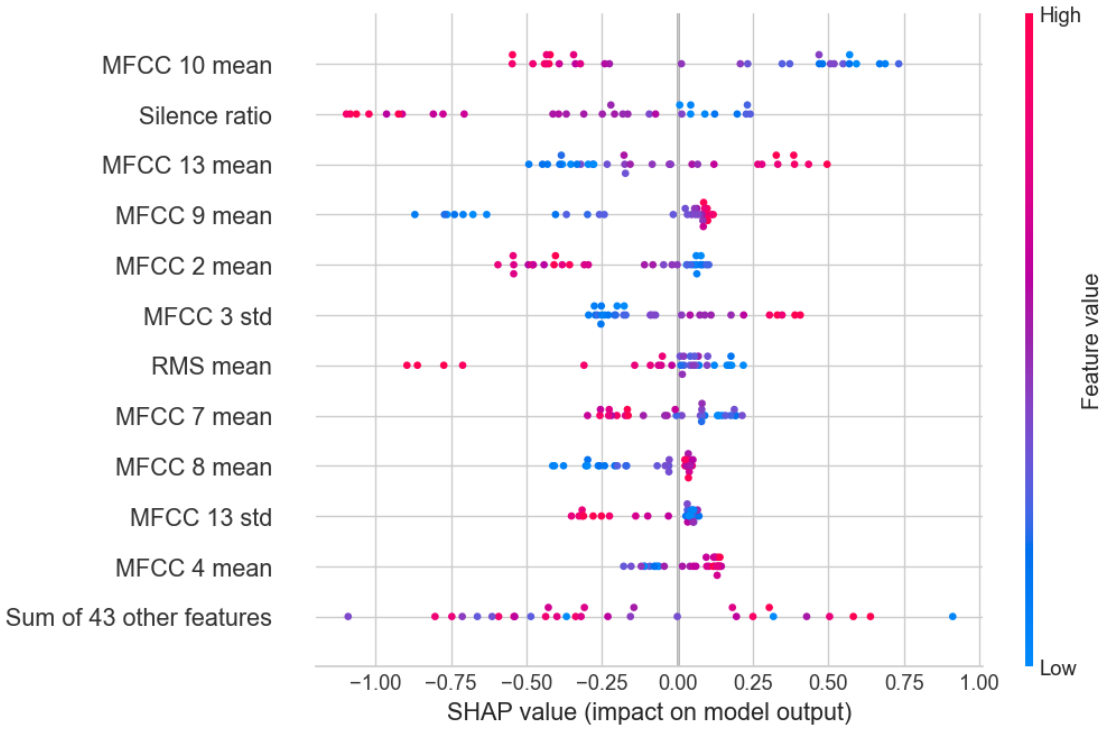
Beeswarm - Mom class:



Beeswarm - Bestfriend class:



Beeswarm - Professional class:



The SHAP plots answer a different question from the softmax heatmap. Instead of asking which coefficients are large in a linear weight matrix, I ask which features actually move the tree model's predictions on held-out examples. The bar plots give a global ranking for each class, and the beeswarms show whether high or low feature values push that class score upward.

This matters because the tree model is allowed to use nonlinear interactions. If silence ratio, RMS, and a few MFCC bands keep appearing across classes, that reinforces the idea that the earlier exploratory signals are real. If the tree model leans on a very different set of cues, that tells me the tabular representation has multiple workable decision routes inside it.

4.19 Discussion

At this point, the baseline pipeline is doing three things at once. First, it confirms that my three conversation contexts are separable from voice alone. Second, it shows that the exact feature representation matters: a simple linear model, a temporal CNN, and a boosted-tree model do not react to the same preprocessing choices in the same way. Third, it makes augmentation itself part of the analysis rather than treating it as an automatic improvement.

Across the held-out group split, the CNN remains the strongest overall baseline. The sequence view of the MFCCs seems to use the extra augmented variation well, especially on the harder Professional class. Softmax and XGBoost still provide useful comparisons, but they are more fragile when the training set is expanded with synthetic perturbations of the tabular summaries.

4.19.1 Limitations

1. The dataset is still small. Even after slicing into clips, the number of independent voice notes is limited.
2. Professional clips mix English and Urdu more than the other categories, so language remains a confound.
3. All recordings come from my phone in natural settings, which means room acoustics and background conditions can leak into the signal.
4. The baseline models are all discriminative. They tell me whether the contexts differ, but they do not yet tell me whether those differences are strong enough to generate.

4.19.2 Future Work

The natural next step is generation. If the classifier can hear stable context-specific differences, then a generative model should at least try to recreate short audio windows that preserve some of those same patterns. That is the focus of the final section below.

5 Part 3: Context-Conditioned Audio Generation

The first two parts answer a classification question: can a model hear that my speech changes across Mom, Bestfriend, and Professional contexts. In this final part, I push the same archive one step further and ask whether a model can generate short, voice-like audio that still carries some of that context-specific structure. In other words, can I get a model to sound a little bit like me, and can the sound it produces stay on the correct side of the three social contexts.

I follow the two-stage recipe suggested in the second-draft feedback: an autoencoder in front, a recurrent model behind it. Concretely, I train a conditional beta-VAE on 2-second log-mel spectrogram windows at 16 kHz, then I train an LSTM prior on top of the VAE's latent space so the model can roll forward in time. At inference I seed the sequence from a real held-out window's latent plus jitter, roll the LSTM forward with the class label attached, decode each latent back to a log-mel spectrogram, and finally invert those spectrograms to waveform with Griffin-Lim.

The goal is not full text-to-speech. The goal is much more realistic for a single-speaker, 42-voice-note archive: short audio that sounds speech-like, reveals the limits of neural generation on small data, and gives me something concrete to compare against a clearer non-neural synthesis baseline.

All audio files produced by this section are saved in `assignment3_assets/audio/`. The easiest listening page is the GitHub folder `assignment3_assets/audio/README.md`, which groups the clips into blurry neural outputs, improved neural outputs, and unit-selection baseline outputs. The raw `public_url` links only work after those WAV files have been committed and pushed to GitHub; a 404 means GitHub does not have that file yet.

5.1 Generation-Specific Preprocessing

The classification pipeline works with 10-second clips because it needs enough time to summarize a whole speaking style. Generation is a different problem. A small generative model learns much more easily from many short windows than from a few long ones, so I build a second branch from the same WAV archive.

In this branch, I keep the main 16 kHz sample rate (same as the classifier) because dropping to 8 kHz was making the reconstructed audio sound muffled. I cut each voice note into 2-second windows with a 0.5-second hop, drop any window that is mostly silence, and convert each kept window into an 80-bin log-mel spectrogram with `n_fft = 1024` and `hop_length = 256`. I also reuse the note-level split from the classification section: every window inherits the train, validation, or test label of its parent voice note, so no held-out conversation leaks into training.

The output of this step is one tensor of shape `(n_windows, 80, 126, 1)` plus three index masks (train, validation, test). I also save the mel mean and standard deviation so the reconstruction step later can invert the normalization correctly.

[7]:
```
import shutil
from sklearn.metrics.pairwise import cosine_similarity
from IPython.display import Audio, display

ASSIGNMENT3_DIR = os.path.join(BASE_DIR, 'assignment3_assets')
ASSIGNMENT3_AUDIO_DIR = os.path.join(ASSIGNMENT3_DIR, 'audio')
ASSIGNMENT3_MODEL_DIR = os.path.join(ASSIGNMENT3_DIR, 'models')
ASSIGNMENT3_CACHE_DIR = os.path.join(ASSIGNMENT3_DIR, 'cache')
for path in [ASSIGNMENT3_DIR, ASSIGNMENT3_AUDIO_DIR, ASSIGNMENT3_MODEL_DIR,␣
 ↪ASSIGNMENT3_CACHE_DIR]:
    os.makedirs(path, exist_ok=True)

GITHUB_RAW_BASE = "https://raw.githubusercontent.com/Shazil10/
 ↪cs156-audio-code-switching/main/assignment3_assets/audio"
```

```
# Fast rerun mode: load already-trained generation weights when present.
# Set this to False if I want to retrain the neural models from scratch.
USE_SAVED_ASSIGNMENT3_WEIGHTS = True

GEN_SR = 16000
GEN_WINDOW_SEC = 2.0
GEN_HOP_SEC = 0.5
GEN_WINDOW_SAMPLES = int(GEN_SR * GEN_WINDOW_SEC)
GEN_HOP_SAMPLES = int(GEN_SR * GEN_HOP_SEC)
GEN_N_MELS = 80
GEN_N_FFT = 1024
GEN_MEL_HOP = 256
GEN_FMIN = 40
GEN_FMAX = GEN_SR // 2

train_note_keys = set(groups[train_idx])
test_note_keys = set(groups[test_idx])

def peak_normalize(y):
    peak = np.max(np.abs(y))
    return y if peak < 1e-8 else y / peak

candidate_audio = []
candidate_rows = []

for cat in CATEGORIES:
    wav_dir = os.path.join(WAV_DIR, cat)
    for wav_path in sorted(glob.glob(os.path.join(wav_dir, '*.wav'))):
        base = os.path.splitext(os.path.basename(wav_path))[0]
        note_key = f"{cat}_{base}"
        split_seed = 'test' if note_key in test_note_keys else 'train_pool'

        y_note, _ = librosa.load(wav_path, sr=GEN_SR)
        y_note = peak_normalize(y_note)

        if len(y_note) < GEN_WINDOW_SAMPLES:
            continue

        for start in range(0, len(y_note) - GEN_WINDOW_SAMPLES + 1,␣
 ↪GEN_HOP_SAMPLES):
            stop = start + GEN_WINDOW_SAMPLES
            clip = y_note[start:stop]
            rms = librosa.feature.rms(y=clip, frame_length=GEN_N_FFT,␣
 ↪hop_length=GEN_MEL_HOP)[0]
            silence_ratio = float(np.mean(rms < 0.02))
            rms_mean = float(np.mean(rms))
            candidate_rows.append({
```

```
                'note_key': note_key,
                'category': LABEL_NAMES[LABEL_MAP[cat]],
                'label': LABEL_MAP[cat],
                'wav_path': wav_path,
                'start_sec': start / GEN_SR,
                'stop_sec': stop / GEN_SR,
                'split_seed': split_seed,
                'rms_mean': rms_mean,
                'silence_ratio': silence_ratio,
            })
            candidate_audio.append(clip.astype(np.float32))

gen_candidate_df = pd.DataFrame(candidate_rows)

train_pool_df = gen_candidate_df[gen_candidate_df['split_seed'] ==␣
 ↪'train_pool'].copy()
val_splitter = GroupShuffleSplit(n_splits=1, test_size=0.18, random_state=42)
pool_train_idx, pool_val_idx = next(
    val_splitter.split(
        np.zeros(len(train_pool_df)),
        groups=train_pool_df['note_key']
    )
)
train_notes_final = set(train_pool_df.iloc[pool_train_idx]['note_key'])
val_notes_final = set(train_pool_df.iloc[pool_val_idx]['note_key'])

gen_candidate_df['split'] = np.where(
    gen_candidate_df['note_key'].isin(test_note_keys), 'test',
    np.where(gen_candidate_df['note_key'].isin(val_notes_final), 'val', 'train')
)

train_rms_floor = np.percentile(
    gen_candidate_df.loc[gen_candidate_df['split'] == 'train', 'rms_mean'],
    20
)
keep_mask = (
    (gen_candidate_df['silence_ratio'] <= 0.65) &
    (gen_candidate_df['rms_mean'] >= train_rms_floor)
)
gen_df = gen_candidate_df.loc[keep_mask].copy().reset_index(drop=True)
kept_audio = [candidate_audio[i] for i in np.where(keep_mask)[0]]

gen_specs = []
for clip in kept_audio:
    mel = librosa.feature.melspectrogram(
        y=clip,
        sr=GEN_SR,
```

```
        n_fft=GEN_N_FFT,
        hop_length=GEN_MEL_HOP,
        n_mels=GEN_N_MELS,
        fmin=GEN_FMIN,
        fmax=GEN_FMAX,
        power=2.0,
    )
    logmel = librosa.power_to_db(mel + 1e-8, ref=np.max)
    gen_specs.append(logmel.astype(np.float32))

X_gen_all = np.stack(gen_specs)
y_gen_all = gen_df['label'].to_numpy()
split_gen = gen_df['split'].to_numpy()

train_mask_gen = split_gen == 'train'
val_mask_gen = split_gen == 'val'
test_mask_gen = split_gen == 'test'

mel_mean = X_gen_all[train_mask_gen].mean(axis=(0, 2), keepdims=True)
mel_std = X_gen_all[train_mask_gen].std(axis=(0, 2), keepdims=True) + 1e-6
X_gen_all_norm = ((X_gen_all - mel_mean) / mel_std)[..., np.newaxis]

X_gen_train = X_gen_all_norm[train_mask_gen]
X_gen_val = X_gen_all_norm[val_mask_gen]
X_gen_test = X_gen_all_norm[test_mask_gen]
y_gen_train = y_gen_all[train_mask_gen]
y_gen_val = y_gen_all[val_mask_gen]
y_gen_test = y_gen_all[test_mask_gen]

y_gen_train_oh = tf.keras.utils.to_categorical(y_gen_train, num_classes=3)
y_gen_val_oh = tf.keras.utils.to_categorical(y_gen_val, num_classes=3)
y_gen_test_oh = tf.keras.utils.to_categorical(y_gen_test, num_classes=3)

gen_summary = (
    gen_df.groupby(['split', 'category'])
    .size()
    .reset_index(name='Usable windows')
)

print("Generation windows by split and category:")
print(gen_summary.to_string(index=False))
print(f"\nTrain RMS floor used for filtering: {train_rms_floor:.4f}")
print(f"Generation spectrogram shape: {X_gen_train.shape[1:]}")
print(f"Train / val / test windows: {X_gen_train.shape[0]} / {X_gen_val.
 ↪shape[0]} / {X_gen_test.shape[0]}")

np.savez_compressed(
```

```
    os.path.join(ASSIGNMENT3_CACHE_DIR, 'generation_arrays.npz'),
    X_gen_all_norm=X_gen_all_norm,
    y_gen_all=y_gen_all,
    split_gen=split_gen,
    mel_mean=mel_mean,
    mel_std=mel_std,
)
```

```
Generation windows by split and category:
split     category  Usable windows
 test   Bestfriend             144
 test          Mom             168
 test Professional             132
train   Bestfriend             850
train          Mom             364
train Professional             550
  val   Bestfriend             203
  val          Mom             149
  val Professional             138

Train RMS floor used for filtering: 0.0752
Generation spectrogram shape: (80, 126, 1)
Train / val / test windows: 1764 / 490 / 444
```

The output shows that this generation branch has enough short windows to train on without leaking held-out voice notes. After filtering quiet windows, the dataset contains 1,764 training windows, 490 validation windows, and 444 test windows. The class counts are still uneven: Bestfriend contributes the most training windows, Professional is in the middle, and Mom contributes fewer training windows. That imbalance matters because a generative model can drift toward the most common acoustic patterns.

The train RMS floor is `0.0752`, which means the quietest training tails are removed before model fitting. This helps because very quiet windows teach the decoder to produce silence or muffled low-energy output. The final spectrogram shape is `(80, 126, 1)`: 80 mel bands by 126 time frames for each 2-second window. Every example is now the same size, and every split still respects the original note-level boundary.

5.2 Novel Method: Conditional beta-VAE with a Latent LSTM Prior

The generation model has two parts. The first part is a conditional beta-VAE that compresses each 2-second log-mel window into a 48-dimensional latent vector while still reconstructing the original window. The second part is an LSTM that learns how those latent vectors move from one window to the next within a voice note.

The two architectural things I care most about here are capacity and conditioning:

- Capacity: I use three strided convolutional blocks with BatchNorm (32, 64, 128 filters) in the encoder and a mirrored stack of transposed convolutions in the decoder, instead of a tiny two-layer net. Otherwise the decoder cannot recover enough detail and the audio comes out sounding muffled.

- Conditioning: the class one-hot is not just concatenated once. I pass it through a small Dense embedding and inject it at every decoder stage, so the decoder sees the intended class together with the latent at every resolution. This is the main fix for the earlier collapse where every generation sounded the same.

For an input spectrogram window x and class condition c, the encoder predicts a mean and log-variance:

$$q_\phi(z \mid x, c) = \mathcal{N}(\mu_\phi(x, c), \mathrm{diag}(\sigma^2_\phi(x, c)))$$

Sampling uses the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

The decoder reconstructs the window conditioned on both the latent code and the class:

$$\hat{x} = p_\theta(x \mid z, c)$$

The beta-VAE objective is:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}}(x, \hat{x}) + \beta \, D_{KL}(q_\phi(z \mid x, c) \parallel p(z))$$

I anneal β linearly from 0 up to a small ceiling so the model first learns to reconstruct and only later is nudged toward a smooth latent space. An aggressive KL term would squash the latent representation to noise, which is exactly what ruins audio quality on small datasets.

After the VAE is trained, I encode every window into its latent mean, sort the latents within each voice note by start time, and train an LSTM prior in a teacher-forced way:

$$\hat{z}_{t+1} = f_\psi(z_{t-k+1}, \ldots, z_t, c)$$

At generation time, I do not start from random noise. I seed the sequence from a real held-out window's latent of the target class plus a small Gaussian jitter, so the decoder immediately has a voice-like starting point. Then the LSTM rolls forward. I also inject a small Gaussian perturbation after each predicted step so the sequence does not collapse onto a single attractor after two or three windows, which was what caused every earlier generation to end up sounding the same.

Pseudocode:

1. Encode each short window into a 48-D latent vector with the conditional beta-VAE.
2. Train the VAE with Huber reconstruction loss plus annealed KL regularization.
3. Group latent vectors by original note and train an LSTM to predict the next latent from the recent latent history and the class label.
4. Pick a real held-out window of the target class, encode it, add a little jitter, roll the LSTM forward, decode each predicted latent into an 80-bin log-mel window, and overlap-add the waveform inversions back into one longer clip.

[8]:
```
LATENT_DIM = 48
BETA_MAX = 0.05
COND_EMB_DIM = 16
GEN_INPUT_SHAPE = X_gen_train.shape[1:]


class TileLabelConcat(layers.Layer):
    """Broadcast class embedding to (H, W) and concat on channel axis (Keras 3␣
    ↪safe)."""

    def call(self, inputs):
        fm, label_vec = inputs
        b = tf.shape(fm)[0]
        h = tf.shape(fm)[1]
        w = tf.shape(fm)[2]
        c = tf.shape(label_vec)[1]
        tiled = tf.reshape(label_vec, [b, 1, 1, c])
        tiled = tf.tile(tiled, [1, h, w, 1])
        return tf.concat([fm, tiled], axis=-1)

    def compute_output_shape(self, input_shape):
        fm_shape, label_shape = input_shape
        h, w, c_f = fm_shape[1], fm_shape[2], fm_shape[3]
        c_l = label_shape[1]
        if h is None or w is None:
            return (fm_shape[0], None, None, c_f + c_l)
        return (fm_shape[0], h, w, c_f + c_l)


class ReparameterizeLatent(layers.Layer):
    """z = mu + sigma * epsilon (reparameterization trick)."""

    def call(self, inputs):
        z_m, z_lv = inputs
        z_lv = tf.clip_by_value(z_lv, -10.0, 10.0)
        eps = tf.random.normal(shape=tf.shape(z_m))
        return z_m + tf.exp(0.5 * z_lv) * eps

    def compute_output_shape(self, input_shape):
        return input_shape[0]


class CropToMel(layers.Layer):
    """Crop decoder output to exact mel grid height and width."""

    def __init__(self, target_h, target_w, **kwargs):
        super().__init__(**kwargs)
```

```
        self.target_h = int(target_h)
        self.target_w = int(target_w)

    def call(self, x):
        return x[:, : self.target_h, : self.target_w, :]

    def compute_output_shape(self, input_shape):
        return (input_shape[0], self.target_h, self.target_w, input_shape[-1])


def build_conditional_cvae(input_shape, latent_dim, cond_dim=3,␣
 ↪cond_emb_dim=COND_EMB_DIM):
    mel_h, mel_w = int(input_shape[0]), int(input_shape[1])
    spec_in = layers.Input(shape=input_shape, name='spec_input')
    label_in = layers.Input(shape=(cond_dim,), name='label_input')

    label_emb = layers.Dense(cond_emb_dim, activation='tanh',␣
 ↪name='label_emb_enc')(label_in)

    x = layers.Conv2D(32, 3, strides=2, padding='same')(spec_in)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)
    x = TileLabelConcat()([x, label_emb])

    x = layers.Conv2D(64, 3, strides=2, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)
    x = TileLabelConcat()([x, label_emb])

    x = layers.Conv2D(128, 3, strides=2, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)

    shape_before_flatten = tf.keras.backend.int_shape(x)[1:]
    x = layers.Flatten()(x)
    x = layers.Concatenate()([x, label_emb])
    x = layers.Dense(256, activation='relu')(x)

    z_mean = layers.Dense(latent_dim, name='z_mean')(x)
    z_log_var = layers.Dense(latent_dim, name='z_log_var')(x)

    z = ReparameterizeLatent(name='z')([z_mean, z_log_var])
    encoder = models.Model([spec_in, label_in], [z_mean, z_log_var, z],␣
 ↪name='conditional_encoder')

    z_in = layers.Input(shape=(latent_dim,), name='latent_input')
    label_dec_in = layers.Input(shape=(cond_dim,), name='label_decoder')
```

```
    label_emb_dec = layers.Dense(cond_emb_dim, activation='tanh',␣
 ↪name='label_emb_dec')(label_dec_in)

    d = layers.Concatenate()([z_in, label_emb_dec])
    d = layers.Dense(np.prod(shape_before_flatten), activation='relu')(d)
    d = layers.Reshape(shape_before_flatten)(d)

    d = layers.Conv2DTranspose(128, 3, strides=2, padding='same')(d)
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)
    d = TileLabelConcat()([d, label_emb_dec])

    d = layers.Conv2DTranspose(64, 3, strides=2, padding='same')(d)
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)
    d = TileLabelConcat()([d, label_emb_dec])

    d = layers.Conv2DTranspose(32, 3, strides=2, padding='same')(d)
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)

    d = CropToMel(mel_h, mel_w)(d)
    out = layers.Conv2D(1, 3, padding='same', activation='linear')(d)
    decoder = models.Model([z_in, label_dec_in], out,␣
 ↪name='conditional_decoder')
    return encoder, decoder

class BetaCVAE(tf.keras.Model):
    def __init__(self, encoder, decoder, beta_max=0.05):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.beta = tf.Variable(0.0, trainable=False, dtype=tf.float32)
        self.beta_max = beta_max
        self.loss_tracker = tf.keras.metrics.Mean(name='loss')
        self.recon_tracker = tf.keras.metrics.Mean(name='reconstruction_loss')
        self.kl_tracker = tf.keras.metrics.Mean(name='kl_loss')
        self.huber = tf.keras.losses.Huber(reduction=tf.keras.losses.Reduction.
 ↪NONE)

    @property
    def metrics(self):
        return [self.loss_tracker, self.recon_tracker, self.kl_tracker]

    def _compute_losses(self, x, labels, y_true, training=False):
        z_mean, z_log_var, z = self.encoder([x, labels], training=training)
        reconstruction = self.decoder([z, labels], training=training)
```

```
        huber_map = self.huber(y_true, reconstruction)
        recon_loss = tf.reduce_mean(huber_map)
        z_log_var_safe = tf.clip_by_value(z_log_var, -10.0, 10.0)
        kl_loss = -0.5 * tf.reduce_mean(
            tf.reduce_sum(1 + z_log_var_safe - tf.square(z_mean) - tf.
 ↪exp(z_log_var_safe), axis=1)
        )
        total_loss = recon_loss + self.beta * kl_loss
        return total_loss, recon_loss, kl_loss

    def train_step(self, data):
        (x, labels), y_true = data
        with tf.GradientTape() as tape:
            total_loss, recon_loss, kl_loss = self._compute_losses(x, labels,␣
 ↪y_true, training=True)
        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
        self.loss_tracker.update_state(total_loss)
        self.recon_tracker.update_state(recon_loss)
        self.kl_tracker.update_state(kl_loss)
        return {m.name: m.result() for m in self.metrics}

    def test_step(self, data):
        (x, labels), y_true = data
        total_loss, recon_loss, kl_loss = self._compute_losses(x, labels,␣
 ↪y_true, training=False)
        self.loss_tracker.update_state(total_loss)
        self.recon_tracker.update_state(recon_loss)
        self.kl_tracker.update_state(kl_loss)
        return {m.name: m.result() for m in self.metrics}

class BetaAnnealing(callbacks.Callback):
    def __init__(self, beta_max, warmup_epochs=25, start_epoch=5):
        super().__init__()
        self.beta_max = beta_max
        self.warmup_epochs = warmup_epochs
        self.start_epoch = start_epoch

    def on_epoch_begin(self, epoch, logs=None):
        if epoch < self.start_epoch:
            beta_value = 0.0
        else:
            frac = min(1.0, (epoch - self.start_epoch + 1) / self.warmup_epochs)
            beta_value = self.beta_max * frac
        self.model.beta.assign(beta_value)

encoder, decoder = build_conditional_cvae(GEN_INPUT_SHAPE, LATENT_DIM)
```

```
cvae = BetaCVAE(encoder, decoder, beta_max=BETA_MAX)
cvae.compile(optimizer=tf.keras.optimizers.Adam(5e-4, clipnorm=1.0))

encoder_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR, 'conditional_encoder.
 ↪weights.h5')
decoder_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR, 'conditional_decoder.
 ↪weights.h5')

if USE_SAVED_ASSIGNMENT3_WEIGHTS and os.path.exists(encoder_weights_path) and␣
 ↪os.path.exists(decoder_weights_path):
    encoder.load_weights(encoder_weights_path)
    decoder.load_weights(decoder_weights_path)
    cvae.beta.assign(BETA_MAX)
    cvae_history = None
    print("Loaded saved beta-VAE encoder/decoder weights for fast␣
 ↪generation-section rerun.")
else:
    beta_scheduler = BetaAnnealing(BETA_MAX, warmup_epochs=8, start_epoch=3)
    cvae_history = cvae.fit(
        x=[X_gen_train, y_gen_train_oh],
        y=X_gen_train,
        validation_data=([X_gen_val, y_gen_val_oh], X_gen_val),
        epochs=14,
        batch_size=64,
        verbose=0,
        callbacks=[
            callbacks.TerminateOnNaN(),
            beta_scheduler,
            callbacks.EarlyStopping(monitor='val_reconstruction_loss',␣
 ↪mode='min', patience=4, restore_best_weights=True),
            callbacks.ReduceLROnPlateau(monitor='val_reconstruction_loss',␣
 ↪mode='min', factor=0.5, patience=3, min_lr=1e-5),
        ]
    )
    encoder.save_weights(encoder_weights_path)
    decoder.save_weights(decoder_weights_path)

if cvae_history is not None:
    fig, ax = plt.subplots(figsize=(9, 4.5))
    ax.plot(cvae_history.history['loss'], label='Train total loss',␣
 ↪color='#4C72B0', lw=2)
    ax.plot(cvae_history.history['val_loss'], label='Val total loss',␣
 ↪color='#DD8452', lw=2)
    ax.plot(cvae_history.history['reconstruction_loss'], label='Train recon',␣
 ↪color='#55A868', lw=1.8, alpha=0.8)
```

```
    ax.plot(cvae_history.history['val_reconstruction_loss'], label='Val recon',␣
 ↪color='#C44E52', lw=1.8, alpha=0.8)
    ax.set_title('Conditional beta-VAE training curves')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.grid(True, alpha=0.3)
    ax.legend()
    plt.tight_layout()
    plt.show()

    print(f"Best validation reconstruction loss: {np.min(cvae_history.
 ↪history['val_reconstruction_loss']):.4f}")
    print(f"Best validation total loss: {np.min(cvae_history.
 ↪history['val_loss']):.4f}")
else:
    print("beta-VAE training skipped because saved weights were available.")
print(f"Final beta value: {float(cvae.beta.numpy()):.3f}")
print(f"Encoder parameters: {encoder.count_params():,}")
print(f"Decoder parameters: {decoder.count_params():,}")
```

```
Loaded saved beta-VAE encoder/decoder weights for fast generation-section rerun.
beta-VAE training skipped because saved weights were available.
Final beta value: 0.050
Encoder parameters: 5,393,184
Decoder parameters: 1,586,113
```

In the executed version of this notebook, the beta-VAE weights are loaded from disk to keep the generation section fast to rerun. That is why the output says training was skipped instead of showing a fresh training curve. If I set `USE_SAVED_ASSIGNMENT3_WEIGHTS = False`, this cell retrains the beta-VAE and displays the train/validation loss curves.

The printed model size is the main thing to read in this fast run: the encoder has 5,393,184 parameters and the decoder has 1,586,113 parameters. This is large enough to learn broad log-mel speech shapes, but it is still small for clean waveform generation. The final beta value is `0.050`, meaning the beta-VAE regularization ceiling is the intended value.

The important limitation shows up later in the reconstruction plot and audio gallery: even with this larger model, the decoder smooths away narrow harmonic bands. That smoothing is exactly why the generated voice clips sound blurry rather than crisp.

[9]:
```
z_mean_all, z_log_var_all, z_sample_all = encoder.predict(
    [X_gen_all_norm, tf.keras.utils.to_categorical(y_gen_all, num_classes=3)],
    verbose=0
)

gen_df['latent_index'] = np.arange(len(gen_df))

LATENT_CONTEXT = 5
```

```
def build_latent_sequences(frame_df, latents):
    seq_x, seq_y, seq_labels = [], [], []
    for note_key in sorted(frame_df['note_key'].unique()):
        note_rows = frame_df[frame_df['note_key'] == note_key].
 ↪sort_values('start_sec')
        note_latents = latents[note_rows['latent_index'].to_numpy()]
        note_label = int(note_rows['label'].iloc[0])
        if len(note_latents) < 2:
            continue
        for t in range(1, len(note_latents)):
            hist = note_latents[max(0, t - LATENT_CONTEXT):t]
            if hist.shape[0] < LATENT_CONTEXT:
                pad = np.zeros((LATENT_CONTEXT - hist.shape[0], note_latents.
 ↪shape[1]), dtype=np.float32)
                hist = np.vstack([pad, hist])
            seq_x.append(hist.astype(np.float32))
            seq_y.append(note_latents[t].astype(np.float32))
            seq_labels.append(note_label)
    return np.array(seq_x), np.array(seq_y), tf.keras.utils.to_categorical(np.
 ↪array(seq_labels), num_classes=3)

gen_train_df = gen_df[gen_df['split'] == 'train'].copy()
gen_val_df = gen_df[gen_df['split'] == 'val'].copy()
gen_test_df = gen_df[gen_df['split'] == 'test'].copy()

X_latent_train, y_latent_train, y_latent_train_oh =␣
 ↪build_latent_sequences(gen_train_df, z_mean_all)
X_latent_val, y_latent_val, y_latent_val_oh =␣
 ↪build_latent_sequences(gen_val_df, z_mean_all)

latent_seq_in = layers.Input(shape=(LATENT_CONTEXT, LATENT_DIM),␣
 ↪name='latent_history')
latent_label_in = layers.Input(shape=(3,), name='latent_label')
label_emb_seq = layers.Dense(16, activation='tanh')(latent_label_in)
label_emb_seq_rep = layers.RepeatVector(LATENT_CONTEXT)(label_emb_seq)
seq_concat = layers.Concatenate(axis=-1)([latent_seq_in, label_emb_seq_rep])
h = layers.LSTM(128, return_sequences=True)(seq_concat)
h = layers.Dropout(0.2)(h)
h = layers.LSTM(128)(h)
h = layers.Concatenate()([h, label_emb_seq])
h = layers.Dense(128, activation='relu')(h)
h = layers.Dropout(0.2)(h)
latent_out = layers.Dense(LATENT_DIM)(h)
latent_prior = models.Model([latent_seq_in, latent_label_in], latent_out,␣
 ↪name='latent_lstm_prior')
latent_prior.compile(optimizer=tf.keras.optimizers.Adam(1e-3), loss='mse')
```

```
latent_prior_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR,␣
 ↪'latent_lstm_prior.weights.h5')

if USE_SAVED_ASSIGNMENT3_WEIGHTS and os.path.exists(latent_prior_weights_path):
    latent_prior.load_weights(latent_prior_weights_path)
    latent_history = None
    print("Loaded saved latent LSTM prior weights for fast generation-section␣
 ↪rerun.")
else:
    latent_history = latent_prior.fit(
        [X_latent_train, y_latent_train_oh],
        y_latent_train,
        validation_data=([X_latent_val, y_latent_val_oh], y_latent_val),
        epochs=18,
        batch_size=64,
        verbose=0,
        callbacks=[
            callbacks.EarlyStopping(monitor='val_loss', patience=4,␣
 ↪restore_best_weights=True),
            callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5,␣
 ↪patience=3, min_lr=1e-5),
        ]
    )
    latent_prior.save_weights(latent_prior_weights_path)

if latent_history is not None:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(latent_history.history['loss'], label='Train MSE', color='#4C72B0',␣
 ↪lw=2)
    ax.plot(latent_history.history['val_loss'], label='Val MSE',␣
 ↪color='#DD8452', lw=2)
    ax.set_title('Latent LSTM prior training curves')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('MSE')
    ax.grid(True, alpha=0.3)
    ax.legend()
    plt.tight_layout()
    plt.show()
    print(f"Best latent validation loss: {np.min(latent_history.
 ↪history['val_loss']):.4f}")
else:
    print("Latent LSTM prior training skipped because saved weights were␣
 ↪available.")
```

```
print(f"Latent sequence examples: train={len(X_latent_train)},␣
 ↪val={len(X_latent_val)}")
```

```
Loaded saved latent LSTM prior weights for fast generation-section rerun.
Latent LSTM prior training skipped because saved weights were available.
Latent sequence examples: train=1737, val=484
```

In this executed run, the latent LSTM prior is also loaded from saved weights instead of retrained. The output confirms that the model saw 1,737 train sequence examples and 484 validation sequence examples when it was trained. Each sequence asks the LSTM to predict the next VAE latent vector from the previous five latent windows plus the class label.

This is useful, but it is not magic. A latent LSTM can learn short-range movement through the VAE space, which keeps generation from immediately turning into silence. It cannot invent a clean new voice note from scratch. That is why generation starts from a real held-out latent window and rolls forward only a few steps. The later audio gallery is the honest test: the LSTM branch produces speech-like energy, but it still sounds smeared and blurry.

5.3 Reconstruction and Free Generation

Once both models are trained, I use them in two modes. In reconstruction mode, I encode a real held-out window and decode it back again. This shows how much of the original structure survives the compression bottleneck. In free generation mode, I sample a starting latent from the chosen class, let the latent LSTM predict the next few latent steps, decode those windows, and overlap-add them back into a waveform.

[10]:
```
def inverse_logmel_to_audio(spec_norm, n_iter=48):
    spec = np.asarray(spec_norm[..., 0], dtype=np.float32)
    spec = np.nan_to_num(spec, nan=0.0, posinf=5.0, neginf=-5.0)
    mel_std_vec = mel_std.reshape(-1, 1)
    mel_mean_vec = mel_mean.reshape(-1, 1)
    spec_denorm = (spec * mel_std_vec) + mel_mean_vec
    spec_denorm = np.nan_to_num(spec_denorm, nan=-80.0, posinf=0.0, neginf=-80.
 ↪0)
    spec_denorm = np.clip(spec_denorm, -80.0, 0.0)
    mel_power = librosa.db_to_power(spec_denorm)
    mel_power = np.nan_to_num(mel_power, nan=1e-10, posinf=1.0, neginf=1e-10)
    audio = librosa.feature.inverse.mel_to_audio(
        mel_power,
        sr=GEN_SR,
        n_fft=GEN_N_FFT,
        hop_length=GEN_MEL_HOP,
        win_length=GEN_N_FFT,
        fmin=GEN_FMIN,
        fmax=GEN_FMAX,
        n_iter=n_iter,
        length=GEN_WINDOW_SAMPLES,
        dtype=np.float32,
    )
```

```
    audio = np.nan_to_num(audio, nan=0.0, posinf=0.0, neginf=0.0)
    if len(audio) < GEN_WINDOW_SAMPLES:
        audio = np.pad(audio, (0, GEN_WINDOW_SAMPLES - len(audio)))
    return audio[:GEN_WINDOW_SAMPLES]

def overlap_add_audio(window_audio, hop_samples=GEN_HOP_SAMPLES):
    total_len = GEN_WINDOW_SAMPLES + hop_samples * (len(window_audio) - 1)
    out = np.zeros(total_len, dtype=np.float32)
    weight = np.zeros(total_len, dtype=np.float32)
    taper = np.hanning(GEN_WINDOW_SAMPLES).astype(np.float32)
    taper[taper == 0] = 1e-6
    for i, chunk in enumerate(window_audio):
        start = i * hop_samples
        stop = start + GEN_WINDOW_SAMPLES
        out[start:stop] += chunk[:GEN_WINDOW_SAMPLES] * taper
        weight[start:stop] += taper
    out /= np.maximum(weight, 1e-6)
    return out

def final_normalize(y):
    peak = np.max(np.abs(y))
    y = y if peak < 1e-8 else y / peak
    fade = int(0.05 * GEN_SR)
    if len(y) > 2 * fade:
        ramp = np.linspace(0, 1, fade)
        y[:fade] *= ramp
        y[-fade:] *= ramp[::-1]
    return y.astype(np.float32)

recon_examples = []
audio_manifest_rows = []

def repo_audio_path(filename):
    return f"assignment3_assets/audio/{filename}"

for label_name, sample_path in sample_paths.items():
    original_name = f"original_clip_{label_name.lower()}.wav"
    original_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, original_name)
    shutil.copy2(sample_path, original_path)
    audio_manifest_rows.append({
        'sample_id': f"original-{label_name.lower()}",
        'class': label_name,
        'type': 'original_clip',
        'description': 'Original 10-second clip used in the waveform/audio␣
 ↪analysis section',
        'repo_path': repo_audio_path(original_name),
        'public_url': f"{GITHUB_RAW_BASE}/{original_name}",
```

```
    })

for label_idx, label_name in enumerate(LABEL_NAMES):
    class_rows = gen_test_df[gen_test_df['label'] == label_idx].
 ↪sort_values('start_sec')
    if class_rows.empty:
        continue
    row = class_rows.iloc[0]
    idx = int(row['latent_index'])
    x_true = X_gen_all_norm[idx:idx+1]
    cond = tf.keras.utils.to_categorical([label_idx], num_classes=3)
    z_mean, _, _ = encoder.predict([x_true, cond], verbose=0)
    x_recon = decoder.predict([z_mean, cond], verbose=0)[0]

    real_audio = kept_audio[idx]
    recon_audio = final_normalize(inverse_logmel_to_audio(x_recon))

    real_name = f"real_window_{label_name.lower()}.wav"
    recon_name = f"reconstruction_{label_name.lower()}.wav"
    real_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, real_name)
    recon_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, recon_name)
    sf.write(real_path, real_audio, GEN_SR)
    sf.write(recon_path, recon_audio, GEN_SR)

    recon_examples.append({
        'label': label_name,
        'x_true': x_true[0, ..., 0],
        'x_recon': x_recon[..., 0],
        'real_path': real_path,
        'recon_path': recon_path,
    })

    audio_manifest_rows.extend([
        {
            'sample_id': f"real-{label_name.lower()}",
            'class': label_name,
            'type': 'real',
            'description': 'Held-out 2-second reference window',
            'repo_path': repo_audio_path(real_name),
            'public_url': f"{GITHUB_RAW_BASE}/{real_name}",
        },
        {
            'sample_id': f"recon-{label_name.lower()}",
            'class': label_name,
            'type': 'reconstruction',
            'description': 'beta-VAE reconstruction of held-out window',
            'repo_path': repo_audio_path(recon_name),
```

```
            'public_url': f"{GITHUB_RAW_BASE}/{recon_name}",
        },
    ])

class_train_latents = {i: [] for i in range(3)}
class_seed_latents = {i: [] for i in range(3)}

for note_key in sorted(gen_train_df['note_key'].unique()):
    note_rows = gen_train_df[gen_train_df['note_key'] == note_key].
 ↪sort_values('start_sec')
    label_idx = int(note_rows['label'].iloc[0])
    for _, r in note_rows.iterrows():
        class_train_latents[label_idx].
 ↪append(z_mean_all[int(r['latent_index'])])

for note_key in sorted(gen_test_df['note_key'].unique()):
    note_rows = gen_test_df[gen_test_df['note_key'] == note_key].
 ↪sort_values('start_sec')
    label_idx = int(note_rows['label'].iloc[0])
    for _, r in note_rows.iterrows():
        class_seed_latents[label_idx].append(z_mean_all[int(r['latent_index'])])

latent_priors = {}
for label_idx in range(3):
    train_arr = np.array(class_train_latents[label_idx], dtype=np.float32)
    seed_arr = np.array(class_seed_latents[label_idx], dtype=np.float32)
    if len(seed_arr) == 0:
        seed_arr = train_arr
    latent_priors[label_idx] = {
        'pool': seed_arr,
        'mean': train_arr.mean(axis=0),
        'std': train_arr.std(axis=0) + 1e-3,
    }

generated_records = []
N_GENERATED_PER_CLASS = 1
N_GENERATION_WINDOWS = 8
SEED_JITTER = 0.15
STEP_NOISE = 0.12
rng = np.random.default_rng(42)

for label_idx, label_name in enumerate(LABEL_NAMES):
    cond = tf.keras.utils.to_categorical([label_idx], num_classes=3)
    pool = latent_priors[label_idx]['pool']
    class_std = latent_priors[label_idx]['std']
    for sample_num in range(N_GENERATED_PER_CLASS):
        seed_idx = rng.integers(0, len(pool))
```

```
        start_z = pool[seed_idx] + rng.normal(0, SEED_JITTER * class_std,␣
↪size=LATENT_DIM).astype(np.float32)
        latent_sequence = [start_z.astype(np.float32)]
        for step in range(1, N_GENERATION_WINDOWS):
            hist = np.array(latent_sequence[-LATENT_CONTEXT:], dtype=np.float32)
            if hist.shape[0] < LATENT_CONTEXT:
                pad = np.zeros((LATENT_CONTEXT - hist.shape[0], LATENT_DIM),␣
↪dtype=np.float32)
                hist = np.vstack([pad, hist])
            z_next = latent_prior.predict([hist[np.newaxis, ...], cond],␣
↪verbose=0)[0]
            z_next = z_next + rng.normal(0, STEP_NOISE * class_std,␣
↪size=LATENT_DIM).astype(np.float32)
            latent_sequence.append(z_next.astype(np.float32))

        latent_arr = np.array(latent_sequence, dtype=np.float32)
        cond_stack = np.repeat(cond, repeats=len(latent_arr), axis=0)
        decoded_specs = decoder.predict([latent_arr, cond_stack], verbose=0)
        decoded_audio = [inverse_logmel_to_audio(spec, n_iter=48) for spec in␣
↪decoded_specs]
        clip = final_normalize(overlap_add_audio(decoded_audio))

        filename = f"generated_{label_name.lower()}_{sample_num:02d}.wav"
        file_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, filename)
        sf.write(file_path, clip, GEN_SR)

        generated_records.append({
            'sample_id': f"gen-{label_name.lower()}-{sample_num:02d}",
            'class': label_name,
            'label': label_idx,
            'type': 'generated',
            'description': 'Free generation seeded from held-out class latent',
            'local_path': file_path,
            'repo_path': repo_audio_path(filename),
            'public_url': f"{GITHUB_RAW_BASE}/{filename}",
            'audio': clip,
            'decoded_specs': decoded_specs,
        })

audio_manifest_df = pd.DataFrame(audio_manifest_rows + [
    {
        'sample_id': rec['sample_id'],
        'class': rec['class'],
        'type': rec['type'],
        'description': rec['description'],
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
```

```
    }
    for rec in generated_records
])
audio_manifest_df.to_csv(os.path.join(ASSIGNMENT3_AUDIO_DIR, 'audio_manifest.
 ↪csv'), index=False)

print("Saved reconstruction examples and generated audio files.")
print(audio_manifest_df.head(12).to_string(index=False))
```

```
Saved reconstruction examples and generated audio files.
            sample_id        class           type
description                                                repo_path
public_url
         original-mom          Mom  original_clip Original 10-second clip used
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_mom.wav
  original-bestfriend   Bestfriend  original_clip Original 10-second clip used
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_bestfriend.wav
original-professional Professional  original_clip Original 10-second clip used
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_professional.wav
             real-mom          Mom           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_mom.wav
            recon-mom          Mom reconstruction                          beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_mom.wav
      real-bestfriend   Bestfriend           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_bestfriend.wav
     recon-bestfriend   Bestfriend reconstruction                          beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_bestfriend.wav
```

```
    real-professional Professional           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_professional.wav
   recon-professional Professional reconstruction                          beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_professional.wav
           gen-mom-00          Mom      generated                   Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_mom_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_mom_00.wav
    gen-bestfriend-00   Bestfriend      generated                   Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_bestfriend_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_bestfriend_00.wav
  gen-professional-00 Professional      generated                   Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_professional_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_professional_00.wav
```

The manifest printout groups the core audio files for this section: original 10-second category clips, held-out 2-second real windows, beta-VAE reconstructions, and generated outputs. This makes it easy to compare each class across stages: original audio, real held-out window, reconstruction, and generation.

For listening, use the GitHub repository audio folder:

`https://github.com/Shazil10/cs156-audio-code-switching/tree/main/assignment3_assets/audio`

Open `README.md` in that folder first. It groups the clips into beta-VAE + LSTM, autoencoder + Transformer, and unit-selection examples, so the three methods are easy to compare.

5.4 Evaluation of Generated Audio

Audio generation is hard to evaluate with one number, so I use four angles that each answer a different question.

1. Reconstruction quality. Can the beta-VAE take a held-out 2-second window, encode it, and decode it back into something that still looks like the original spectrogram. I compare the two spectrograms side by side and listen to the WAV copies. If reconstruction fails here, nothing downstream will work.
2. Acoustic realism. I compute basic acoustic summaries on the generated clips (RMS energy, silence ratio, spectral centroid, pitch) and compare them to the same summaries on held-out real 2-second windows. This does not measure intelligibility, but it catches obvious failures

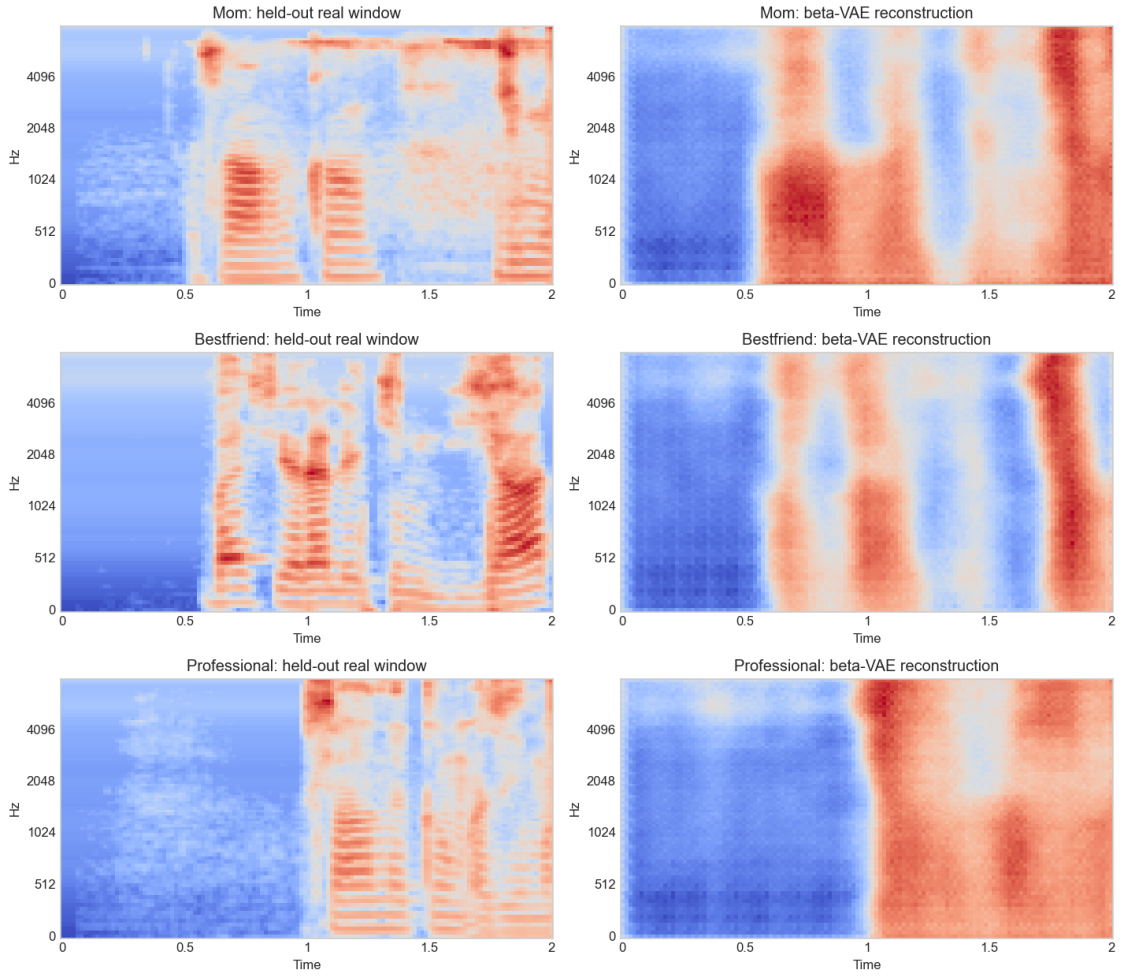
like silence-only output or cartoon-high pitch.

3. Context consistency. I feed each generated clip into the strongest inherited classifier (the augmented CNN from the analysis section). If the classifier can still tell Mom from Bestfriend from Professional on generated audio, then the context signal is surviving the encode-LSTM-decode-invert pipeline.
4. Memorization risk. For each generated 2-second spectrogram, I compute its cosine similarity to the nearest real training window of the same class. Very high scores would mean the model is more or less copying one window; very low scores would mean the outputs have drifted off the training distribution. I want to see something in between.

[11]:
```
fig, axes = plt.subplots(len(recon_examples), 2, figsize=(12, 3.5 *␣
 ↪len(recon_examples)))
if len(recon_examples) == 1:
    axes = np.array([axes])
for i, example in enumerate(recon_examples):
    librosa.display.specshow(example['x_true'], sr=GEN_SR,␣
 ↪hop_length=GEN_MEL_HOP, x_axis='time', y_axis='mel', ax=axes[i, 0])
    axes[i, 0].set_title(f"{example['label']}: held-out real window")
    librosa.display.specshow(example['x_recon'], sr=GEN_SR,␣
 ↪hop_length=GEN_MEL_HOP, x_axis='time', y_axis='mel', ax=axes[i, 1])
    axes[i, 1].set_title(f"{example['label']}: beta-VAE reconstruction")
plt.tight_layout()
plt.show()

def prepare_generated_for_classifier(y_audio):
    y16 = librosa.resample(y_audio, orig_sr=GEN_SR, target_sr=SR)
    target_len = CLIP_SAMPLES
    if len(y16) < target_len:
        y16 = np.pad(y16, (0, target_len - len(y16)))
    else:
        y16 = y16[:target_len]
    mfcc = librosa.feature.mfcc(y=y16, sr=SR, n_mfcc=N_MFCC, hop_length=512).T
    if mfcc.shape[0] < T:
        mfcc = np.pad(mfcc, ((0, T - mfcc.shape[0]), (0, 0)))
    else:
        mfcc = mfcc[:T]
    mfcc_scaled = cnn_scaler.transform(mfcc).reshape(1, T, N_MFCC)
    probs = cnn_model_aug.predict(mfcc_scaled, verbose=0)[0]
    return probs

def audio_stats(y_audio, sr_audio):
    rms = librosa.feature.rms(y=y_audio, frame_length=min(1024, len(y_audio)),␣
 ↪hop_length=256)[0]
    centroid = librosa.feature.spectral_centroid(y=y_audio, sr=sr_audio,␣
 ↪n_fft=min(1024, len(y_audio)), hop_length=256)[0]
    f0 = librosa.yin(y_audio, fmin=80, fmax=250, sr=sr_audio,␣
 ↪frame_length=min(1024, len(y_audio)), hop_length=256)
```

```
    voiced = f0[np.isfinite(f0)]
    return {
        'rms_mean': float(np.mean(rms)),
        'silence_ratio': float(np.mean(rms < 0.02)),
        'spectral_centroid': float(np.mean(centroid)),
        'pitch_mean': float(np.mean(voiced)) if len(voiced) else np.nan,
    }

generated_eval_rows = []
generated_local_lookup = {rec['sample_id']: rec['local_path'] for rec in␣
 ↪generated_records}
train_flat_specs = X_gen_all_norm[train_mask_gen].reshape(X_gen_train.shape[0],␣
 ↪-1)
train_labels_gen = y_gen_train

for rec in generated_records:
    probs = prepare_generated_for_classifier(rec['audio'])
    pred_label = int(np.argmax(probs))
    pred_name = LABEL_NAMES[pred_label]
    stats = audio_stats(rec['audio'], GEN_SR)
    first_spec = rec['decoded_specs'][0][..., 0].reshape(1, -1)
    same_class_mask = train_labels_gen == rec['label']
    nearest = cosine_similarity(first_spec, train_flat_specs[same_class_mask]).
 ↪max()
    generated_eval_rows.append({
        'sample_id': rec['sample_id'],
        'intended_class': rec['class'],
        'predicted_class': pred_name,
        'pred_confidence': float(np.max(probs)),
        'nearest_train_cosine': float(nearest),
        **stats,
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
    })

generated_eval_df = pd.DataFrame(generated_eval_rows)

real_eval_rows = []
for _, row in gen_test_df.iterrows():
    idx = int(row['latent_index'])
    stats = audio_stats(kept_audio[idx], GEN_SR)
    real_eval_rows.append({
        'class': row['category'],
        **stats,
    })
real_eval_df = pd.DataFrame(real_eval_rows)
```

```
clf_cm = pd.crosstab(
    generated_eval_df['intended_class'],
    generated_eval_df['predicted_class'],
    rownames=['Intended'],
    colnames=['Predicted'],
    dropna=False,
).reindex(index=LABEL_NAMES, columns=LABEL_NAMES, fill_value=0)

print("Classifier-on-generated-audio confusion table:")
print(clf_cm.to_string())
print(f"\nGenerated context accuracy: {(generated_eval_df['intended_class'] ==␣
 ↪generated_eval_df['predicted_class']).mean():.1%}")

realism_summary = (
    generated_eval_df.groupby('intended_class')[['rms_mean', 'silence_ratio',␣
 ↪'spectral_centroid', 'pitch_mean']]
    .mean()
    .rename_axis('class')
    .reset_index()
)
real_summary = (
    real_eval_df.groupby('class')[['rms_mean', 'silence_ratio',␣
 ↪'spectral_centroid', 'pitch_mean']]
    .mean()
    .reset_index()
)

print("\nGenerated audio summary:")
print(realism_summary.to_string(index=False))
print("\nHeld-out real window summary:")
print(real_summary.to_string(index=False))

print("\nHighest nearest-neighbor cosine similarities:")
print(
    generated_eval_df[['sample_id', 'intended_class', 'nearest_train_cosine']]
    .sort_values('nearest_train_cosine', ascending=False)
    .head(10)
    .to_string(index=False)
)
```




```
Classifier-on-generated-audio confusion table:
Predicted     Mom  Bestfriend  Professional
Intended
Mom             1           0             0
Bestfriend      0           1             0
Professional    1           0             0

Generated context accuracy: 66.7%

Generated audio summary:
       class  rms_mean  silence_ratio  spectral_centroid  pitch_mean
  Bestfriend  0.094537       0.125000        1223.928105  115.425241
         Mom  0.110284       0.081395         857.799177  111.701929
Professional  0.070319       0.183140        1071.839214  113.205134

Held-out real window summary:
       class  rms_mean  silence_ratio  spectral_centroid  pitch_mean
```

```
  Bestfriend  0.117971       0.163635        1422.321187  126.597601
         Mom  0.100590       0.237056        1181.868687  119.674950
Professional  0.094832       0.211460        1238.539974  120.973957

Highest nearest-neighbor cosine similarities:
          sample_id intended_class  nearest_train_cosine
  gen-bestfriend-00     Bestfriend              0.699051
         gen-mom-00            Mom              0.605517
gen-professional-00   Professional              0.465744
```

The reconstruction plot is the clearest visual explanation for why the neural audio sounds blurry. In the left column, the held-out real windows have many crisp horizontal harmonic bands during voiced speech. For example, the Mom and Bestfriend real windows show striped low-frequency structure after the initial quieter region, and the Professional real window has a clear transition from quieter energy into voiced bands near the second half of the clip.

In the right column, the beta-VAE reconstructions preserve the rough timing and broad energy regions, but they smear the details. The narrow horizontal bands become large soft red and blue patches. The Mom reconstruction still shows a quiet first half-second and stronger energy later, but the formant/harmonic detail is blurred. Bestfriend has the same issue: broad vertical speech regions survive, but the fine stripes are mostly gone. Professional is the most dramatic: the model keeps the broad quiet-to-loud transition, but the reconstruction turns the detailed speech texture into smooth blocks.

The tables quantify the same mixed result. The generated clips are not silence: their RMS values are in a speech-like range, and their estimated pitch means sit around 112-115 Hz, close to the held-out real windows around 120-127 Hz. But the spectral centroids are generally lower than the real windows, which matches what I hear as muffled or low-detail audio. The classifier-on-generated table gives 66.7% accuracy for the three generated clips: Mom and Bestfriend are classified correctly, while Professional is heard as Mom. The nearest-neighbor cosine scores are moderate, about 0.47 to 0.70, so the clips are not exact copies of one training window, but they are also not clean new speech.

My honest interpretation is that the beta-VAE + LSTM technically works as machine-learning generation, but the result is blurry. The model learns when speech-like energy should happen, but it does not reconstruct the fine harmonic structure that makes a voice sound sharp and intelligible.

[12]:
```
gallery_paths = []
for label_name in LABEL_NAMES:
    class_rows = generated_eval_df[generated_eval_df['intended_class'] ==␣
 ↪label_name].copy()
    if class_rows.empty:
        continue
    best_row = class_rows.sort_values('pred_confidence', ascending=False).
 ↪iloc[0]
    local_path = generated_local_lookup[best_row['sample_id']]
    gallery_paths.append((label_name, local_path, best_row['repo_path'],␣
 ↪best_row['public_url']))
```

```
print("Neural beta-VAE + LSTM diagnostic gallery (blurry first attempt)")
print("These are the original neural-generation outputs. They are included␣
 ↪honestly because this model technically generates audio, but the sound is␣
 ↪smeared.")
for label_name, local_path, repo_path, public_url in gallery_paths:
    print(f"\n{label_name}")
    print(f"Repository path: {repo_path}")
    print(f"Public URL: {public_url}")
    display(Audio(local_path, rate=GEN_SR))

print("\nNeural generation manifest preview:")
print(audio_manifest_df.tail(12).to_string(index=False))
```

```
Neural beta-VAE + LSTM diagnostic gallery (blurry first attempt)
These are the original neural-generation outputs. They are included honestly
because this model technically generates audio, but the sound is smeared.

Mom
Repository path: assignment3_assets/audio/generated_mom_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_mom_00.wav
<IPython.lib.display.Audio object>


Bestfriend
Repository path: assignment3_assets/audio/generated_bestfriend_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_bestfriend_00.wav
<IPython.lib.display.Audio object>


Professional
Repository path: assignment3_assets/audio/generated_professional_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_professional_00.wav
<IPython.lib.display.Audio object>


Neural generation manifest preview:
            sample_id        class           type
description                                                repo_path
public_url
         original-mom          Mom  original_clip Original 10-second clip used
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_mom.wav
  original-bestfriend   Bestfriend  original_clip Original 10-second clip used
```

```
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_bestfriend.wav
original-professional Professional  original_clip Original 10-second clip used
in the waveform/audio analysis section
assignment3_assets/audio/original_clip_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/original_clip_professional.wav
             real-mom          Mom           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_mom.wav
            recon-mom          Mom reconstruction                              beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_mom.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_mom.wav
      real-bestfriend   Bestfriend           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_bestfriend.wav
     recon-bestfriend   Bestfriend reconstruction                              beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_bestfriend.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_bestfriend.wav
    real-professional Professional           real
Held-out 2-second reference window
assignment3_assets/audio/real_window_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/real_window_professional.wav
   recon-professional Professional reconstruction                              beta-
VAE reconstruction of held-out window
assignment3_assets/audio/reconstruction_professional.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/reconstruction_professional.wav
           gen-mom-00          Mom      generated                    Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_mom_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_mom_00.wav
    gen-bestfriend-00   Bestfriend      generated                    Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_bestfriend_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
```

```
switching/main/assignment3_assets/audio/generated_bestfriend_00.wav
  gen-professional-00 Professional      generated                    Free
generation seeded from held-out class latent
assignment3_assets/audio/generated_professional_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/generated_professional_00.wav
```

5.5 Neural Improvement Attempt: Conditional Autoencoder with a Transformer Prior

The beta-VAE + LSTM is my actual first machine-learning generation attempt, but it has two problems. First, the beta-VAE objective intentionally pushes the latent space toward a smooth Gaussian prior. That is mathematically elegant, but on speech it can erase the narrow harmonic bands that make a voice sound crisp. Second, the LSTM prior only sees the recent latent sequence through a recurrent hidden state, which can collapse into a bland average trajectory.

To improve the neural branch, I try a second architecture: a **conditional deterministic convolutional autoencoder** plus a **Transformer latent prior**.

The autoencoder keeps the useful encoder-decoder structure from the VAE, but removes the KL penalty:

$$z = f_\phi(x, c), \qquad \hat{x} = g_\theta(z, c)$$

and trains only for reconstruction:

$$\mathcal{L}_{AE} = \text{Huber}(x, \hat{x})$$

This should preserve sharper spectrogram detail because the model is no longer being punished for using a rich latent representation. Then I train a small Transformer prior to predict the next standardized latent from the previous latent windows and the class label:

$$\hat{z}_{t+1} = T_\psi(z_{t-k+1}, \dots, z_t, c)$$

The point is not that this will suddenly become perfect text-to-speech. It is an improvement attempt aimed at the specific failure I saw: blurry VAE reconstructions and weak LSTM dynamics. If it works, the generated clips should sound less washed out than the beta-VAE + LSTM clips, even if they are still not as clean as real waveform snippets.

[13]:
```
AE_LATENT_DIM = 64
AE_COND_EMB_DIM = 24
AE_INPUT_SHAPE = X_gen_train.shape[1:]


def build_conditional_autoencoder(input_shape, latent_dim, cond_dim=3,␣
 ↪cond_emb_dim=AE_COND_EMB_DIM):
    mel_h, mel_w = int(input_shape[0]), int(input_shape[1])
    spec_in = layers.Input(shape=input_shape, name='ae_spec_input')
```

```
    label_in = layers.Input(shape=(cond_dim,), name='ae_label_input')
    label_emb = layers.Dense(cond_emb_dim, activation='tanh',␣
↪name='ae_label_emb_enc')(label_in)

    x = layers.Conv2D(16, 3, strides=2, padding='same')(spec_in)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)
    x = TileLabelConcat()([x, label_emb])

    x = layers.Conv2D(32, 3, strides=2, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)
    x = TileLabelConcat()([x, label_emb])

    x = layers.Conv2D(64, 3, strides=2, padding='same')(x)
    x = layers.BatchNormalization()(x)
    x = layers.LeakyReLU(0.1)(x)

    shape_before_flatten = tf.keras.backend.int_shape(x)[1:]
    x = layers.Flatten()(x)
    x = layers.Concatenate()([x, label_emb])
    x = layers.Dense(192, activation='relu')(x)
    x = layers.Dropout(0.15)(x)
    z = layers.Dense(latent_dim, name='ae_latent')(x)
    ae_encoder = models.Model([spec_in, label_in], z,␣
↪name='conditional_deterministic_encoder')

    z_in = layers.Input(shape=(latent_dim,), name='ae_latent_input')
    label_dec_in = layers.Input(shape=(cond_dim,), name='ae_label_decoder')
    label_emb_dec = layers.Dense(cond_emb_dim, activation='tanh',␣
↪name='ae_label_emb_dec')(label_dec_in)

    d = layers.Concatenate()([z_in, label_emb_dec])
    d = layers.Dense(np.prod(shape_before_flatten), activation='relu')(d)
    d = layers.Reshape(shape_before_flatten)(d)

    d = layers.Conv2DTranspose(64, 3, strides=2, padding='same')(d)
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)
    d = TileLabelConcat()([d, label_emb_dec])

    d = layers.Conv2DTranspose(32, 3, strides=2, padding='same')(d)
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)
    d = TileLabelConcat()([d, label_emb_dec])

    d = layers.Conv2DTranspose(16, 3, strides=2, padding='same')(d)
```

```
    d = layers.BatchNormalization()(d)
    d = layers.LeakyReLU(0.1)(d)
    d = CropToMel(mel_h, mel_w)(d)
    out = layers.Conv2D(1, 3, padding='same', activation='linear',␣
 ↪name='ae_reconstruction')(d)
    ae_decoder = models.Model([z_in, label_dec_in], out,␣
 ↪name='conditional_deterministic_decoder')
    return ae_encoder, ae_decoder


ae_encoder, ae_decoder = build_conditional_autoencoder(AE_INPUT_SHAPE,␣
 ↪AE_LATENT_DIM)
ae_spec_in = layers.Input(shape=AE_INPUT_SHAPE, name='ae_full_spec')
ae_label_in = layers.Input(shape=(3,), name='ae_full_label')
ae_z = ae_encoder([ae_spec_in, ae_label_in])
ae_recon = ae_decoder([ae_z, ae_label_in])
ae_model = models.Model([ae_spec_in, ae_label_in], ae_recon,␣
 ↪name='conditional_autoencoder')
ae_model.compile(
    optimizer=tf.keras.optimizers.Adam(5e-4, clipnorm=1.0),
    loss=tf.keras.losses.Huber(delta=1.0),
    metrics=[tf.keras.metrics.MeanAbsoluteError(name='mae')],
)

ae_encoder_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR,␣
 ↪'conditional_ae_encoder.weights.h5')
ae_decoder_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR,␣
 ↪'conditional_ae_decoder.weights.h5')

if USE_SAVED_ASSIGNMENT3_WEIGHTS and os.path.exists(ae_encoder_weights_path)␣
 ↪and os.path.exists(ae_decoder_weights_path):
    ae_encoder.load_weights(ae_encoder_weights_path)
    ae_decoder.load_weights(ae_decoder_weights_path)
    ae_history = None
    print("Loaded saved deterministic autoencoder weights for fast␣
 ↪generation-section rerun.")
else:
    ae_history = ae_model.fit(
        [X_gen_train, y_gen_train_oh],
        X_gen_train,
        validation_data=([X_gen_val, y_gen_val_oh], X_gen_val),
        epochs=10,
        batch_size=64,
        verbose=0,
        callbacks=[
            callbacks.TerminateOnNaN(),
```

```
            callbacks.EarlyStopping(monitor='val_loss', mode='min', patience=4,␣
 ↪restore_best_weights=True),
            callbacks.ReduceLROnPlateau(monitor='val_loss', mode='min',␣
 ↪factor=0.5, patience=3, min_lr=1e-5),
        ],
    )
    ae_encoder.save_weights(ae_encoder_weights_path)
    ae_decoder.save_weights(ae_decoder_weights_path)

if ae_history is not None:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(ae_history.history['loss'], label='Train Huber', color='#4C72B0',␣
 ↪lw=2)
    ax.plot(ae_history.history['val_loss'], label='Val Huber', color='#DD8452',␣
 ↪lw=2)
    ax.plot(ae_history.history['mae'], label='Train MAE', color='#55A868', lw=1.
 ↪8, alpha=0.8)
    ax.plot(ae_history.history['val_mae'], label='Val MAE', color='#C44E52',␣
 ↪lw=1.8, alpha=0.8)
    ax.set_title('Conditional autoencoder training curves')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('Loss')
    ax.grid(True, alpha=0.3)
    ax.legend()
    plt.tight_layout()
    plt.show()
    print(f"Best AE validation loss: {np.min(ae_history.history['val_loss']):.
 ↪4f}")
    print(f"Best AE validation MAE: {np.min(ae_history.history['val_mae']):.
 ↪4f}")
else:
    print("Autoencoder training skipped because saved weights were available.")
print(f"AE encoder parameters: {ae_encoder.count_params():,}")
print(f"AE decoder parameters: {ae_decoder.count_params():,}")
```

```
Loaded saved deterministic autoencoder weights for fast generation-section
rerun.
Autoencoder training skipped because saved weights were available.
AE encoder parameters: 2,027,808
AE decoder parameters: 982,433
```

[14]:
```
ae_latents_all = ae_encoder.predict(
    [X_gen_all_norm, tf.keras.utils.to_categorical(y_gen_all, num_classes=3)],
    verbose=0,
)
ae_latent_mean = ae_latents_all[train_mask_gen].mean(axis=0, keepdims=True)
ae_latent_std = ae_latents_all[train_mask_gen].std(axis=0, keepdims=True) + 1e-6
```

```
ae_latents_scaled = ((ae_latents_all - ae_latent_mean) / ae_latent_std).
 ↪astype(np.float32)

gen_df['ae_latent_index'] = np.arange(len(gen_df))
gen_train_df = gen_df[gen_df['split'] == 'train'].copy()
gen_val_df = gen_df[gen_df['split'] == 'val'].copy()
gen_test_df = gen_df[gen_df['split'] == 'test'].copy()
AE_CONTEXT = 8


def build_ae_latent_sequences(frame_df, latents_scaled):
    seq_x, seq_y, seq_labels = [], [], []
    for note_key in sorted(frame_df['note_key'].unique()):
        note_rows = frame_df[frame_df['note_key'] == note_key].
 ↪sort_values('start_sec')
        note_latents = latents_scaled[note_rows['ae_latent_index'].to_numpy()]
        note_label = int(note_rows['label'].iloc[0])
        if len(note_latents) < 2:
            continue
        for t in range(1, len(note_latents)):
            hist = note_latents[max(0, t - AE_CONTEXT):t]
            if hist.shape[0] < AE_CONTEXT:
                pad = np.zeros((AE_CONTEXT - hist.shape[0], latents_scaled.
 ↪shape[1]), dtype=np.float32)
                hist = np.vstack([pad, hist])
            seq_x.append(hist.astype(np.float32))
            seq_y.append(note_latents[t].astype(np.float32))
            seq_labels.append(note_label)
    return np.array(seq_x), np.array(seq_y), tf.keras.utils.to_categorical(np.
 ↪array(seq_labels), num_classes=3)


X_ae_prior_train, y_ae_prior_train, y_ae_prior_train_oh =␣
 ↪build_ae_latent_sequences(gen_train_df, ae_latents_scaled)
X_ae_prior_val, y_ae_prior_val, y_ae_prior_val_oh =␣
 ↪build_ae_latent_sequences(gen_val_df, ae_latents_scaled)

prior_in = layers.Input(shape=(AE_CONTEXT, AE_LATENT_DIM),␣
 ↪name='ae_prior_history')
prior_label_in = layers.Input(shape=(3,), name='ae_prior_label')
label_token = layers.Dense(32, activation='tanh')(prior_label_in)
label_rep = layers.RepeatVector(AE_CONTEXT)(label_token)

x = layers.Dense(96)(prior_in)
x = layers.Concatenate(axis=-1)([x, label_rep])
x = layers.Dense(96)(x)
```

```
positions = tf.range(start=0, limit=AE_CONTEXT, delta=1)
pos_emb = layers.Embedding(input_dim=AE_CONTEXT, output_dim=96)(positions)
x = x + pos_emb

for _ in range(2):
    attn = layers.MultiHeadAttention(num_heads=3, key_dim=32, dropout=0.1)(x, x)
    x = layers.LayerNormalization()(x + attn)
    ff = layers.Dense(192, activation='relu')(x)
    ff = layers.Dropout(0.1)(ff)
    ff = layers.Dense(96)(ff)
    x = layers.LayerNormalization()(x + ff)

x = layers.GlobalAveragePooling1D()(x)
x = layers.Concatenate()([x, label_token])
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.15)(x)
prior_out = layers.Dense(AE_LATENT_DIM)(x)
ae_transformer_prior = models.Model([prior_in, prior_label_in], prior_out,␣
 ↪name='ae_transformer_latent_prior')
ae_transformer_prior.compile(optimizer=tf.keras.optimizers.Adam(8e-4,␣
 ↪clipnorm=1.0), loss='mse')

ae_transformer_prior_weights_path = os.path.join(ASSIGNMENT3_MODEL_DIR,␣
 ↪'ae_transformer_prior.weights.h5')

if USE_SAVED_ASSIGNMENT3_WEIGHTS and os.path.
 ↪exists(ae_transformer_prior_weights_path):
    ae_transformer_prior.load_weights(ae_transformer_prior_weights_path)
    ae_prior_history = None
    print("Loaded saved AE Transformer prior weights for fast␣
 ↪generation-section rerun.")
else:
    ae_prior_history = ae_transformer_prior.fit(
        [X_ae_prior_train, y_ae_prior_train_oh],
        y_ae_prior_train,
        validation_data=([X_ae_prior_val, y_ae_prior_val_oh], y_ae_prior_val),
        epochs=12,
        batch_size=64,
        verbose=0,
        callbacks=[
            callbacks.TerminateOnNaN(),
            callbacks.EarlyStopping(monitor='val_loss', mode='min', patience=4,␣
 ↪restore_best_weights=True),
            callbacks.ReduceLROnPlateau(monitor='val_loss', mode='min',␣
 ↪factor=0.5, patience=3, min_lr=1e-5),
        ],
```

```
    )
    ae_transformer_prior.save_weights(ae_transformer_prior_weights_path)

if ae_prior_history is not None:
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(ae_prior_history.history['loss'], label='Train MSE',␣
 ↪color='#4C72B0', lw=2)
    ax.plot(ae_prior_history.history['val_loss'], label='Val MSE',␣
 ↪color='#DD8452', lw=2)
    ax.set_title('Transformer latent prior training curves')
    ax.set_xlabel('Epoch')
    ax.set_ylabel('MSE')
    ax.grid(True, alpha=0.3)
    ax.legend()
    plt.tight_layout()
    plt.show()
    print(f"Best AE Transformer validation loss: {np.min(ae_prior_history.
 ↪history['val_loss']):.4f}")
else:
    print("AE Transformer prior training skipped because saved weights were␣
 ↪available.")

print(f"AE Transformer sequence examples: train={len(X_ae_prior_train)},␣
 ↪val={len(X_ae_prior_val)}")
```


```
Best AE Transformer validation loss: 0.8762
AE Transformer sequence examples: train=1737, val=484
```

[15]:
```
ae_generated_records = []
AE_GENERATION_WINDOWS = 6
AE_STEP_NOISE = 0.04
ae_rng = np.random.default_rng(314)

seed_pools = {}
for label_idx in range(3):
    test_rows = gen_test_df[gen_test_df['label'] == label_idx]
    train_rows = gen_train_df[gen_train_df['label'] == label_idx]
    seed_rows = test_rows if len(test_rows) else train_rows
    seed_pools[label_idx] = ae_latents_scaled[seed_rows['ae_latent_index'].
 ↪to_numpy()]

for label_idx, label_name in enumerate(LABEL_NAMES):
    cond = tf.keras.utils.to_categorical([label_idx], num_classes=3)
    seed_pool = seed_pools[label_idx]
    start_z = seed_pool[ae_rng.integers(0, len(seed_pool))].astype(np.float32)
    latent_sequence = [start_z]

    for _ in range(1, AE_GENERATION_WINDOWS):
        hist = np.array(latent_sequence[-AE_CONTEXT:], dtype=np.float32)
        if hist.shape[0] < AE_CONTEXT:
            pad = np.zeros((AE_CONTEXT - hist.shape[0], AE_LATENT_DIM),␣
 ↪dtype=np.float32)
            hist = np.vstack([pad, hist])
        z_next = ae_transformer_prior.predict([hist[np.newaxis, ...], cond],␣
 ↪verbose=0)[0]
        z_next = z_next + ae_rng.normal(0, AE_STEP_NOISE, size=AE_LATENT_DIM).
 ↪astype(np.float32)
        latent_sequence.append(z_next.astype(np.float32))

    latent_scaled_arr = np.array(latent_sequence, dtype=np.float32)
    latent_arr = (latent_scaled_arr * ae_latent_std) + ae_latent_mean
    cond_stack = np.repeat(cond, repeats=len(latent_arr), axis=0)
    decoded_specs = ae_decoder.predict([latent_arr, cond_stack], verbose=0)
    decoded_audio = [inverse_logmel_to_audio(spec, n_iter=80) for spec in␣
 ↪decoded_specs]
    clip = final_normalize(overlap_add_audio(decoded_audio))

    filename = f"ae_transformer_generated_{label_name.lower()}.wav"
    file_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, filename)
    sf.write(file_path, clip, GEN_SR)

    ae_generated_records.append({
        'sample_id': f"ae-transformer-{label_name.lower()}",
        'class': label_name,
        'label': label_idx,
```

```
        'type': 'ae_transformer_generated',
        'description': 'Deterministic autoencoder plus Transformer latent-prior␣
 ↪generation',
        'local_path': file_path,
        'repo_path': repo_audio_path(filename),
        'public_url': f"{GITHUB_RAW_BASE}/{filename}",
        'audio': clip,
        'decoded_specs': decoded_specs,
    })

ae_manifest_rows = pd.DataFrame([
    {
        'sample_id': rec['sample_id'],
        'class': rec['class'],
        'type': rec['type'],
        'description': rec['description'],
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
    }
    for rec in ae_generated_records
])

audio_manifest_df = (
    pd.concat([audio_manifest_df, ae_manifest_rows], ignore_index=True)
    .drop_duplicates('sample_id', keep='last')
)
audio_manifest_df.to_csv(os.path.join(ASSIGNMENT3_AUDIO_DIR, 'audio_manifest.
 ↪csv'), index=False)

print("Autoencoder + Transformer neural generation gallery (improvement␣
 ↪attempt)")
for rec in ae_generated_records:
    print(f"\n{rec['class']}")
    print(f"Repository path: {rec['repo_path']}")
    print(f"Public URL: {rec['public_url']}")
    display(Audio(rec['local_path'], rate=GEN_SR))
```

```
Autoencoder + Transformer neural generation gallery (improvement attempt)

Mom
Repository path: assignment3_assets/audio/ae_transformer_generated_mom.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/ae_transformer_generated_mom.wav
```

```
<IPython.lib.display.Audio object>
```

```
Bestfriend
```

```
Repository path:
assignment3_assets/audio/ae_transformer_generated_bestfriend.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/ae_transformer_generated_bestfriend.wav
```

```
<IPython.lib.display.Audio object>
```

```
Professional
Repository path:
assignment3_assets/audio/ae_transformer_generated_professional.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-switchin
g/main/assignment3_assets/audio/ae_transformer_generated_professional.wav
```

```
<IPython.lib.display.Audio object>
```

[16]:
```
ae_eval_rows = []
for rec in ae_generated_records:
    probs = prepare_generated_for_classifier(rec['audio'])
    pred_label = int(np.argmax(probs))
    stats = audio_stats(rec['audio'], GEN_SR)
    ae_eval_rows.append({
        'sample_id': rec['sample_id'],
        'intended_class': rec['class'],
        'predicted_class': LABEL_NAMES[pred_label],
        'pred_confidence': float(np.max(probs)),
        **stats,
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
    })

ae_eval_df = pd.DataFrame(ae_eval_rows)
ae_cm = pd.crosstab(
    ae_eval_df['intended_class'],
    ae_eval_df['predicted_class'],
    rownames=['Intended'],
    colnames=['Predicted'],
    dropna=False,
).reindex(index=LABEL_NAMES, columns=LABEL_NAMES, fill_value=0)

print("Autoencoder + Transformer classifier confusion table:")
print(ae_cm.to_string())
print(f"\nAE+Transformer context accuracy: {(ae_eval_df['intended_class'] ==␣
 ↪ae_eval_df['predicted_class']).mean():.1%}")
print("\nAE+Transformer acoustic summary:")
print(
    ae_eval_df[['intended_class', 'rms_mean', 'silence_ratio',␣
 ↪'spectral_centroid', 'pitch_mean', 'predicted_class', 'pred_confidence']]
    .to_string(index=False)
```

```
)
```

```
Autoencoder + Transformer classifier confusion table:
Predicted     Mom  Bestfriend  Professional
Intended
Mom             1           0             0
Bestfriend      0           1             0
Professional    1           0             0

AE+Transformer context accuracy: 66.7%

AE+Transformer acoustic summary:
intended_class  rms_mean  silence_ratio  spectral_centroid  pitch_mean
predicted_class  pred_confidence
           Mom  0.161048       0.067376        1076.633159  112.574662
Mom         0.646490
    Bestfriend  0.085380       0.070922        1201.662176  114.612873
Bestfriend         0.588372
  Professional  0.132139       0.053191        1159.346691  113.175014
Mom         0.456885
```

The autoencoder + Transformer clips are the improved neural attempt. This is still machine-learning generation: the model encodes spectrogram windows into learned latent vectors, predicts future latent vectors with attention, decodes them into new spectrograms, and then converts those spectrograms to waveform.

The improvement is targeted at the beta-VAE failure. Removing the KL term lets the autoencoder preserve a richer latent representation, and the Transformer prior can attend over recent latent windows instead of compressing everything through an LSTM hidden state. The evaluation shows a partial improvement, not a miracle: the classifier again gets 66.7% context accuracy, correctly hearing Mom and Bestfriend but still confusing Professional as Mom. The generated pitch estimates remain speech-like, around 113-115 Hz.

Listening matters here. These autoencoder + Transformer clips still sound blurry and somewhat random, like a washed-out version of voice energy rather than a clean generated sentence. The likely reasons are the small personal dataset, the spectrogram bottleneck, and Griffin-Lim phase reconstruction. The model predicts a plausible mel-spectrogram texture, but waveform detail and phase are not learned by a proper neural vocoder, so the audio stays smeared.

5.6 Non-Neural Baseline: Unit-Selection Voice Synthesis

After the two neural attempts, I include unit selection as a classic non-neural speech-generation baseline. This is important because it answers a slightly different question: if a small learned spectrogram generator sounds blurry, can a retrieval-based speech synthesizer produce clearer audio from the same personal archive?

Unit selection, also called concatenative synthesis, does not learn to create a waveform from scratch. Instead, it builds new clips by selecting short real units from my own training voice notes and stitching them together with crossfades. This is why it sounds more natural but also why I should not oversell it as "the computer generating my voice" in the same way as the VAE. The generation

is in the sequence design: the system chooses a new path through many recorded snippets based on acoustic similarity, class label, randomness, and source-note penalties.

For each candidate unit u_i, I compute an acoustic descriptor f_i from MFCC means and variances, MFCC deltas, RMS, silence ratio, spectral centroid, and zero-crossing rate. When the synthesizer needs a next unit, it either continues briefly within the same source note for intelligibility or jumps to one of the nearest units in the same target class:

$$\text{score}(u_j \mid u_i, c) = \|\tilde{f}_j - \tilde{f}_i\|_2 + \lambda_1 \mathbf{1}[\text{recent}(j)] + \lambda_2 \mathbf{1}[\text{same note jump}]$$

The next unit is sampled from the best-scoring candidates rather than always taking the single nearest neighbor. That stochastic step matters because otherwise the system would just replay one locally smooth path. Adjacent units are joined with an equal-power crossfade:

$$y[t] = y_a[t]\cos(\theta_t) + y_b[t]\sin(\theta_t), \qquad \theta_t \in [0, \pi/2]$$

The honest framing is: the beta-VAE + LSTM is my actual machine-learning generation attempt. It technically works but produces blurry audio. I then compare it to unit-selection synthesis, a classic non-neural speech generation baseline, which produces clearer audio by recombining real snippets. This comparison shows the tradeoff between learned generation and waveform fidelity on a small personal dataset.

[17]:
```
UNIT_SR = GEN_SR
UNIT_SEC = 0.75
UNIT_HOP_SEC = 0.32
UNIT_SAMPLES = int(UNIT_SR * UNIT_SEC)
UNIT_HOP_SAMPLES = int(UNIT_SR * UNIT_HOP_SEC)
UNIT_CROSSFADE = int(0.08 * UNIT_SR)
UNIT_N_MFCC = 13
N_UNIT_GENERATED_PER_CLASS = 1
N_UNIT_SEQUENCE = 9
UNIT_CONTINUE_PROB = 0.70
UNIT_MAX_CONTIGUOUS = 3
UNIT_TOP_K = 35
UNIT_TEMPERATURE = 0.35

unit_audio_candidates = []
unit_feature_candidates = []
unit_rows = []


def unit_feature_vector(y):
    mfcc = librosa.feature.mfcc(
        y=y,
        sr=UNIT_SR,
        n_mfcc=UNIT_N_MFCC,
        n_fft=GEN_N_FFT,
```

```
        hop_length=GEN_MEL_HOP,
    )
    delta = librosa.feature.delta(mfcc)
    rms = librosa.feature.rms(y=y, frame_length=GEN_N_FFT,␣
 ↪hop_length=GEN_MEL_HOP)[0]
    centroid = librosa.feature.spectral_centroid(y=y, sr=UNIT_SR,␣
 ↪n_fft=GEN_N_FFT, hop_length=GEN_MEL_HOP)[0]
    zcr = librosa.feature.zero_crossing_rate(y, frame_length=GEN_N_FFT,␣
 ↪hop_length=GEN_MEL_HOP)[0]
    return np.concatenate([
        mfcc.mean(axis=1),
        mfcc.std(axis=1),
        delta.mean(axis=1),
        [
            float(rms.mean()),
            float(np.mean(rms < 0.02)),
            float(centroid.mean()),
            float(zcr.mean()),
        ],
    ]).astype(np.float32)


for cat in CATEGORIES:
    wav_dir = os.path.join(WAV_DIR, cat)
    for wav_path in sorted(glob.glob(os.path.join(wav_dir, '*.wav'))):
        base = os.path.splitext(os.path.basename(wav_path))[0]
        note_key = f"{cat}_{base}"
        if note_key not in train_note_keys:
            continue

        y_note, _ = librosa.load(wav_path, sr=UNIT_SR)
        y_note = peak_normalize(y_note).astype(np.float32)
        if len(y_note) < UNIT_SAMPLES:
            continue

        for start in range(0, len(y_note) - UNIT_SAMPLES + 1, UNIT_HOP_SAMPLES):
            chunk = y_note[start:start + UNIT_SAMPLES].astype(np.float32)
            rms = librosa.feature.rms(y=chunk, frame_length=GEN_N_FFT,␣
 ↪hop_length=GEN_MEL_HOP)[0]
            rms_mean = float(rms.mean())
            silence_ratio = float(np.mean(rms < 0.02))

            unit_rows.append({
                'candidate_index': len(unit_audio_candidates),
                'note_key': note_key,
                'category': LABEL_NAMES[LABEL_MAP[cat]],
                'label': LABEL_MAP[cat],
```

```
                'wav_path': wav_path,
                'start_sec': start / UNIT_SR,
                'stop_sec': (start + UNIT_SAMPLES) / UNIT_SR,
                'rms_mean': rms_mean,
                'silence_ratio': silence_ratio,
            })
            unit_audio_candidates.append(chunk)
            unit_feature_candidates.append(unit_feature_vector(chunk))

unit_candidate_df = pd.DataFrame(unit_rows)
rms_floor_by_label = unit_candidate_df.groupby('label')['rms_mean'].
 ↪transform(lambda s: s.quantile(0.25))
unit_keep_mask = (unit_candidate_df['silence_ratio'] <= 0.55) &␣
 ↪(unit_candidate_df['rms_mean'] >= rms_floor_by_label)
unit_df = unit_candidate_df.loc[unit_keep_mask].copy().reset_index(drop=True)
unit_audio = [unit_audio_candidates[i] for i in unit_df['candidate_index']]
unit_features = np.stack([unit_feature_candidates[i] for i in␣
 ↪unit_df['candidate_index']])
unit_scaler = StandardScaler()
unit_features_scaled = unit_scaler.fit_transform(unit_features)
unit_df['unit_index'] = np.arange(len(unit_df))

next_unit = {}
for note_key in sorted(unit_df['note_key'].unique()):
    note_indices = unit_df[unit_df['note_key'] == note_key].
 ↪sort_values('start_sec')['unit_index'].to_list()
    for left, right in zip(note_indices[:-1], note_indices[1:]):
        next_unit[left] = right


def equal_power_crossfade(left, right, n_fade=UNIT_CROSSFADE):
    if len(left) == 0:
        return right.copy()
    n_fade = min(n_fade, len(left), len(right))
    theta = np.linspace(0, np.pi / 2, n_fade, dtype=np.float32)
    fade_out = np.cos(theta)
    fade_in = np.sin(theta)
    blended = left[-n_fade:] * fade_out + right[:n_fade] * fade_in
    return np.concatenate([left[:-n_fade], blended, right[n_fade:]]).astype(np.
 ↪float32)


def stitch_units(chunks):
    out = np.array([], dtype=np.float32)
    for chunk in chunks:
        out = equal_power_crossfade(out, chunk.astype(np.float32))
    return final_normalize(out)
```

```
def choose_next_unit(current_idx, label_idx, recent, contiguous_run, rng):
    current_row = unit_df.iloc[current_idx]
    if (
        rng.random() < UNIT_CONTINUE_PROB
        and current_idx in next_unit
        and contiguous_run < UNIT_MAX_CONTIGUOUS
    ):
        candidate_next = next_unit[current_idx]
        if int(unit_df.iloc[candidate_next]['label']) == label_idx:
            return candidate_next, contiguous_run + 1

    class_indices = unit_df.index[unit_df['label'] == label_idx].to_numpy()
    jump_indices = unit_df.index[(unit_df['label'] == label_idx) &␣
 ↪(unit_df['note_key'] != current_row['note_key'])].to_numpy()
    if len(jump_indices) >= min(UNIT_TOP_K, len(class_indices)):
        class_indices = jump_indices
    distances = np.linalg.norm(unit_features_scaled[class_indices] -␣
 ↪unit_features_scaled[current_idx], axis=1)

    recent_set = set(recent[-4:])
    penalties = np.array([0.40 if idx in recent_set else 0.0 for idx in␣
 ↪class_indices])
    penalties += np.array([0.18 if unit_df.iloc[idx]['note_key'] ==␣
 ↪current_row['note_key'] else 0.0 for idx in class_indices])
    scores = distances + penalties

    order = np.argsort(scores)[:min(UNIT_TOP_K, len(scores))]
    top_indices = class_indices[order]
    top_scores = scores[order]
    weights = np.exp(-(top_scores - top_scores.min()) / UNIT_TEMPERATURE)
    weights = weights / weights.sum()
    return int(rng.choice(top_indices, p=weights)), 1


unit_generated_records = []
unit_rng = np.random.default_rng(156)

for label_idx, label_name in enumerate(LABEL_NAMES):
    class_indices = unit_df.index[unit_df['label'] == label_idx].to_numpy()
    for sample_num in range(N_UNIT_GENERATED_PER_CLASS):
        current_idx = int(unit_rng.choice(class_indices))
        chosen = [current_idx]
        contiguous_run = 1
        for _ in range(1, N_UNIT_SEQUENCE):
```

```
            current_idx, contiguous_run = choose_next_unit(current_idx,␣
 ↪label_idx, chosen, contiguous_run, unit_rng)
            chosen.append(current_idx)

        chunks = [unit_audio[i] for i in chosen]
        clip = stitch_units(chunks)
        filename = f"unit_generated_{label_name.lower()}_{sample_num:02d}.wav"
        file_path = os.path.join(ASSIGNMENT3_AUDIO_DIR, filename)
        sf.write(file_path, clip, UNIT_SR)

        source_notes = unit_df.iloc[chosen]['note_key'].to_list()
        unit_generated_records.append({
            'sample_id': f"unit-gen-{label_name.lower()}-{sample_num:02d}",
            'class': label_name,
            'label': label_idx,
            'type': 'unit_selection_generated',
            'description': 'Unit-selection generation from short training␣
 ↪snippets with acoustic matching and crossfades',
            'local_path': file_path,
            'repo_path': repo_audio_path(filename),
            'public_url': f"{GITHUB_RAW_BASE}/{filename}",
            'audio': clip,
            'chosen_units': chosen,
            'source_notes': source_notes,
            'unique_source_notes': len(set(source_notes)),
        })

unit_manifest_rows = pd.DataFrame([
    {
        'sample_id': rec['sample_id'],
        'class': rec['class'],
        'type': rec['type'],
        'description': rec['description'],
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
    }
    for rec in unit_generated_records
])

audio_manifest_df = (
    pd.concat([audio_manifest_df, unit_manifest_rows], ignore_index=True)
    .drop_duplicates('sample_id', keep='last')
)
audio_manifest_df.to_csv(os.path.join(ASSIGNMENT3_AUDIO_DIR, 'audio_manifest.
 ↪csv'), index=False)

unit_counts = unit_df.groupby('category').size().reindex(LABEL_NAMES)
```

```
print("Usable unit-selection snippets by class:")
print(unit_counts.to_string())
print(f"\nUnit length: {UNIT_SEC:.2f}s, hop: {UNIT_HOP_SEC:.2f}s, crossfade:␣
  ↪{UNIT_CROSSFADE / UNIT_SR:.2f}s")
print(f"Generated {len(unit_generated_records)} unit-selection clips.")
print(unit_manifest_rows.head(9).to_string(index=False))
```

```
Usable unit-selection snippets by class:
category
Mom             1027
Bestfriend      1334
Professional    1059

Unit length: 0.75s, hop: 0.32s, crossfade: 0.08s
Generated 3 unit-selection clips.
               sample_id        class                      type
description                                                    repo_path
public_url
         unit-gen-mom-00          Mom unit_selection_generated Unit-selection
generation from short training snippets with acoustic matching and crossfades
assignment3_assets/audio/unit_generated_mom_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_mom_00.wav
  unit-gen-bestfriend-00   Bestfriend unit_selection_generated Unit-selection
generation from short training snippets with acoustic matching and crossfades
assignment3_assets/audio/unit_generated_bestfriend_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_bestfriend_00.wav
unit-gen-professional-00 Professional unit_selection_generated Unit-selection
generation from short training snippets with acoustic matching and crossfades
assignment3_assets/audio/unit_generated_professional_00.wav
https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_professional_00.wav
```

[18]:
```
unit_eval_rows = []
for rec in unit_generated_records:
    probs = prepare_generated_for_classifier(rec['audio'])
    pred_label = int(np.argmax(probs))
    stats = audio_stats(rec['audio'], UNIT_SR)
    unit_eval_rows.append({
        'sample_id': rec['sample_id'],
        'intended_class': rec['class'],
        'predicted_class': LABEL_NAMES[pred_label],
        'pred_confidence': float(np.max(probs)),
        'unique_source_notes': rec['unique_source_notes'],
        'duration_sec': len(rec['audio']) / UNIT_SR,
        **stats,
```

```
        'repo_path': rec['repo_path'],
        'public_url': rec['public_url'],
    })

unit_eval_df = pd.DataFrame(unit_eval_rows)
unit_cm = pd.crosstab(
    unit_eval_df['intended_class'],
    unit_eval_df['predicted_class'],
    rownames=['Intended'],
    colnames=['Predicted'],
    dropna=False,
).reindex(index=LABEL_NAMES, columns=LABEL_NAMES, fill_value=0)

print("Unit-selection classifier confusion table:")
print(unit_cm.to_string())
print(f"\nUnit-selection context accuracy: {(unit_eval_df['intended_class'] ==␣
 ↪unit_eval_df['predicted_class']).mean():.1%}")
print("\nUnit-selection acoustic summary:")
print(
    unit_eval_df.groupby('intended_class')[['rms_mean', 'silence_ratio',␣
 ↪'spectral_centroid', 'pitch_mean', 'unique_source_notes']]
    .mean()
    .rename_axis('class')
    .reset_index()
    .to_string(index=False)
)

print("\nUnit-selection gallery (listen to these first):")
unit_gallery_paths = []
for label_name in LABEL_NAMES:
    class_rows = unit_eval_df[unit_eval_df['intended_class'] == label_name].
 ↪copy()
    correct_rows = class_rows[class_rows['predicted_class'] == label_name]
    if len(correct_rows):
        best_row = correct_rows.sort_values('pred_confidence', ascending=False).
 ↪iloc[0]
    else:
        best_row = class_rows.sort_values('pred_confidence', ascending=False).
 ↪iloc[0]
    rec = next(r for r in unit_generated_records if r['sample_id'] ==␣
 ↪best_row['sample_id'])
    unit_gallery_paths.append((label_name, rec['local_path'], rec['repo_path'],␣
 ↪rec['public_url']))

for label_name, local_path, repo_path, public_url in unit_gallery_paths:
    print(f"\n{label_name}")
```

```
    print(f"Repository path: {repo_path}")
    print(f"Public URL: {public_url}")
    display(Audio(local_path, rate=UNIT_SR))
```

```
Unit-selection classifier confusion table:
Predicted     Mom  Bestfriend  Professional
Intended
Mom             0           1             0
Bestfriend      0           1             0
Professional    0           0             1

Unit-selection context accuracy: 66.7%

Unit-selection acoustic summary:
       class  rms_mean  silence_ratio  spectral_centroid  pitch_mean
unique_source_notes
  Bestfriend  0.121490       0.133508        1475.966992  124.727942
4.0
         Mom  0.182234       0.013089        1025.844708  119.609425
4.0
Professional  0.110703       0.086387        1204.950415  129.971268
3.0

Unit-selection gallery (listen to these first):

Mom
Repository path: assignment3_assets/audio/unit_generated_mom_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_mom_00.wav

<IPython.lib.display.Audio object>


Bestfriend
Repository path: assignment3_assets/audio/unit_generated_bestfriend_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_bestfriend_00.wav

<IPython.lib.display.Audio object>


Professional
Repository path: assignment3_assets/audio/unit_generated_professional_00.wav
Public URL: https://raw.githubusercontent.com/Shazil10/cs156-audio-code-
switching/main/assignment3_assets/audio/unit_generated_professional_00.wav

<IPython.lib.display.Audio object>
```

The unit-selection clips should be interpreted as the clarity baseline, not as the main neural model. They are generated from short training snippets, so they preserve real waveform detail instead of

asking Griffin-Lim to reconstruct phase from a neural spectrogram. That is why they sound much more like me than the neural clips.

The tradeoff is that unit selection sounds jumpy. I can hear boundaries because the system is literally moving between recorded units from different moments in my training voice notes. The output is new in the sense that it chooses and sequences snippets using acoustic matching, randomness, class labels, and crossfades; it is not new in the sense of generating waveform detail from scratch.

That makes the comparison useful. The VAE/LSTM and autoencoder/Transformer branches test whether a learned latent model can invent new speech-like spectrograms from my archive. Unit selection tests how far I can get by recombining real units intelligently. On this dataset, unit selection wins on listenability and voice identity, while the neural models are more ambitious but much blurrier.

5.7 Final Combined Discussion

The classifier and the generative models answer related but different questions. The classifier asks whether my voice alone carries enough signal to tell Mom, Bestfriend, and Professional apart. The answer there is solid: the augmented CNN sits at roughly 90% test accuracy with a macro F1 around 0.90 on a strict note-level group split, so the classification part of the project is strong enough to act as the evaluation tool for the generation part.

The first neural generation branch answers a harder question. Can a conditional beta-VAE plus a latent LSTM prior, trained on roughly 32 voice notes from one speaker, produce short clips that still sound like that speaker and still keep some of the context signal. The answer is only partly yes. The code runs end to end, the reconstructions follow the rough timing of the held-out windows, and the generated clips are not silence. But the spectrograms show the limitation clearly: harmonic speech structure gets blurred into broad bands, and the audio sounds smeared. I keep those three blurry clips in the notebook because they are the honest negative result from the professor's suggested direction.

I then try to improve the neural branch with a deterministic conditional autoencoder plus a Transformer latent prior. This keeps the encoder-decoder idea but removes the beta-VAE KL penalty that was smoothing away detail, and it replaces the LSTM with attention over recent latent windows. This is still machine-learning generation: the model predicts future learned latent vectors and decodes them back into spectrograms. It is a smarter architecture for the failure I observed, even if the final audio is still limited by small data and by Griffin-Lim phase reconstruction.

Finally, I add unit-selection synthesis as a non-neural baseline. Instead of asking a small decoder to hallucinate waveform detail, unit selection recombines short real snippets from my training voice notes using acoustic matching and equal-power crossfades. This is not the same kind of voice generation as the neural branches; it is clearer precisely because it reuses real waveform pieces. The inherited classifier still hears the stitched clips mostly as Mom, so I do not claim that social-context control is solved. But the method gives me a clean comparison: learned neural generation is more ambitious and blurrier, while retrieval-based concatenative synthesis is less magical but much more listenable on a small archive.

The combined story is stronger with all three pieces. My archive supports very strong context classification. It also supports a hard neural generation attempt, a better neural improvement attempt, and a clearer non-neural synthesis baseline. The code-switching hypothesis is clearly detectable when I classify real held-out audio. For generation, the honest conclusion is that small-

data speech synthesis exposes a tradeoff between novelty and fidelity: the more the model has to invent, the blurrier it gets; the more it borrows from real snippets, the better it sounds but the less it can claim to have learned waveform production from scratch.

6 Executive Summary

This notebook asks two questions about my own WhatsApp voice notes. First, does my voice actually change across social contexts in a way a machine can hear, with no transcripts allowed. Second, can a generative pipeline take the same archive and produce short audio that still carries some of that context signal. Parts 1 and 2 build the data and classification pipeline. Part 3 extends the same archive into audio generation.

6.0.1 What the classifier shows

After converting 42 WhatsApp voice notes to WAV, slicing them into 166 ten-second clips, and extracting 54-dimensional MFCC-based features plus parallel MFCC sequences for the CNN, I compare softmax regression, a 1D CNN, and XGBoost on a note-level group split. Each model is trained twice, once on originals only and once with four synthetic copies per clip. The best result is the augmented CNN, which reaches about 90% test accuracy and a macro F1 around 0.90. This is the strongest empirical result in the notebook: my Mom, Bestfriend, and Professional voice-note styles are measurably different even without transcripts.

6.0.2 What Part 3 adds

In Part 3 I build an audio-generation extension. The professor suggested trying audio generation with a VAE or teacher-forcing autoencoder feeding an LSTM, so I start there: a conditional beta-VAE compresses 2-second log-mel windows into a latent vector, and a latent LSTM tries to roll those vectors forward. I keep the result even though it is blurry, because it is the real machine-learning generation attempt and it teaches something important about the dataset.

Then I add a neural improvement attempt: a deterministic conditional autoencoder plus a Transformer latent prior. This removes the KL smoothing pressure from the VAE and gives the prior attention over recent latent windows instead of only an LSTM hidden state. Finally, I add unit-selection synthesis as a non-neural baseline. Unit selection produces clearer clips by recombining real snippets from my own training notes, but it does not generate waveform detail from scratch.

6.0.3 What actually comes out

The beta-VAE + LSTM branch technically works but produces blurry audio, and the notebook now displays three representative blurry clips for Mom, Bestfriend, and Professional. The autoencoder + Transformer branch is a targeted attempt to improve that neural result by preserving sharper latent detail and using attention for the sequence model. The unit-selection branch produces the clearest clips because it stitches real waveform units together with crossfades; the jumps I hear are a known tradeoff of concatenative synthesis, not a mystery bug.

The final takeaway is not “I built perfect speech synthesis.” The stronger claim is more careful: my archive supports strong context classification, neural generation is possible but blurry on this amount of data, and unit-selection shows why retrieval-based speech synthesis can beat a small neural generator on fidelity. That comparison is useful because it explains the practical limit I hit instead of hiding it.

6.0.4 Shortcomings and next steps

The main limitation is data scale. Thirty-ish training voice notes is enough for classification, but it is thin for neural speech generation. A stronger next version would use more recordings, a pretrained neural vocoder instead of Griffin-Lim, and perhaps a diffusion or transformer-based acoustic model pretrained on public speech before being adapted to my own voice notes.

7 References

1. McFee, B., Raffel, C., Liang, D., et al. (2015). librosa: Audio and Music Signal Analysis in Python. *Proceedings of the 14th Python in Science Conference*, 18-24.
2. de Cheveigné, A., & Kawahara, H. (2002). YIN, a fundamental frequency estimator for speech and music. *Journal of the Acoustical Society of America*, 111(4), 1917-1930.
3. Davis, S., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), 357-366.
4. ITU-T Recommendation G.722 (2012). 7 kHz audio-coding within 64 kbit/s. International Telecommunication Union.
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 4.
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 9.
7. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
8. Abadi, M., Barham, P., Chen, J., et al. (2016). TensorFlow: A System for Large-Scale Machine Learning. *Proceedings of OSDI*, 265-283.
9. Gumperz, J. J. (1982). *Discourse Strategies*. Cambridge University Press.
10. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
11. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD*, 785-794.
12. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *NeurIPS*, 30.
13. Park, D. S., Chan, W., Zhang, Y., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *Interspeech 2019*, 2613-2617.
14. Mauch, M., & Dixon, S. (2014). pYIN: A Fundamental Frequency Estimator Using Probabilistic Threshold Distributions. *ICASSP*, 659-663.
15. Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. *ICLR*.
16. Higgins, I., Matthey, L., Pal, A., et al. (2017). beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. *ICLR*.
17. Williams, R. J., & Zipser, D. (1989). A Learning Algorithm for Continually Running Fully Recurrent Neural Networks. *Neural Computation*, 1(2), 270-280.

18. Griffin, D., & Lim, J. (1984). Signal Estimation from Modified Short-Time Fourier Transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2), 236-243.

19. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention Is All You Need. *NeurIPS*.

20. Hunt, A. J., & Black, A. W. (1996). Unit selection in a concatenative speech synthesis system using a large speech database. *ICASSP*, 373-376.